Course: Python

Ali ZAINOUL <ali.zainoul.az@gmail.com>

CrystalClearCode
November 6, 2025



1 Introduction
- Historique
- Caractéristiques du langage
- Versions de Python
- Installation de Python
  - Installation de Python sur Windows
  - Installation de Python sur Linux
  - Installation de Python sur MacOS
- Panorama de la bibliothèque standard
- Modules d'extension et commande pip
- Principe de fonctionnement de l'interpréteur
- Interpréteur officiel CPython et autres interpréteurs
- Interpréteur interactif
- Ressources
- Fonction help() et chaînes documentaires
  - Fonction help() et attribut __doc__
  - Chaînes documentaires (docstrings)

- Principe de l'indentation pour délimiter les blocs d'instruction
- Commentaires
- Mots-clés réservés
- Conventions de nommage
- Programme autonome
- Fonctions intégrées élémentaires
  - Fonction print() - Affichage à l'écran
  - Fonction input() - Saisie utilisateur
  - Fonctions type() et len()

2 Types fondamentaux

- Booléen
  - Objets True et False
- Nombres
  - Les entiers relatifs (int)
  - Les réels (float)
  - Les complexes
- Les chaînes de caractères unicode (str)
  - Manipulation des Chaînes de Caractères
  - Définition avec simple et double guillemets

- Chaînes multilignes avec triple simple ou double guillemets
- Mode raw
- Constructeur
- Indiçage positif et négatif
- Tranche de valeurs (slice)
- Opérateurs +, * et in
- Itération
- Méthodes incontournables de `str`
- L'affichage formaté de variables

3 Les Opérateurs
- Opérateurs Arithmétiques
- Opérateurs Relationnels
- Opérateurs Bit à Bit
- Opérateurs d'Assignation
- Opérateur Ternaire
- Autres opérateurs
- Priorité des opérateurs en Python
- Opérateurs et types fondamentaux
  - Opérateurs et les booléens

- Opérateurs et les entiers
- Opérateurs et les réels
- Opérateurs et les complexes

4 Notations exponentielle, binaire, octale et hexadécimale
- Systèmes de notation et notion de base N
- Fonctions `hex()`, `oct()`, `bin()`, `chr()`, `ord()`

5 Structures conditionnelles et répétitives
- Les structures conditionnelles `if/elif/else`
- Les boucles d'itérations while et for
- Instructions break et continue
- Constructeur enumerate()
- Bloc else dans une structure répétitive

6 Les Fonctions
- Définition et appel d'une fonction
- Nombre variable de paramètres dans une fonction
- Fonctions avec des paramètres clé/valeur
- Les générateurs - Instruction yield

- Notion de scope (Espace de noms) - portée des variables
- Scopes pré-définis (__builtins__)
- Fonction dir()
- Retourner des valeurs, instruction return
- Fonctions génériques (duck typing)
- Valeurs par défaut
- Passage par étiquette
- Nombre d'arguments arbitraire (*args, **kwargs)
- Fonctions anonymes (lambda)
- Fonctions eval(), exec()
- Constructeurs map() et filter()

7 Types de données non-modifiables
- Utilité des types non-modifiables
  - Optimisation mémoire
  - Fonctions `id()` et `hash()`
  - Opérateur `is`
- Les chaînes de caractères unicode (str)
  - Manipulation des Chaînes de Caractères

- Définition avec simple et double guillemets
- Chaînes multilignes avec triple simple ou double guillemets
- Mode raw
- Constructeur
- Indiçage positif et négatif
- Tranche de valeurs (slice)
- Opérateurs +, * et in
- Itération
- Méthodes incontournables de `str`
- L'affichage formaté de variables

Tuple

- Constructeur
- Indiçage
- Itération
- Opérateurs +, * et in
- Méthodes `count()` et `index()`
- Principe du depacking de variables

Frozenset

Tableau d'octets (bytes)

Objet None et fonction `repr()`

8 Types de données modifiables

- Listes (list)
  - Constructeur
  - Indiçage
  - Itération
  - Opérateurs +, * et in
  - Méthodes `append()`, `insert()`, fonction `del()`, `sort()`, `reverse()`, `remove()`, `extend()`, `pop()`, `clear()`
  - Copie et références
- Fonction `sorted()`
- Principe de fonctionnement des objets itérables
- Constructeurs `reversed()` et `range()`
  - Constructeur reversed()
  - L'objet range
- Collections: Dictionnaires (dict)
  - Constructeur
  - Indiçage
  - Opérateur `in`, fonction `del()`
  - Méthodes `keys()`, `values()`, `items()`, `update()`, `get()`
  - Copie et références

- Collections: Classe Set
  - Constructeur
  - Opérateurs - |, &, et ^

9 Notion d'intension (Comprehension) List - Ensemble - Dict
- Liste en intension (comprehension list)
- Ensemble en intension (comprehension set)
- Dictionnaire en intension (comprehension dict)
- Tuple en intension (comprehension tuple)

10 Résumé

11 Modules de la bibliothèque standard
- Interaction avec l'interpréteur : module `sys`
- Interaction avec le système d'exploitation : modules `os` et `pathlib`
- Interaction avec le système de fichiers : module `os.path`
- Expressions rationnelles : module `re`
- Tour d'horizon d'autres modules intéressants de la bibliothèque standard :
  - Module `datetime`
  - Module `math`

- Module `timeit`
- Module `urllib`
- Module `collections`
- Module `csv`
- Module `json`
- Module `sqlite3`

12 Exceptions
- Principe de fonctionnement
- Exceptions pré-définies et arbre d'héritage
- Instructions `try ... except ... else ... finally`
- Propagation des exceptions
- Déclenchement d'exceptions
- Définition d'une exception

13 Manipulation des fichiers
- Fonction `open()` et méthode `close()`
- Méthodes `readline()` et `readlines()` Objet itérable
- Instruction `with` avec les fichiers
- Méthodes `read()` et `write()`

- Méthodes `tell()` et `seek()`
- Méthode `writelines()`
- Modules complémentaires : `struct`, `csv`, `json`, `xml`
  - Module `struct`
  - Module `csv`
  - Module `json`
  - Module `xml`
- Sérialisation avec le module `pickle`
- Sérialisation avec le module `shelve`

14 Les modules et les paquets
- Les Modules en Python
  - Bloc `if __name__ == "__main__"`
  - Les bonnes pratiques
- Paquets
  - Importation de paquet
  - Création d'un paquet
  - Création d'un paquet (__init__.py)

15 Tests unitaires : instruction `assert`, module `unittest`
- Framework `pytest`

16. Les expressions (RegEx)
17. Passer des paramètres à un script avec argparse
18. Exécuter des commandes système
19. Programmation Orientée Objet
    - Concepts fondamentaux de la POO
    - Jargon de la POO
    - Définition d'une classe d'objet
    - Définition d'un objet (état, comportement, identité)
    - Notions de classe, d'objet, d'attribut et de méthode
      - Notion de classe
      - Notion d'attribut
      - Notion de méthode
    - Notion de spécificateurs d'accès
      - Spécificateurs d'accès en Python
      - Résumé Access Specifiers
      - Notion de Name Mangling en Python
    - Encapsulation des données
    - Instanciation d'objets, fonction `isinstance()`

- Constructeur (`__init__`)
- Attributs et méthodes
- Le paramètre `self`
- Surcharge d'affichage (`__str__`)
- Surcharge d'opérateurs
- Propriété, accesseur et mutateur, et décorateur
  - Propriété, accesseur et mutateur, et décorateur
  - Décorateur property
- Espaces de noms global, de l'objet, de la classe
- Variable de classe
- Constructeur à nombre d'arguments arbitraire (`*args`, `**kwargs`)
- Relations UML et diagrammes de classes
  - Relations UML
  - Diagrammes de classes
  - Association
  - Agrégation
  - Composition
  - Dépendance
  - Héritage

- Réalisation / Implémentation
- Différences entre les relations
- Flèches et symboles UML des relations

- Notion d'Héritage de classe (généralisation)
  - Princice de l'héritage
  - Types d'héritage
  - Problème du diamant
  - Fonction `issubclass()`, constructeur `super()` et méthode `mro()`
  - Le constructeur super()
  - La méthode `mro()` et l'attribut `__mro__`
- Notion d'Abstraction
  - Classes abstraites
  - Les Interfaces
- Notion de polymorphisme
  - Compile-time polymorphism
  - Runtime polymorphism
- Notion d'encapsulation
  - Définition de l'encapsulation
  - Lien avec les autres principes
  - Métaphore de la capsule

- Une vision modulaire du système

** Introduction **

Historique

- **Auteur :** Guido VAN ROSSUM, informaticien néerlandais. Il a travaillé sur Python chez CWI aux Pays-Bas.
- **Origine :** Influencé par le langage ABC, VAN ROSSUM a créé Python pour sa :
  - Simplicité,
  - Productivité,
  - Syntaxe claire, aboutissant à un langage convivial, puissant et polyvalent.
- **Date de la première version :** Février 1991

Python ?!

- **Python en bref :**
  - Langage de programmation puissant ;
  - Facile à utiliser et à apprendre ;
  - Open source ;
  - Langage à typage dynamique fort.

- **Avantages de Python :**
  - Adapté aux débutants ;
  - Utilisé à grande échelle dans une variété de projets industriels.



Figure: Python logo

Ecosystème



Figure: Ecosystem Python

Caractéristiques

Python est un langage de programmation qui est :

- Muni d'une syntaxe claire ;
- Interprété ;
- Multi-paradigmes et Multi-Plateformes ;
- qui favorise la :
  - Programmation impérative structurée ;
  - Programmation fonctionnelle ;
  - Programmation Orientée Objet.

Version Python 2.x

- Python 2.x est une version héritée.
- Elle n'est plus maintenue depuis janvier 2020.
- Il est fortement recommandé de migrer vers Python 3.

Version Python 3.x

- Python 3.x reste la version de référence, en amélioration continue.
- Python 3.13 introduit le **GIL optionnel**, ouvrant la voie au vrai parallélisme.
- Performances accrues, typage renforcé et f-strings plus puissants.
- Compatibilité accrue avec les outils modernes (`pyproject.toml`, `ruff`, `mypy`).
- Nouveautés des dernières versions.

Outils nécessaires

- Environnement de développement intégré (IDE)
  Choix entre : PyCharm, Microsoft Visual Studio Code, Sublime Text...



Figure: Meilleur IDE

- Éditeur de texte & interprétation / exécution *via* le terminal

Installation

Installation de Python sur Windows — Étape 1

- Rendez-vous sur le site officiel de Python et téléchargez la dernière version (ex. Python 3.13.x).
- Ouvrez le fichier téléchargé (`python-3.13.x-amd64.exe`).
- Dans la fenêtre d'installation :
  - **Cochez** l'option `Add Python to PATH` avant de cliquer sur `Install Now`.
  - Cette option ajoute automatiquement le chemin de Python dans les variables d'environnement Windows.
- Attendez la fin du processus puis cliquez sur `Close`.

Installation de Python sur Windows — Étape 2

- Si vous avez oublié de cocher l'option `Add Python to PATH` :
- Ouvrez le menu Démarrer → tapez `variables d'environnement`.
- Cliquez sur `Modifier les variables d'environnement du système`.
- Sélectionnez la variable `Path` → `Modifier`.
- Cliquez sur `Nouveau` et ajoutez les chemins suivants :

```
C:\Users\<user_name>\AppData\Local\Programs\Python\Python313\
C:\Users\<user_name>\AppData\Local\Programs\Python\Python313\Scripts\
```

- Ouvrez l'invite de commandes et tapez :

`python --version`	→ affiche la version installée.
`pip --version`	→ vérifie l'accès au gestionnaire de paquets.
`python`	→ lance l'interpréteur Python.

Installation de Python sur Ubuntu

- Mettez à jour les dépôts :

  ```
  sudo apt update
  ```

- Installez Python et `pip` :

  ```
  sudo apt install python3 python3-pip -y
  ```

- Vérifiez le `PATH` :

  ```
  echo $PATH
  ```

  Si `/usr/bin` ou `/usr/local/bin` apparaissent, tout est correct.
  Sinon, ajoutez Python au `PATH` :

  ```
  echo "export PATH=$PATH:/usr/local/bin/python3" >> ~/.bashrc
  source ~/.bashrc
  ```

- Ouvrez le terminal et tapez :

  `python --version` → affiche la version installée.
  `pip --version` → vérifie l'accès au gestionnaire de paquets.
  `python` → lance l'interpréteur Python.

Installation de Python sur MacOS
Utilisation de Homebrew

- Ouvrez le terminal et saisissez la commande suivante :

```
xcode-select --install
```

- Suivez les instructions à l'écran pour installer les outils de ligne de commande Xcode.
- Ensuite, saisissez la commande suivante dans le terminal :

```
brew install python3
```

- Attendez que l'installation soit terminée.
- Ouvrez le terminal et tapez :

`python --version`	→ affiche la version installée.
`pip --version`	→ vérifie l'accès au gestionnaire de paquets.
`python`	→ lance l'interpréteur Python.

Installation de Python - Récapitulatif

Table: Python and pip commands by OS and installation method

OS	Installation	Python	pip	Notes rapides
Windows	Installation manuelle du site officiel	`python`	`pip`	*Python 3 global installé ; venv/virtualenv disponibles*
Windows	Windows Store	`python`	`pip`	*Python global via Microsoft Store ; venv/virtualenv disponibles*
Windows	pyenv	`python`	`pip`	*Redirige vers la version sélectionnée (shim) ; alias `python3` possible*
Windows	uv (Astral)	`python3.x`	`pip3`	*Peut installer Python si absent ; exécutable versionné*
Windows	Conda / Miniconda	`python`	`pip`	*Utilise l'environnement Conda actif ; venv/virtualenv possibles*
macOS	Installation manuelle du site officiel	`python3`	`pip3`	*Python 3 global installé ; venv/virtualenv disponibles*
macOS	App Store	`python3`	`pip3`	*Installation via App Store*
macOS	pyenv	`python`	`pip`	*Redirige vers la version sélectionnée (shim) ; alias `python3` possible*
macOS	uv	`python3.x`	`pip3`	*Peut installer Python si absent ; exécutable versionné*
macOS	Conda / Miniconda	`python`	`pip`	*Utilise l'environnement Conda actif ; venv/virtualenv possibles*
macOS	Homebrew	`python3`	`pip3`	*Evite conflit avec Python système ; venv/virtualenv disponibles*
Ubuntu	Installation manuelle du site officiel	`python3`	`pip3`	*Python 3 global installé ; venv/virtualenv disponibles*
Ubuntu	Ubuntu Store	`python3`	`pip3`	*Installation via Ubuntu Store*
Ubuntu	pyenv	`python`	`pip`	*Redirige vers la version sélectionnée (shim) ; alias `python3` possible*
Ubuntu	uv	`python3.x`	`pip3`	*Peut installer Python si absent ; exécutable versionné*
Ubuntu	Conda / Miniconda	`python`	`pip`	*Utilise l'environnement Conda actif ; venv/virtualenv possibles*

La bibliothèque standard de Python

Panorama de la bibliothèque standard

- La **bibliothèque standard de Python** est une collection de `modules` et de `packages` étendus couvrant divers domaines, tels que l'accès aux fichiers, le traitement de chaînes, la manipulation de données, les communications réseau et bien d'autres encore.
- Ces modules sont inclus dans une installation standard de Python et peuvent être utilisés pour développer une grande variété d'applications sans avoir à installer des bibliothèques tierces.
- Pour plus d'informations sur les modules disponibles dans la bibliothèque standard de Python, consultez la Documentation bibliothèque standard.

Éléments les plus utilisés

- Built-in Functions
- Built-in Constants
- Built-in Types
- Built-in Exceptions
- Text Processing Services
- Data Types
- Numeric and Mathematical Modules
- Functional Programming Modules
- File and Directory Access
- Data Compression and Archiving
- Generic Operating System Services
- Development Tools
- Debugging and Profiling
- Software Packaging and Distribution
- Python Runtime Services
- Importing Modules
- Concurrent Execution
- Networking and Interprocess Communication
- Internet Data Handling
- Structured Markup Processing Tools
- Internet Protocols and Support
- Graphical User Interfaces with Tk

pip

Modules d’extension et commande pip - Introduction

- Les **modules d’extension** sont des **bibliothèques externes** que vous pouvez ajouter à votre environnement Python pour étendre ses fonctionnalités.

Modules d'extension et commande pip - Utilisation de pip

- La commande `pip` est l'outil recommandé pour installer et gérer ces modules d'extension. Elle vous permet de rechercher, télécharger et installer des packages Python à partir du Python Package Index (PyPI).
- Utilisez la commande `pip install` pour installer un module spécifique. Par exemple, pour installer le module `requests`, vous pouvez exécuter : `pip install requests`

L'interpréteur Python

Principe de fonctionnement de l'interpréteur - Introduction

- L'**interpréteur Python** est un programme qui **lit et exécute le code source Python**. Il **traduit le code source** en **instructions exécutables** que l'ordinateur peut **comprendre et exécuter**.

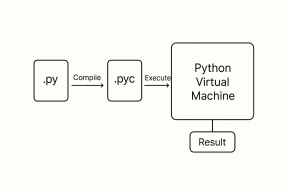


Figure: Principe de fonctionnement de l'interpréteur Python

Principe de fonctionnement de l'interpréteur - Bytecode PYC

- Le **bytecode PYC** est un code intermédiaire généré par l'interpréteur Python lors de la compilation du code source. Il est stocké dans des fichiers avec l'extension `.pyc` et est utilisé pour accélérer le chargement et l'exécution du code Python en évitant la recompilation du code source à chaque exécution.

Interpréteur officiel CPython

- **CPython :** L'interpréteur officiel de Python, écrit en C, est largement utilisé et prend en charge les extensions C. Il est principalement utilisé pour exécuter des applications Python standard.

Autres interpréteurs

- **Micropython :** Une implémentation de Python destinée aux microcontrôleurs et aux systèmes embarqués, offrant une empreinte mémoire réduite et un support pour les plateformes à ressources limitées.
- **Brython :** Un projet visant à implémenter un interpréteur Python dans le navigateur Web, permettant l'exécution de code Python côté client.
- **Pypy :** Une implémentation alternative de Python, écrite en Python, visant à améliorer les performances par rapport à CPython en utilisant la compilation JIT.
- **Numba :** Un compilateur JIT pour Python qui vise à accélérer l'exécution de code numérique en générant du code machine optimisé pour les calculs scientifiques et numériques.

Interpréteur interactif

- Voici un exemple d'utilisation de l'interpréteur interactif:

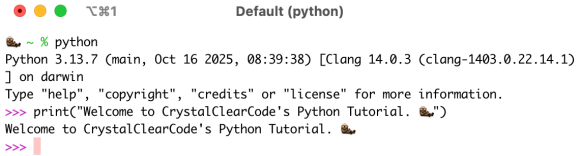


Figure: Premier programme en Python

Ressources - Site Officiel python.org

- Site Internet python.org : La page officielle de Python.org est une ressource précieuse pour les utilisateurs de Python. Elle fournit des informations sur les dernières nouvelles, les événements, les ressources d'apprentissage et bien plus encore.
- Documentation officielle de Python : La documentation officielle de Python est une ressource essentielle pour tout développeur Python. Elle contient des informations détaillées sur la syntaxe du langage, la bibliothèque standard, les modules et bien plus encore.

Ressources - Autres ressources pour apprendre Python

- Real Python : Real Python propose des tutoriels, des articles et des cours pour les programmeurs de tous niveaux, du débutant à l'expert.
- Codecademy : Codecademy propose des cours interactifs en ligne pour apprendre Python, avec des exercices pratiques et des projets.
- LearnPython : LearnPython.org offre des tutoriels interactifs et des exercices pratiques pour apprendre Python à votre propre rythme.

Fonction help()

- La fonction `help()` est utilisée pour obtenir de l'aide et des informations sur les objets Python.
- Elle affiche la documentation (docstring) de l'objet passé en paramètre directement dans la console.
- Voici un exemple d'utilisation de la fonction `help()` pour obtenir de l'aide sur la fonction `print()` :

```
help(print)
```

Fonction help() - Exemple

```
~ % python
Python 3.13.7 (main, Oct 16 2025, 08:39:38) [Clang 14.0.3 (clang-1403.0.22.14.1)
] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> help(print)
```

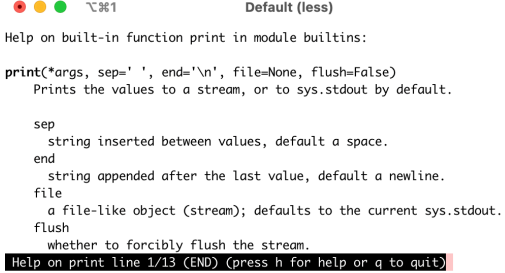


Figure: Exemple de la fonction `help()` pour `print()`

Attribut __doc__

- L'attribut `__doc__` renvoie la docstring d'un objet Python sous forme de chaîne de caractères (`str`).
- À l'instar de `help()`, il permet d'accéder à la documentation interne d'une fonction, d'une classe ou d'un module.
- Exemple :
  `print(print.__doc__)`
  Cela affichera la même documentation que `help(print)`, mais sans mise en forme.
- Cet attribut est souvent utilisé dans les tests ou la génération automatique de documentation.

Attribut __doc__ - Exemple

```
~ % python
Python 3.13.7 (main, Oct 16 2025, 08:39:38) [Clang 14.0.3 (clang-1403.0.22.14.1)
] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> help(print)

>>> print(print.__doc__)
Prints the values to a stream, or to sys.stdout by default.

  sep
    string inserted between values, default a space.
  end
    string appended after the last value, default a newline.
  file
    a file-like object (stream); defaults to the current sys.stdout.
  flush
    whether to forcibly flush the stream.
>>>
```

Figure: Exemple de l'attribut __doc__ pour print()

Chaînes documentaires (docstrings)

- Les **chaînes documentaires**, également appelées `docstrings`, sont des chaînes de caractères utilisées pour documenter les modules, les fonctions, les classes et les méthodes Python.
- Avec l'IDE Visual Studio Code, une personne peut installer l'extension autodocstring.

Chaînes documentaires (docstrings) - Suite

- Voici un exemple d'une docstring pour une fonction simple :

```python
# File / script / module : math_utils.py
def add(num_1: int, num_2: int) -> int:
    """
        Add two numbers.

        Parameters:
        num_1 (int): The first number.
        num_2 (int): The second number.

        Returns:
        int: The sum of the two numbers.
    """
    return num_1 + num_2
```

- L'accès à la documentation d'une telle fonction par exemple peut se faire grâce à l'appel suivant: `print(add.__doc__)` ou `help(add)`.

Indentation en Python

- Python utilise l'**indentation** pour définir des blocs de code.
- Cela diffère de nombreux autres langages de programmation tels que C, C++ ou Java, qui utilisent des accolades pour définir des blocs.
- L'indentation indique à Python quelles lignes de code appartiennent à chaque bloc.
- Utilisez la touche **tabulation** pour indenter votre code.
- Voici un exemple :

```
1 number = 123
2 if number > 0:
3     print("The number is postive.")
4 else:
5     print("The number is negative or equals to zero.")
```

Indentation - Exemple complet

- Voici un exemple complet. illustrant l'indentation en Python :

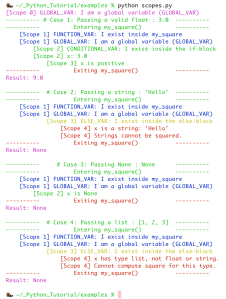


Figure: Exemple d'indentation en Python

Fin de ligne en Python

- En Python, une ligne de code se termine par un caractère de nouvelle ligne. Vous pouvez également utiliser un backslash \ pour indiquer qu'une instruction se poursuit à la ligne suivante. Voici un exemple :

```
x = 5 + \
10
print(x) # Displays : 15
```

```
total_price = item_price * quantity + \
              shipping_cost - discount

print(total_price)
```

Commentaires en Python

- Vous pouvez ajouter des commentaires à votre code Python pour expliquer ce que fait le code.
- Les commentaires commencent par un caractère dièse # et se poursuivent jusqu'à la fin de la ligne.
- Voici un exemple :

```
1 # This is a comment and will be ignored
2 print("Hello, world!")
```

- Dans cet exemple, la première ligne est un commentaire, et Python l'ignore lors de l'exécution du code.
- La deuxième ligne affiche le message `"Hello, world!"` dans la console.
- Les commentaires peuvent être utilisés pour rendre votre code plus lisible et plus facile à comprendre.

Commentaires sur plusieurs lignes

```
# This is a long comment that spans
# multiple lines in the Python code.
# It provides additional explanations or context.

def example_function():
    """
    Another way to create a multiline comment
    in Python, using triple double-quotes.
    Often used for docstrings, which provide documentation
    """
    print("Hello, World!")

# Call the example function
example_function()
```

Mots-clés réservés en Python

- Les **mots-clés réservés** en Python sont des **identificateurs** qui ont une **signification spéciale** pour l'**interpréteur Python**. Ils ne peuvent **pas** être utilisés comme noms de variables, de fonctions ou d'autres identificateurs.
- Il est important de ne pas utiliser les mots-clés réservés comme noms pour vos variables, fonctions ou classes, car cela peut entraîner des erreurs ou un comportement inattendu de votre programme.

Mots-clés réservés en Python - Liste des mots-clés

False	None	True	and	as
assert	async	await	break	class
continue	def	del	elif	else
except	finally	for	from	global
if	import	in	is	lambda
nonlocal	not	or	pass	raise
return	try	while	with	yield

Conventions de codage et style PEP 8

Conventions de nommage et bonnes pratiques

- Utilisez des noms **significatifs** pour améliorer la lisibilité.
- Commentez les sections complexes pour fournir du contexte.
- Évitez les noms d'une seule lettre sauf pour des indices.
- Indentation cohérente touche **TAB** (4 espaces).
- Suivez le guide PEP 8 : PEP 8 Coding Convention
- Plus d'informations sur les conventions de nommage : Cas Camel, Pascal, Snake, Kebab, Upper Case

Conventions de nommage: Constantes, Variables, fonctions

- Suivez le **snake_case** pour :
  - variables (sauf constantes en **UPPER_CASE** parfois appelé **SCREAMING_SNAKE_CASE**)
  - fonctions et méthodes de classe

```python
# Constants in UPPER_CASE
MAX_RETRIES = 5

# Variables in snake_case
user_name = "Alice"

# Functions in snake_case
def calculate_sum(a, b):
    return a + b
```

Conventions de nommage: Classes et méthodes

- Suivez le **snake_case** pour les méthodes.
- Seules les **classes** doivent être en **PascalCase**.

```python
# Classes in PascalCase
class Circle:
    def __init__(self, radius):
        self.radius = radius
    def print_info(self):
        print(f"Circle with radius: {self.radius}")
```

Conventions de nommage: Modules et packages

- Les noms de **modules** et **packages** doivent être en **snake_case**.
- Exemple : structure de fichier Python et package

```
# Module / package in snake_case
# math_utils.py (module)
# utils/geometry/ (package)
```

Conventions de nommage: Exemple Complet

```python
# utils/geometry/ (package)
# math_utils.py (module)

CUSTOM_PI = 3.14159

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return CUSTOM_PI * (self.radius ** 2)

def print_area(circle):
    print(f"Area of the circle: {circle.area()}")

def main():
    c = Circle(CUSTOM_PI)
    print_area(c)

if __name__ == "__main__":
    main()
```

Notion de Programme autonome

- Un **programme autonome**, également connu sous le nom de programme indépendant, est un **programme informatique** qui peut fonctionner de **manière autonome**, sans nécessiter l'exécution dans un environnement spécifique ou l'interaction avec d'autres programmes.
- Ces programmes sont conçus pour être **auto-suffisants** et peuvent être exécutés **indépendamment du système d'exploitation** ou des autres logiciels installés sur l'ordinateur.

Exemple d'un programme autonome

Listing 1: Python code example

```python
# Define a simple function
def add(num_1: int, num_2: int) -> int:
    """
    Add two numbers.
    Parameters:
    num_1 (int): The first number.
    num_2 (int): The second number.
    Returns:
    int: The sum of the two numbers.
    """
    return num_1 + num_2

# Call the function and print the result
result = add(3, 5)
print("The result of addition is:", result) # Output : The result of addition is: 8
```

- Exécution: `python add.py`

Fonction print()

```python
# Print a simple message
print("Hello, world!")

# Print multiple values
x = 10
y = 20
print("The value of x is:", x, "and the value of y is:", y)
print(f"The value of x is: {x} and the value of y is: {y}") # f-string equivalent

# Print with formatting
name = "Alice"
age = 30
print("Hello and welcome", name, ",you are", age, "years old.") # Unwanted space after variable Alice
print("Hello and welcome " + name + ", you are " + str(age) + " years old.") # Concatenation equivalent
print("Hello and welcome {}, you are {} years old.".format(name, age)) # str.format() equivalent
print(f"Hello and welcome {name}, you are {age} years old.") # f-string equivalent
```

Fonction input()

```python
# Take input from the user
name = input("Enter your name: ")
print(f"Hello {name} and welcome to Python Tutorial !")
age = input("Enter your age: ")
print(f"You are {int(age)} years old.")

# Take multiple inputs separated by space simultaneously from the user
name, age = input("Enter your name and age separated by a space: ").split()
print(f"Hello {name}, you are {int(age)} years old.")
```

Fonction type()

- La fonction `type()` est utilisée pour déterminer le type d'une variable ou d'un objet en Python.
- Elle renvoie le type de l'objet passé en paramètre.
- L'attribut `__name__` peut être utilisé pour obtenir le nom du type sous forme de chaîne de caractères.
- Voici un exemple :

```
# Determine the type of a variable
a_number : int = 10
a_string : str = "Hello"
print(type(a_number))  # Output: <class 'int'>
print(type(a_number).__name__)  # Output: int
print(type(a_string))  # Output: <class 'str'>
print(type(a_string).__name__)  # Output: str
```

Fonction len()

- La fonction `len()` est utilisée pour déterminer la longueur (le nombre d'éléments) d'un objet en Python.
- Elle peut être utilisée avec différents types d'objets, tels que les chaînes de caractères, les listes, les tuples, les dictionnaires, etc.
- Voici un exemple :

```
# Determine the length of a list
my_list = [1, 2, 3, 4, 5]
print("Length of the list:", len(my_list))  # Output: 5

# Determine the length of a string
my_string = "Hello, world!"
print("Length of the string:", len(my_string))  # Output: 13
```

** Les fondamentaux **

Les types fondamentaux et les variables

Les booléens (Booleans)

- Les **booléens** peuvent prendre uniquement deux valeurs:
  - soit **Vrai** (**True**) ;
  - soit **Faux** (**False**).
- Ils sont souvent utilisés dans les instructions conditionnelles et les boucles.

Les booléens (Booleans) - Exemple

```python
# Example of manipulating booleans in Python
bigger : int = 10
smaller : int = 5
comparison_boolean : bool = (bigger>=smaller)
print(f"The result of comparing ({bigger}>={smaller}) is: {comparison_boolean}")
# Displays : The result of comparing (10 > 5) is: True

if comparison_boolean:
    if bigger > smaller :
        print(f"{bigger} is indeed greater than {smaller}.")
    else:
        print(f"{bigger} is equal to {smaller}.")
else:
    print(f"{smaller} is greater than or equal to {bigger}.")
```

Les entiers (Integers)

- Les **entiers** sont des nombres entiers tels que -3, -2, -1, 0, 1, 2, 3, etc. Vous pouvez effectuer des opérations arithmétiques de base sur les entiers, telles que:
  - l'addition `+` ;
  - la soustraction `-` ;
  - la multiplication `*` ;
  - la division `/` ;
  - la division entière `//` ;
  - le reste modulo `%` ;
- Dans l'exemple qui suit, on rappelle que :

$$7 = (3 \times 2) + 1$$

Où: 7 étant le **dividende**, 3 le **diviseur**, 2 le **quotient** et 1 le **reste** de la **division euclidienne** de 7 par 3.

Les entiers (Integers) - Exemple

```python
# Example of basic arithmetic operations in Python

# Definition of two variables x and y
x : int = 7
y : int = 3
result_addition = x + y
print(f"Addition: {x} + {y} = {result_addition}")
# Displays: Addition: 7 + 3 = 10

# Subtraction
result_subtraction = x - y
print(f"Subtraction: {x} - {y} = {result_subtraction}")
# Displays: Subtraction: 7 - 3 = 4
```

```python
# Multiplication
result_multiplication = x * y
print(f"Multiplication: {x} * {y} = {result_multiplication}")
# Displays: Multiplication: 7 * 3 = 21

# Division
result_division = x / y
print(f"Division: {x} / {y} = {result_division}")
# Displays: Division: 7 / 3 = 2.3333333333333335
```

```python
# Entire Division (Euclidian Division)
result_entire_division2 = x // 2
print(f"Entire Division: {x} // {2} = {result_entire_division2}")
# Displays: Entire Division: 7 // 2 = 3

# Entire Division (Euclidian Division)
result_entire_division = x // y
print(f"Entire Division: {x} // {y} = {result_entire_division}")
# Displays: Entire Division: 7 // 3 = 2

# Modulo
result_modulo = x % y
print(f"Modulo: {x} % {y} = {result_modulo}")
# Displays: Modulo: 7 % 3 = 1
```

Constructeurs pour les Entiers (Integers)

- Les entiers peuvent être créés en utilisant le constructeur `int()`.
- Il peut prendre divers arguments comme un autre entier, une chaîne de caractères contenant un nombre ou même un flottant.
- Exemple :

```
1 x : int = int(5)
2 y : int = int("10")
3 z : int = int(3.14)
```

- Documentation officielle : int

Les flottants (Floats)

- Les **flottants** sont des nombres décimaux (nombres à virgule), et qui ont un nombre fini de chiffres après la virgule, tels que: 3.14; 2.0 et -1.2345.
- Vous pouvez également effectuer des opérations arithmétiques de base sur les flottants.

Les flottants (Floats) - Exemple

```python
# Example of manipulating floats in Python

x : float = 3.14
y : float = 2
z : float = 3.0
t : float = -0.5
exp : float = 4.569e2  # Scientific notation for 456.9 = 4.569 * 10^2

result_product = x * y
result_addition = result_product + z
print(f"Result of: (({x} * {y}) + {z}) = {result_addition}")
# Displays: Result of: ((3.14 * 2) + 3.0) = 9.280000000000001
```

Constructeurs pour les Flottants (Floats)

- Les flottants peuvent être créés en utilisant le constructeur `float()`.
- Il peut prendre divers arguments comme un entier, une chaîne de caractères contenant un nombre ou même un autre flottant.
- Exemple :

```
1 x : float = float(5)
2 y : float = float("10.5")
3 z : float = float(3.14)
```

- Documentation officielle : float

Les nombres complexes (Complex Numbers)

- Les **nombres complexes** sont des nombres ayant à la fois une partie réelle et une partie imaginaire. Ils sont souvent utilisés en mathématiques et en physique pour représenter des quantités oscillantes telles que les ondes.
- En Python, les nombres complexes sont créés en utilisant la lettre `j` ou `J` pour représenter la partie imaginaire.
- Exemples de nombres complexes: $3 + 2j$, $-1.5 + 0.5j$, $2J$.

Les nombres complexes (Complex Numbers) - Exemple

```python
# Example of manipulation of complex numbers in Python

import cmath
z1 : complex = 3 + 2j
z2 : complex = -1.5 + 0.5j

result_sum : complex = z1 + z2
result_product : complex = z1 * z2

print(f"Result of addition: {z1} + {z2} = {result_sum}")
# Displays: Result of addition: (3+2j) + (-1.5+0.5j) = (1.5+2.5j)

print(f"Result of product: {z1} * {z2} = {result_product}")
# Displays: Result of product: (3+2j) * (-1.5+0.5j) = (-5.5-3.5j)
```

Constructeurs pour les Nombres Complexes (Complex Numbers)

- Les nombres complexes peuvent être créés en utilisant le constructeur `complex()`.
- Il peut prendre deux arguments, la partie réelle et imaginaire, ou un seul argument sous forme de chaîne de caractères contenant les deux parties.
- Exemple :

```
1 import cmath
2 x : complex= complex(3, 2)
3 y : complex = complex("-1.5+0.5j")
```

- Documentation officielle : complex

Manipulation des Chaînes de Caractères

- En Python, les chaînes de caractères (`str`) offrent une variété de méthodes intégrées pour effectuer des opérations sur les chaînes. Vous pouvez consulter la documentation officielle pour en savoir plus : Documentation Python - String Methods.

Manipulation des Chaînes de Caractères - Méthodes

```python
# String methods examples
message = "Hello, World!"

# String methods
string_methods = dir(message)
print("String methods:", string_methods)
```

Manipulation de Chaînes de Caractères - Exemples

```python
# Concatenation
message = "Hello, World!"
new_message = message + " Welcome to Python Tutorial!"
print("Concatenation:", new_message)

# String length
length = len(message)
print("String length:", length)

# Accessing a specific character
first_char = message[0]
print("First character:", first_char)

# Conversion to uppercase
print("Uppercase:", message.upper())
```

Définition avec guillemets simples

- Les guillemets simples (`''`) sont utilisés pour définir une chaîne de caractères.
- Exemple :

```
1 single_quote_string = 'Hello, world!'
```

Définition avec guillemets doubles

- Les guillemets doubles (`""`) sont également utilisés pour définir une chaîne de caractères.
- Exemple :

```
1 double_quote_string = "Hello, world!"
```

Définition avec guillemets simples et doubles

- Les guillemets simples (`''`) ou doubles (`" "`) peuvent être utilisés pour définir une chaîne de caractères.
- Exemple :

```
1 sentance = "He said: 'Hello, world!'"
```

Chaînes multilignes avec triple guillemets simples

- Les triple guillemets simples (`''' '''`) sont utilisés pour définir des chaînes de caractères multilignes.
- Exemple :

```
multiline_text_single_quote_string =
'''Hello,
This is a
multiline string.'''
```

Chaînes multilignes avec triple guillemets doubles

- Les triple guillemets doubles (""" """) sont également utilisés pour définir des chaînes de caractères multilignes.
- Exemple :

```
multiline_text_double_quote_string =
"""Hello,
This is another
multiline string."""
```

Le Mode Raw pour les Chaînes de Caractères

- Le mode raw est un mode spécial pour les chaînes de caractères en Python.
- En utilisant le mode raw, Python ne traite pas les séquences d'échappement spéciales dans les chaînes de caractères.
- Pour activer le mode raw, préfixez la chaîne de caractères avec un `r` (minuscule ou majuscule).
- Utile lors de la manipulation de chaînes de caractères qui contiennent de nombreux caractères d'échappement et que vous ne souhaitez pas que Python les interprète.
- Documentation officielle sur les Chaînes de Caractères

Exemple d'Utilisation du Mode Raw (1/3)

```python
# Example of using raw strings in Python

# Example of regular strings with escape sequences
# The line below will generate a SyntaxError:
# SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated \UXXXXXXXX escape
# string_1 : str = "C:\Users\username\Documents"
# print(string_1)
# To avoid the error, we can use raw strings
raw_string_1 : str = r"C:\Users\username\Documents"
print(raw_string_1)  # Output: C:\Users\username\Documents

# Example of strings with newline characters
string_2 : str = "Line 1\nLine 2\nLine 3"
print(string_2) # Output: Line 1
                #         Line 2
                #         Line 3
raw_string_2 : str = r"Line 1\nLine 2\nLine 3"
print(raw_string_2)  # Output: Line 1\nLine 2\nLine 3
```

Exemple d'Utilisation du Mode Raw (2/3)

```python
# Using raw strings with regular expressions
import re
pattern = r'\d+'
text = 'There are 10 apples and 20 oranges'
matches = re.findall(pattern, text)
print(matches)  # Output: ['10', '20']
```

Exemple d'Utilisation du Mode Raw (3/3)

```python
# Using raw strings with file paths
file_path = r"C:\Users\username\Documents\example.txt"
with open(file_path, 'r') as file:
    content = file.read()
print(content)
```

Constructeurs de la classe `str`

- La classe `str` offre plusieurs constructeurs pour créer des objets de type chaîne de caractères.
- Voici quelques-uns des constructeurs les plus couramment utilisés :
  1. Constructeur vide : Crée une chaîne de caractères vide.
  2. Constructeur avec une chaîne de caractères : Crée une copie de la chaîne de caractères spécifiée.
  3. Constructeur avec un objet convertible en chaîne : Convertit l'objet spécifié en une chaîne de caractères.
  4. Constructeur avec une séquence : Crée une chaîne de caractères à partir de la séquence spécifiée.
- Documentation officielle des constructeurs de la classe `str`

Exemples d'Utilisation des Constructeurs de la classe `str`

```python
# Empty constructor
empty_string = str()
print("# Output: Empty string:", empty_string) # Output:

# Constructor with a string
original_string = "Hello"
copied_string = str(original_string)
print("# Output: Copy of the string:", copied_string) # Output: Hello"

# Constructor with an object convertible to a string
number = 42
converted_string = str(number)
print("# Output: String from a number:", converted_string) # Output: 42

# Constructor with a sequence
sequence = ['a', 'b', 'c']
sequence_string = str(sequence)
print("# Output: String from a sequence:", sequence_string) # Output: ['a', 'b', 'c']
```

Indiçage positif et négatif

- En Python, les chaînes de caractères peuvent être accédées en utilisant des indices positifs et négatifs.
- Les indices positifs commencent à 0 pour le premier caractère et augmentent de 1 pour chaque caractère suivant.
- Les indices négatifs commencent à -1 pour le dernier caractère et diminuent de 1 pour chaque caractère précédent.
- Exemple :

```
1 s = "hello"
2 print("First character:", s[0])
3 # Output : First character: h
4 print("Last character:", s[-1])
5 # Output : Last character: o
```

Tranche de valeurs (slice)

- La tranche de valeurs en Python vous permet d'accéder à une plage de caractères d'une chaîne de caractères.
- Syntaxe : `string[début:fin:pas]`
- Exemple :

```
1 s = "hello"
2 print("Slice:", s[1:4])
3 # Output : Slice: ell
```

Opérateurs +, *, et in

- Python propose plusieurs opérateurs pour la manipulation de chaînes de caractères.
- `+` est utilisé pour la concaténation de chaînes de caractères.
- `*` est utilisé pour la répétition de chaînes de caractères OU pour l'unpacking (voir l'exemple dans le frame suivant).
- `in` est utilisé pour vérifier si une sous-chaîne existe dans une chaîne de caractères.
- Exemple :

```
1 s1 = "hello"
2 s2 = "world"
3 print("Concatenated:", s1 + s2)         # Output: Concatenated: helloworld
4 print("Repeated:", s1 * 3)              # Output: Repeated: hellohellohello
5 print("Contains 'lo'?", 'lo' in s1)     # Output: Contains 'lo'? True
```

Itération

- Les chaînes de caractères en Python peuvent être itérées à l'aide d'une boucle `for`.
- Chaque caractère de la chaîne est parcouru individuellement.
- Exemple :

```
s = "hello"
for char in s:
    print(char)
# Or via index:
for i in range(len(s)):
    print(s[i])
# Or via * starred expression:
print(*s) # Output : h e l l o
```

Méthodes de manipulation des chaînes de caractères

Voici quelques méthodes de manipulation des chaînes de caractères en Python :

- `split()`: Divise une chaîne en une liste de sous-chaînes en fonction d'un séparateur spécifié.
- `replace()`: Remplace toutes les occurrences d'une sous-chaîne par une autre dans la chaîne.
- `lower()`: Convertit tous les caractères de la chaîne en minuscules.
- `upper()`: Convertit tous les caractères de la chaîne en majuscules.
- `strip()`: Supprime les espaces (ou d'autres caractères spécifiés) au début et à la fin de la chaîne.
- `join()`: Joindre les éléments d'une séquence en une chaîne, en les séparant par une chaîne délimiteur.

Exemples

```python
# Example of string manipulation methods in Python
s = "Hello, world!"
# Splitting the string into a list of words
words = s.split(",")
print("Split:", words)  # Output: Split: ['Hello', ' world!']
# Replacing 'world' with 'Python'
new_s = s.replace("world", "Python")
print("Replace:", new_s)  # Output: Replace: Hello, Python!
# Converting to lowercase
lower_s = s.lower()
print("Lowercase:", lower_s)  # Output: Lowercase: hello, world!
# Converting to uppercase
upper_s = s.upper()
print("Uppercase:", upper_s)  # Output: Uppercase: HELLO, WORLD!
# Stripping whitespace
stripped_s = s.strip()
print("Stripped:", stripped_s)  # Output: Stripped: Hello, world!
# Joining a list of words into a single string
words = ['Hello', 'world']
joined_s = ' '.join(words)
print("Joined:", joined_s)  # Output: Joined: Hello world
```

Les f-strings en Python

- Les f-strings (formatted string literals) ont été introduites dans Python 3.6 en tant que fonctionnalité puissante de formatage de chaînes.
- Elles permettent l'insertion directe d'expressions Python dans des littéraux de chaînes.
- Les f-strings sont créées en préfixant une chaîne avec le caractère 'f' ou 'F'.
- Les expressions Python sont placées entre accolades à l'intérieur de la chaîne.

Sortie formatée avec les f-strings

```python
# Example of formatted output with f-string
name = "Ali"
age = 30
print(f"Hello, {name}! You are {age} years old.")

# Explanation
# The f-string is a concise way to format strings in Python.
# It allows embedding expressions inside string literals,
# using curly braces {} to enclose the expressions.
# In this example, variables 'name' and 'age' are inserted
# into the string to create a personalized greeting.
```

Sortie formatée avec les f-strings - Suite

```python
# Example 2
decimal = 2
amount = 45.6789
print(f"The total amount is ${amount:.{decimal}f}.")
# Output: The total amount is $45.68.
# Equivalent .format() method:
print("The total amount is ${:.{}f}.".format(amount, decimal))

# Explanation
# The f-string also supports formatting options.
# In this example, the variable 'amount' is formatted
# as a floating-point number with two decimal places,
# creating a string that displays the total amount with proper currency formatting.
```

La méthode format() en Python

- La méthode format() est une autre méthode de formatage de chaînes disponible en Python.
- Elle utilise des espaces réservés, appelés dans la chaîne de format, auxquels des valeurs sont passées en tant qu'arguments à la méthode format().
- La méthode format() peut effectuer un formatage plus avancé, tel que spécifier la largeur et la précision des nombres.

Sortie formatée avec la méthode format()

```
# Example of formatted output with the format() method
item = "apple"
qty = 5
print("You have {} {}{}."
.format(qty, item, 's' if qty > 1 else ''))

# Explanation
# The format() method is another way to format strings.
# It uses placeholders enclosed in curly braces {} within the string.
# The values to be inserted are provided as arguments to the format() method.
# In this example, the quantity and item variables are inserted into the string,
# and 's' is included conditionally based on the quantity.
```

Sortie formatée avec la méthode format() - Suite

```python
# Example 2
price = 19.99
discount = 0.15
final_price = price * (1 - discount)
print("The final price after a {}% discount is ${:.2f}.".format(int(discount * 100), final_price))

# Explanation
# The format() method allows for more complex formatting.
# In this example, the final price after applying a discount
# is formatted to display the percentage discount and the final price
# with two decimal places.
```

Les opérateurs

Les Opérateurs

Définition : En programmation informatique, un **opérateur** est un symbole ou un caractère qui indique au compilateur d'effectuer des processus mathématiques ou logiques spécifiques.

- Les opérateurs présentés ici sont communs à de nombreux langages de programmation. Python fournit ces opérateurs intégrés suivants:

- opérateurs **arithmétiques**;
- opérateurs **relationnels**;
- opérateurs **d'assignation**;
- opérateurs **logiques**;
- opérateurs **bit à bit**;
- opérateurs **divers** (Misc).

La Table de vérité

Un concept important lié à la manipulation des opérateurs est la table de vérité. Supposons que nous avons deux propositions (P) et (Q). Alors :

P	Q	P ET Q	P OU Q	P XOR Q	NON P	NON Q
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	0	1
1	1	1	1	0	0	0

Opérateurs Arithmétiques

Le tableau suivant explicite les opérateurs arithmétiques intégrés en Python. **Cet exemple** montre ces opérateurs arithmétiques dans un programme Python.

Opérateur	Signification
+	Ajoute la valeur de l'opérande du côté droit (RHS) à la valeur de l'opérande du côté gauche (LHS) (`a+b`)
−	Soustrait la valeur de l'opérande du (RHS) à la valeur de l'opérande du (LHS) (`a-b`)
*	Multiplie la valeur de l'opérande (RHS) a-fois (LHS) (`a*b`)
/	Divise la valeur de l'opérande du côté gauche (LHS) par la valeur de l'opérande du côté droit (RHS) (`a/b`)
//	(Division Entière/Euclidienne) Divise la valeur de l'opérande du côté gauche (LHS) par la valeur de l'opérande du côté droit (RHS) (`a//b`)
%	Le modulo renvoie le reste de la division euclidienne de la valeur de l'opérande du côté gauche (LHS) par la valeur de l'opérande du côté droit (RHS) (`a%b`)
**	Élève la valeur de l'opérande (LHS) à la puissance de la valeur de l'opérande (RHS) (`a**b`)

Opérateurs Relationnels

Le tableau suivant explicite les opérateurs relationnels intégrés en Python. **Cet exemple** montre ces opérateurs relationnels dans un programme Python.

Opérateur	Signification
==	Évalué à vrai seulement si les deux opérandes sont égales
!=	Évalué à vrai seulement si les deux opérandes ne sont pas égales
>	Évalué à vrai seulement si la valeur de l'opérande du côté gauche (LHS) est strictement supérieure à celle du côté droit (RHS)
<	Évalué à vrai seulement si la valeur de l'opérande du côté droit (RHS) est strictement supérieure à celle du côté gauche (LHS)
>=	Évalué à vrai seulement si la valeur de l'opérande du côté gauche (LHS) est strictement supérieure ou égale à celle du côté droit (RHS)
<=	Évalué à vrai seulement si la valeur de l'opérande du côté droit (RHS) est strictement supérieure ou égale à celle du côté gauche (LHS)

Opérateurs Logiques

Le tableau suivant explicite les opérateurs logiques intégrés en Python. **Cet exemple** montre ces opérateurs logiques dans un programme Python.

Opérateur	Signification
`and`	Opérateur ET logique. La condition est vraie seulement si les deux opérandes ne sont pas nulles
`or`	Opérateur OU logique. La condition est vraie seulement si l'une des opérandes n'est pas nulle
`not`	Opérateur NON logique. Il inverse l'état logique. Si la condition est vraie, alors l'opérateur NON logique la rendra fausse, et vice versa.

Opérateurs Bit à Bit

Le tableau suivant explicite les opérateurs bit à bit intégrés en Python. **Cet exemple** montre ces opérateurs bit à bit dans un programme Python.

Opérateur	Signification
&	Opérateur ET binaire, évalue à VRAI seulement si les deux bits sont VRAIS
|	Opérateur OU binaire, évalue à VRAI seulement si l'un des bits est VRAI
^	Opérateur OU exclusif binaire, évalue à VRAI seulement si un bit est VRAI dans un opérande mais pas dans les deux.
~	Opérateur NON binaire unaire (ou opérateur de complément). L'opérateur de complément inverse simplement les bits (même logique que NON).
<<	Opérateur de décalage binaire à gauche. La valeur des opérandes du côté gauche est déplacée vers la gauche par le nombre de bits spécifié par l'opérande du côté droit.
>>	Opérateur de décalage binaire à droite. La valeur des opérandes du côté gauche est déplacée vers la droite par le nombre de bits spécifié par l'opérande du côté droit.

Opérateurs d'Assignation

Le tableau suivant explicite les opérateurs d'assignation intégrés en Python. **Cet exemple** montre ces opérateurs d'assignation dans un programme Python.

Opérateur	Signification
=	Opérateur d'assignation. Il assigne les valeurs des opérandes du côté droit (RHS) aux opérandes du côté gauche (LHS).
+=	Opérateur d'assignation ET d'addition. Il ajoute l'opérande du côté droit (RHS) à l'opérande du côté gauche (LHS) et assigne le résultat à l'opérande du côté gauche (LHS).
−=	Opérateur d'assignation ET de soustraction. Il soustrait l'opérande du côté droit (RHS) à l'opérande du côté gauche (LHS) et assigne le résultat à l'opérande du côté gauche (LHS).
*=	Opérateur d'assignation ET de multiplication. Il multiplie l'opérande du côté droit (RHS) à l'opérande du côté gauche (LHS) et assigne le résultat à l'opérande du côté gauche (LHS).
/=	Opérateur d'assignation ET de division. Il divise l'opérande du côté gauche (LHS) par l'opérande du côté droit (RHS) et assigne le résultat à l'opérande du côté gauche (LHS).
%=	Opérateur d'assignation ET de modulo. Il assigne le reste de la division euclidienne de l'opérande du côté gauche (LHS) par l'opérande du côté droit (RHS) à l'opérande du côté gauche (LHS).

Opérateurs d'Assignation - Suite

Opérateur	Signification
<<=	Opérateur d'assignation ET de décalage à gauche.
>>=	Opérateur d'assignation ET de décalage à droite.
&=	Opérateur d'assignation ET logique.
^=	Opérateur d'assignation ET de OU exclusif logique.
|=	Opérateur d'assignation ET de OU inclusif logique.

Opérateur Ternaire

- La signature de l'**opérateur Ternaire** est la suivante: `value_if_condition_is_true if condition else value_if_condition_is_false`

```
1 # Ternary Operator
2 x = 7
3 y = 4
4 max_value = x if x >= y else y
5 print(f"Max value between x and y: {max_value}")
```

Autres opérateurs

- Voici un exemple d'autres opérateurs que l'on peut trouver en Python:

```python
# Other operators in Python

# Membership Operators
list_a = [1, 2, 3, 4, 5]
print(f"2 in list_a: {2 in list_a}")  # True if 2 is in list_a
print(f"6 not in list_a: {6 not in list_a}")  # True if 6 is not in list_a

# Identity Operators
z = 10
print(f"x is z: {x is z}")  # True if x and z reference the same object
print(f"x is not z: {x is not z}")  # True if x and z reference different objects
```

Opérateur Morse (Walrus) :=

- Introduit avec **Python 3.8**, l'opérateur **morse** (`:=`) permet d'**affecter une valeur à une variable au sein d'une expression**, tout en retournant cette valeur.
- Il s'écrit sous la forme : `variable := expression`.
- Il est principalement utilisé pour **éviter la répétition d'un même calcul** et ainsi **simplifier le code**.
- Ses avantages : **réduction de la duplication** et **amélioration de la lisibilité**.
- Ses limites : il **ne s'utilise que dans des expressions** et un **usage excessif peut nuire à la clarté**.
- Pour plus de détails : Opérateur morse (walrus operator).

Opérateur Morse (Walrus) := (Exemples)

```python
# Example 1: usage in a list comprehension
numbers = [1, 2, 3, 4, 5]
squared_numbers = [(square := num ** 2) for num in numbers]
print(squared_numbers)  # Output: [1, 4, 9, 16, 25]
print(square)  # Output: 25

# Example 2: usage in a while loop
while (user_input := input("Enter 'quit' to exit: ")) != 'quit':
    print("You entered:", user_input)

# Example 3: usage in an if-else statement
value = 10
if (result := value * 2) > 15:
    print("Result is greater than 15")
else:
    print("Result is less than or equal to 15")
```

Priorité des opérateurs en Python

- La priorité des opérateurs en Python détermine l'ordre dans lequel les opérations sont effectuées lorsqu'une expression contient plusieurs opérateurs.
- Les opérateurs avec une plus haute priorité sont évalués en premier.
- Voici un tableau illustrant la priorité des opérateurs du plus prioritaire au moins prioritaire :

Priorité des opérateurs en Python - Tableau

Priorité	Opérateurs
1	Parenthèses ()
2	Puissance **
3	Multiplication *, Division /, Division entière //, Modulo %
4	Addition +, Soustraction -
5	Opérateurs de comparaison (>, <, >=, <=, ==, !=)
6	Opérateurs logiques (and, or, not)

Opérateurs pour les Booléens (Booleans)

- Les booléens en Python prennent en charge certains des opérateurs mentionnés précédemment, notamment :
  - `&` : opérateur de bitwise AND
  - `|` : opérateur de bitwise OR

Opérateurs pour les Booléens (Booleans) - Exemple

- Voici des exemples d'utilisation :

```
x = True
y = False
result_bitwise_and = x & y # False
result_bitwise_or = x | y # True
```

Opérateurs pour les Entiers (Integers)

- Les entiers en Python supportent plusieurs opérateurs, y compris :
  - `>>` : décalage à droite
  - `<<` : décalage à gauche
  - `|` : opérateur de bitwise OR
  - `&` : opérateur de bitwise AND
  - `^` : opérateur de bitwise XOR
  - `/` : division
  - `//` : division entière
  - `%` : modulo
  - `**` : exponentiation

Opérateurs pour les Entiers (Integers) - Exemple

- Voici des exemples d'utilisation :

```
x = 5
y = 2
result_shift_right = x >> y # 1
result_shift_left = x << y # 20
result_bitwise_or = x | y # 7
result_bitwise_and = x & y # 0
result_bitwise_xor = x ^ y # 7
result_division = x / y # 2.5
result_floor_division = x // y # 2
result_modulo = x % y # 1
result_exponentiation = x ** y # 25
```

Opérateurs pour les Flottants (Floats)

- Les flottants en Python prennent en charge certains des opérateurs mentionnés précédemment, notamment :
  - `/` : division
  - `//` : division entière
  - `%` : modulo
  - `**` : exponentiation

Opérateurs pour les Flottants (Floats) - Exemple

- Voici des exemples d'utilisation :

```
x = 5.5
y = 2.0
result_division = x / y # 2.75
result_floor_division = x // y # 2.0
result_modulo = x % y # 1.5
result_exponentiation = x ** y # 30.25
```

Opérateurs pour les Nombres Complexes (Complex Numbers)

- Les nombres complexes en Python prennent en charge certains des opérateurs mentionnés précédemment, notamment :
  - `**` : exponentiation
- Voici un exemple d'utilisation :

```
1 z1 = 3 + 2j
2 z2 = 2 + 1j
3 result_exponentiation = z1 ** z2
```

Opérateurs pour les Nombres Complexes (Complex Numbers) - Exemple

- Voici un exemple d'utilisation :

```
import cmath
z1 = 3 + 2j
z2 = 2 + 1j
result_exponentiation = z1 ** z2
```

Systèmes de notation

Une image illustrant l'équivalence entre les systèmes de notation:

Decimal	Binary	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Figure: Number systems

Notations Exponentielle

- En Python, la notation exponentielle est utilisée pour représenter les nombres en notation scientifique.
- Exemple:
  - `1.23e-4` représente 1.23×10^{-4}.
  - `2.5e2` représente 2.5×10^{2}.

Notations Binaire

- En Python, la notation binaire est utilisée pour représenter les nombres en base 2.
- Exemple:
  - `0b1010` représente 10_{10} en base 10.
  - `0b1101` représente 13_{10} en base 10.

Notations Octale

- En Python, la notation octale est utilisée pour représenter les nombres en base 8.
- Exemple:
  - `0o10` représente 8_{10} en base 10.
  - `0o17` représente 15_{10} en base 10.

Notations Hexadécimale

- En Python, la notation hexadécimale est utilisée pour représenter les nombres en base 16.
- Exemple:
  - `0xA` représente 10_{10} en base 10.
  - `0xFF` représente 255_{10} en base 10.

Fonction `hex()`

- La fonction `hex()` convertit un entier en sa représentation hexadécimale sous forme de chaîne.
- **Usage**: `hex(number)`.
- **Exemple**:

```
1 # Example of function hex()
2 x = 255
3 print("Hexadecimal representation of x: ", hex(x))
```

Fonction `oct()`

- La fonction `oct()` convertit un entier en sa représentation octale sous forme de chaîne.
- **Usage**: `oct(number)`.
- **Exemple**:

```
1 # Example of function oct()
2 x = 255
3 print("Octal representation of x: ", oct(x))
```

Fonction `bin()`

- La fonction `bin()` convertit un entier en sa représentation binaire sous forme de chaîne.
- **Usage**: `bin(number)`.
- **Exemple**:

```
1 # Example of function bin()
2 x = 255
3 print("Binary representation of x: ", bin(x))
```

Fonction `chr()`

- La fonction `chr()` renvoie le caractère Unicode correspondant au nombre entier donné.
- **Usage**: `chr(number)`.
- **Exemple**:

```
1 # Example of function chr()
2 x = 65
3 print("Unicode Character of x: ", chr(x))
```

Fonction `ord()`

- La fonction `ord()` renvoie l'entier Unicode correspondant au caractère donné.
- **Usage**: `ord(character)`.
- **Exemple**:

```
1 # Example of function ord()
2 x = 'A'
3 print("Unicode integer code of x :", ord(x))
```

** Structures conditionnelles et répétitives **

Structure conditionnelle `if` ...

- En algorithmique et en programmation, nous appelons **structure conditionnelle** l'ensemble des instructions qui testent si une condition est vraie ou non. Il y en a trois :
  - La structure conditionnelle **if** :

```
if condition:
      my_instructions
```

Structure conditionnelle `if ...else`

- **Suite :**
  - La structure conditionnelle **if ...else** :

```
if condition:
      my_instructions
else:
      my_other_instructions
```

Structure conditionnelle `if …elif …else`

- **Suite :**
  - La structure conditionnelle **if …elif …else :**

```
if condition_1:
      my_instructions_1
elif condition_2:
      my_instructions_2
else:
      my_other_instructions
```

Structure conditionnelle boucle `while`

- De plus, une **structure conditionnelle itérative** permet d'exécuter plusieurs fois (itérations) la même série d'instructions. L'instruction **while** exécute les blocs d'instructions tant que la condition de while est vraie. Le même principe s'applique à la boucle **for** (pour), les deux structures sont détaillées ci-dessous :
  - La structure conditionnelle itérative **while** :

```
while condition:
        mes_instructions
```

Structure conditionnelle: boucle `for`

- **Suite** :
  - La structure conditionnelle itérative `for` :

```
for i in range(begin, end, step):
      my_instructions
```

- **Remarque** : La différence majeure entre la boucle for et la boucle `while` est que la boucle for est utilisée lorsque le nombre d'itérations est connu, tandis que l'exécution se fait dans la boucle `while` jusqu'à ce que l'instruction dans le programme soit prouvée fausse.

Structure conditionnelle: boucle `for`

- La structure conditionnelle element-based `for` :

```
for element in collection:
    my_instructions
```

Instructions break et continue

- L'instruction `break` permet de sortir prématurément d'une boucle.
- L'instruction `continue` permet de passer à l'itération suivante dans une boucle.
- Elles sont souvent utilisées pour contrôler le flux d'exécution dans les boucles.

```
# Example of break and continue statements
for i in range(10):
    if i == 5:
        break  # Exit the loop when i equals 5
    print(i, end=' ')
print("\n")

for i in range(10):
    if i % 2 == 0:
        continue  # Skip the current iteration if i is even
    print(i, end=' ')
```

Constructeur enumerate()

- Le constructeur `enumerate()` est utilisé pour **énumérer les éléments** d'une séquence tout en conservant leur **indice**.
- Elle renvoie un objet énumérable (`enumerate`) qui produit des tuples contenant à la fois l'index et la valeur de chaque élément de la séquence.
- Elle est couramment utilisée dans les boucles pour accéder à la fois à l'indice et à la valeur de chaque élément.

Constructeur enumerate() - Exemple

```python
# Example of enumerate constructor usage
fruits : list[str] = ['apple', 'banana', 'cherry'] # Creating a list of fruits
# Printing the enumerate object and its type
print(f"Enumerating fruits: {enumerate(fruits)}") # Outputs the enumerate object <enumerate object at
    0x10fcd2160>
print(f"Type of enumerate object: {type(enumerate(fruits))}") # Outputs <class 'enumerate'>
# Converting enumerate object to a list of tuples
print(f"Enumerating fruits: {list(enumerate(fruits))}") # Outputs [(0, 'apple'), (1, 'banana'), (2, 'cherry')]
# Printing the type of the first element in the list of tuples
print(f"Type of first element of list(enumerate(fruits)) : {type(list(enumerate(fruits))[0])}") # Outputs
    <class 'tuple'>
# Using enumerate in a for loop
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

Bloc else dans une structure répétitive

- En Python, les boucles `for` et `while` peuvent avoir un bloc `else` qui est exécuté lorsque la boucle se termine normalement (sans être interrompue par un `break`).
- Le bloc `else` est exécuté après que la condition de la boucle devienne fausse.

```python
# Examples of else block on loop
for i in range(5):
    print(i)
else:
    print("Loop completed without break")
# This will print after the loop
```

```python
for i in range(5):
    if i == 3:
        break
else:
    print("Loop completed without break")
# This will not print because the loop was exited with break
```

** Les Fonctions **

Fonctions en Python

- On peut vouloir définir une fonction afin de l'utiliser dans un programme, la syntaxe générale pour déclarer et définir une fonction est la suivante :

```
1 def function_name(arg_1 : type_arg_1, arg_2 : type_arg_2, ...) -> return_type:
2     # function implementation
```

- Une fonction sera toujours définie avec le mot-clé `def`.
- Veuillez noter l'importance de l'indentation dans l'implémentation de la fonction.

Fonctions en Python : exemple 1

Définir une fonction sans paramètres et sans retour :

```
1 # Definition of the utility function printLine
2 def print_line():
3     print("------------")
4
5 # Testing our function
6 print_line()
```

Fonctions en Python : exemple 2

Définir une fonction qui prend un nombre en tant que paramètre et renvoie son carré :

```python
# Definition of our square function
def my_square(n : float) -> float:
    return n**2

# Testing our function
input_variable : float = float(input("Please input a real number : "))
print(f"The square of {input_variable} is {my_square(input_variable)}")
```

Notez l'utilisation de l'opérateur $**$ pour calculer la puissance d'un nombre.

Notion de récursivité : fonctions récursives

Définitions :

- Un algorithme est appelé **récursif** s'il est défini par lui-même.
- De la même manière, une fonction est appelée **récursive** si elle fait référence à elle-même.

Fonctions en Python : exemple 3

Définir une fonction récursive avec un paramètre et sans retour (le retour renvoie vide) :

```
# Definition of our recursive function my_count
def my_count(n):
    print(n)
    if n == 0 :
        print("Terminated.")
        return
    my_count(n-1)

# Testing our function
my_count(10)
```

Nombre variable de paramètres dans une fonction

- Python vous permet de créer des fonctions avec un nombre variable de paramètres. Exemple: fonction `print()`.
- La syntaxe générale de telles fonctions est :

```
# Definition of the function nameFunction with variable arguments
def function_name(*args):
    # instructions
```

```
# Example:
def print_foo(*args):
    for arg in args:
        print(arg)

# Test our function
print_foo("Hello", 1, 2, 3, "World")
''' # Output:
Hello
1
2
3
World
'''
```

Fonctions avec des paramètres clé/valeur

- Les paramètres peuvent également être gérés sous forme de clés/valeurs pour peupler un dictionnaire :
- La syntaxe générale de telles fonctions est :

```
# Definition of the function nameFunction with keyword arguments
def function_name(**kwargs):
    # instructions
```

```
# Example:
def print_dict(**kwargs):
    for (k, v) in kwargs.items():
        print(k, v)

# Test our function
printDict(Day_1="Monday", Day_2="Tuesday")
''' # Output:
Day_1 Monday
Day_2 Tuesday
'''
```

Générateurs

- Un générateur est implémenté avec le mot-clé `yield`, il permet de créer un flux de données. Une sorte de réservoir de données ; qui peut être appelé morceau par morceau par `next()`. Other doc.

```python
# Define a generator function
def my_generator():
    for i in range(4, 0, -1):
        yield i
# Create an object of generator type
gen = my_generator()
# Print the type of the generator
print(f"Type of gen object: {type(gen)}") # Output : <class 'generator'>
# Print the results
print(next(gen)) # Output : 4
print(next(gen)) # Output : 3
print(next(gen)) # Output : 2
print(next(gen)) # Output : 1
# print(next(gen)) # Error, because we stop at 1
'''
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
'''
```

Espace de noms (Scopes)

- En Python, chaque fonction crée son propre espace de noms local lorsqu'elle est appelée. Les variables déclarées à l'intérieur de cette fonction sont accessibles uniquement à l'intérieur de cette fonction, à moins qu'elles ne soient déclarées comme globales.
- L'espace de noms global est accessible depuis n'importe où dans le script Python et peut être modifié à l'aide du mot-clé `global`.

Exemple d'espace de noms

```python
# Example of local and global namespaces in Python
x = 10  # Global variable

def function():
    y = 20  # Local variable
    print("Local variables:", locals())

fonction()
print("Global variables:", globals())
```

Notion de Scope

- Le **scope** (**portée**) d'une variable en Python fait référence à la région du code où la variable peut être utilisée.
- Python utilise une règle de portée lexicale, ce qui signifie que la portée d'une variable est déterminée par son emplacement dans le code source.
- Le concept de portée concerne la visibilité des variables dans le code.

Notion de scope - Exemple complet

```python
# Scope 1: Global Scope
global_variable = "I am global"

def outer_function():
    # Scope 2: Outer Function Scope
    outer_variable = "I am in the outer function"

    def inner_function():
        # Scope 3: Inner Function Scope
        inner_variable = "I am in the inner function"
        print("Inner variable:", inner_variable)
        print("Outer variable inside the inner function:", outer_variable)
        print("Global variable inside the inner function:", global_variable)
inner_function()
print("Outer variable:", outer_variable)
# Uncommenting the line below would result in an error
# print("Inner variable outside the inner function:", inner_variable)
outer_function()
print("Global variable:", global_variable)
# Uncommenting the line below would result in an error
# print("Outer variable outside the outer function:", outer_variable)
```

Portée de Variables Globales

- Les variables globales sont déclarées en dehors de toute fonction et peuvent être utilisées à travers tout le programme.

```
# Example of the notion of global variable
global_var = 42

def print_global():
    print("Global variable:", global_var)

print_global()  # Displays "Global variable: 42"
'''
The global_var is accessible inside print_global()'s scope even if print_global() method has no variable
     named global_var.
'''
```

Portée de Variables Locales

- Les variables locales sont déclarées à l'intérieur d'une fonction et sont accessibles uniquement à l'intérieur de cette fonction.

```
def example_function():
    local_variable = "I am local"
    print(local_variable)

example_function()  # Displays "I am local"
# Uncommenting the line below would result in an error
# print(local_variable)
```

Différence entre la Portée Locale et Globale

En Python, la portée des variables peut être locale ou globale, ce qui détermine où la variable peut être utilisée ou modifiée.

- **Portée Locale :** Une variable déclarée à l'intérieur d'une fonction a une portée locale. Elle n'est accessible qu'à l'intérieur de cette fonction.
- **Portée Globale :** Une variable déclarée en dehors de toutes les fonctions a une portée globale. Elle peut être utilisée dans l'ensemble du programme.

Il est important de comprendre la différence entre la portée locale et globale pour éviter les erreurs et assurer une utilisation correcte des variables.

Exemple illustrant la différence de portées

```python
# Declaration of a global variable
index = 0

def modify_index():
    # Redefining the global variable inside the function modify_index()
    index = 42
    print("Value of index inside the function:", index) # Displays: 42

# Calling the function
modify_index()

# Displaying the value of index outside the function
print("Value of index outside the function:", index) # Displays: 0
```

Effet de la boucle `for` sur la portée des variables

```python
# Global variable definition
i = "Global variable"

# Loop using the same variable name
for i in range(3):
    print("Inside the loop:", i)

# After the loop, the global variable is overwritten
print("After the loop:", i)  # Displays: 2
```

Explication :

- La variable `i` utilisée dans la boucle `for` n'a pas de portée locale.
- Elle réécrit la variable globale `i` définie avant la boucle.
- En Python, les boucles `for` ne créent pas de nouvelle portée : la variable d'itération existe encore après la boucle.

Exemple illustrant la différence de portées (Suite)

- Dans cet exemple, la variable `index` est initialement déclarée en tant que variable globale, et vaut 0.
- Elle est ensuite redéfinie à l'intérieur de la fonction `modify_index`.
- L'appel de la fonction `modify_index` imprime la variable locale `index` qui vaut 42.
- La dernière ligne imprime la valeur de la variable globale `index` qui elle vaut 0.

Mot-clé `global` en Python

- Le mot-clé `global` en Python est utilisé pour déclarer qu'une variable à l'intérieur d'une fonction appartient à l'espace de noms global plutôt qu'à l'espace de noms local. Cela signifie que la variable sera accessible et modifiable à la fois à l'intérieur et à l'extérieur de la fonction.
- L'utilisation du mot-clé `global` permet de modifier une variable définie en dehors de la fonction à l'intérieur de cette fonction, tout en maintenant sa portée globale.

Mot-clé `global` en Python - Exemple pratique

- Voici un exemple illustrant l'utilisation du mot-clé `global` en Python :

```
global_var = 0
print("Global Scope *before* calling foo() and bar(): global_var = ", global_var)
# Output : Global Scope *before* calling foo(): global_var =  0
def foo():
    global_var = 1
    print("Inside foo()'s Local Scope: global_var = ", global_var)

foo()
# Output : Inside foo()'s Local Scope: global_var =  1
print("Global Scope *after* calling foo(): global_var = ", global_var)
# Output : Global Scope *after* calling foo(): global_var =  0

def bar():
    global global_var
    global_var = 2
    print("Inside bar()'s Local Scope: global_var = ", global_var)

bar()
# Output : Inside bar()'s Local Scope: global_var =  2
print("Global Scope *after* calling bar(): global_var = ", global_var)
# Output : Global Scope *after* calling bar(): global_var =  2
```

Scopes pré-définis (__builtins__)

- En Python, le module `__builtins__` contient les fonctions et les types de données intégrés qui sont disponibles dans tous les espaces de noms.
- Ces fonctions et types de données sont prêts à l'emploi sans avoir besoin d'importer des modules supplémentaires.

Exemple de scopes pré-définis (__builtins__)

```
1 # List of builtin functions __builtins__
2 print("List of builtin functions __builtins__: ")
3 print(dir(__builtins__))
```

Exemple complet

- Voici un exemple illustrant les scopes en Python :

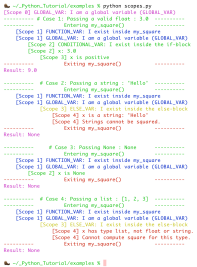


Figure: Scopes

Fonction dir()

- En Python, la fonction `dir()` est utilisée pour obtenir la liste des noms de symboles (variables, fonctions, classes, etc.) définis dans un module ou un objet.
- Elle retourne une liste de chaînes de caractères contenant les noms des symboles.

Exemple de la fonction dir()

```
# Usage of dir() function in Python
print(" List of symbols and methods of module math: ")
import math
print(dir(math))
list = [1,2,3]
print(" List of methods of class list: ")
print(dir(list))
```

Fonctions - Retourner des valeurs, instruction return

- L'instruction `return` est utilisée dans les fonctions Python pour renvoyer une valeur à l'appelant de la fonction. Lorsque Python rencontre une instruction `return`, il quitte immédiatement la fonction et renvoie l'expression associée à `return` à l'appelant. Cela permet à une fonction de calculer un résultat et de le renvoyer à l'endroit où elle a été appelée.

Exemples de l'instruction return

```python
def add(a, b):
    result = a + b
    return result

sum_result = add(3, 5)
print(sum_result)  # Output: 8
```

```python
def square_and_cube(x):
    square = x ** 2
    cube = x ** 3
    return square, cube

result = square_and_cube(4)
print(result)  # Output: (16, 64)
```

```python
def hello():
    print("Hello World!")
    return

result = hello()
print(result)  # Output: None
```

Le Duck Typing en Python

Définition : Le Duck Typing est un concept en Python qui se concentre sur le comportement d'un objet plutôt que sur son type. Il suit le principe selon lequel "si cela ressemble à un canard, nage comme un canard et fait le cri d'un canard, alors c'est probablement un canard". En Python, cela signifie que le type ou la classe d'un objet est déterminé par ses méthodes et propriétés plutôt que par son héritage explicite ou ses annotations de type.

Spécifications sur le Duck Typing

- Le Duck Typing permet une flexibilité dans les arguments de fonction ou de méthode en fonction du comportement qu'ils présentent plutôt que de leur type explicite.
- Le concept met l'accent sur l'importance du comportement de l'objet plutôt que sur son type formel, favorisant une approche de programmation plus dynamique et ouverte.
- Un exemple de Duck Typing en Python est la fonction intégrée `len()`, qui fonctionne sur n'importe quel objet qui définit une méthode `__len__()`, indépendamment de son type explicite.
- Référence : Real Python - Duck Typing

Exemples

```python
# Example of len() function
class CustomList:
    def __init__(self, items):
        self.items = items

    def __len__(self):
        return len(self.items)

custom_list_instance = CustomList([1, 2, 3, 4, 5])
print(len(custom_list_instance))  # Output: 5
```

```python
class Dog:
    def sound(self):
        return "Woof!"

class Cat:
    def sound(self):
        return "Meow!"

class Duck:
    def sound(self):
        return "Quack!"

def make_sound(animal):
    return animal.sound()

# Usage of duck typing
dog_instance = Dog()
cat_instance = Cat()
duck_instance = Duck()

print(make_sound(dog_instance))     # Output: Woof!
print(make_sound(cat_instance))     # Output: Meow!
print(make_sound(duck_instance))    # Output: Quack!
```

Fonctions - Valeurs par défaut

- En Python, les fonctions peuvent avoir des paramètres avec des valeurs par défaut. Ces valeurs sont utilisées lorsque l'appelant de la fonction ne fournit pas de valeur pour ces paramètres. Les valeurs par défaut permettent de définir des paramètres optionnels pour les fonctions.

Exemple - Valeurs par défaut

```
1 def greet(name="World"):
2     print("Hello", name)
3
4 greet()  # Output: Hello World
5 greet("Alice")  # Output: Hello Alice
```

Fonctions - Passage par étiquette

- En Python, les arguments peuvent être passés à une fonction par leur nom, ce qui permet de spécifier explicitement quel argument doit être affecté à quel paramètre. Cela offre une flexibilité dans l'ordre des arguments lors de l'appel d'une fonction.

Exemple - Passage par étiquette

```
def greet(first_name, last_name):
    print("Hello", first_name, last_name)

greet(last_name="Doe", first_name="John")  # Output: Hello John Doe
```

Fonctions - Nombre d'arguments arbitraire (*args, **kwargs)

- En Python, il est possible de définir des fonctions prenant un nombre variable d'arguments. L'utilisation de *args permet de capturer un nombre arbitraire d'arguments positionnels, tandis que **kwargs permet de capturer un nombre arbitraire d'arguments nommés.

Exemple - Nombre d'arguments arbitraire (*args, **kwargs)

```python
def sum(*args):
    result = 0
    for num in args:
        result += num
    return result

print(sum(1, 2, 3, 4))  # Output: 10
```

```python
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(key + ":", value)

print_info(name="Alice", age=30, city="New York")
# Output:
# name: Alice
# age: 30
# city: New York
```

Exemple 1 - Mélange de *args et **kwargs

```python
def print_info(*args, **kwargs):
    print("Positional arguments:")
    for arg in args:
        print(arg)
    print("Keyword arguments:")
    for key, value in kwargs.items():
        print(key + ":", value)

print_info("Alice", 30, city="New York", occupation="Engineer")
# Output:
# Positional arguments:
# Alice
# 30
# Keyword arguments:
# city: New York
# occupation: Engineer
```

Exemple 2 - Mélange de *args et **kwargs

```python
def create_person(**kwargs):
    person = {}
    for key, value in kwargs.items():
        person[key] = value
    return person

def print_person_info(*args, **kwargs):
    print("Printing person information:")
    for person in args:
        print("Name:", person["name"])
        print("Age:", person["age"])
        for key, value in kwargs.items():
            if value:
                print(key.capitalize() + ":", person[key])
        print("\n")

# Creating persons using **kwargs
person1 = create_person(name="Alice", age=30, city="New York", occupation="Engineer")
person2 = create_person(name="Bob", age=25, city="Los Angeles", occupation="Artist")
person3 = create_person(name="Charlie", age=35, city="Chicago")

# Printing person information
print_person_info(person1, person2, person3)
```

Lambda Functions

- Les fonctions `Lambda` sont:
  - de petite taille ;
  - anonymes
  - sujettes à une syntaxe plus restrictive ;
  - concises ;
  - peuvent prendre n'importe quel nombre d'arguments.
- On définit une fonction `lambda` avec le mot-clé: **lambda**.
- La syntaxe générale d'une fonction `lambda` est:

```
lambda args : implementation
```

Lambda Functions: examples

```
# Example 1: lambda function with 1 parameter
f = lambda x : 2 * x
print( f(50) )
#Output: 100.
```

```
# Example 2: lambda function with 2 parameters
fun = lambda a, b : a ** b
print(fun(7, 2))
#Output: 49.
```

Usage of Lambda Functions

- The utility of a Lambda function is when you use it as an anonymous function inside another function.
- A complete example is shown below:

```
# Define a multiplier function
def multiplier(n):
    return lambda x: x * n

# Create instances of the multiplier function
Doubler = multiplier(2)
Tripler = multiplier(3)

# Print the results
print(Doubler(33))  # Outputs: 66
print(Tripler(33))  # Outputs: 99
```

Fonctions eval(), exec()

- Les fonctions `eval()` et `exec()` permettent d'exécuter du code Python dynamiquement à partir d'une chaîne de caractères.
- `eval()` évalue une expression Python et retourne la valeur.
- `exec()` exécute du code Python et ne retourne rien.

Exemples d’utilisation

Exemple avec eval():

```
x = 10
result = eval('x ** 2 + 5')
print(result)  # Output: 105
```

Exemple avec exec():

```
code = '''for i in range(5): print(i)'''
exec(code)  # Output: 0 1 2 3 4
```

Constructeurs map() et filter()

- `map()` applique une fonction à chaque élément d'un itérable (comme une liste) et retourne un itérable contenant les résultats.
- `filter()` filtre les éléments d'un itérable en fonction d'une fonction de filtrage et retourne un itérable contenant les éléments filtrés.

Exemples d'utilisation

Exemple avec map():

```
numbers = [1, 2, 3, 4, 5]
squared = map(lambda x: x ** 2, numbers)
print(list(squared))  # Output: [1, 4, 9, 16, 25]
```

Exemple avec filter():

```
numbers = [1, 2, 3, 4, 5]
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers))  # Output: [2, 4]
```

** Structures de données **

Structures de données immuables

Utilité des types non-modifiables (optimisation mémoire)

- Les types non-modifiables, tels que les tuples et les chaînes de caractères, offrent l'avantage de l'optimisation mémoire.
- En Python, une fois qu'un objet non modifiable est créé, son contenu ne peut pas être modifié.
- Cela permet à l'interpréteur Python d'effectuer certaines optimisations en termes de gestion de la mémoire, car il sait que ces objets resteront inchangés.
- Ainsi, les types non-modifiables peuvent contribuer à réduire l'utilisation de la mémoire dans les programmes Python, ce qui peut être crucial pour les applications exigeantes en termes de performances et d'efficacité.

Fonction `id()`

- La fonction `id()` retourne l'identifiant de l'objet donné. Cet identifiant est unique pour chaque objet et reste constant pendant toute la durée de vie de l'objet.
- Documentation officielle
- **Usage**: `id(object)`.
- **Exemple**:

```
1 # Example of using the id() function
2 x = 42
3 print("ID of x:", id(x))
```

Fonction `hash()`

- La fonction `hash()` retourne la valeur de hachage d'un objet. Les objets qui sont égaux doivent avoir le même hachage, mais l'inverse n'est pas nécessairement vrai.
- Documentation officielle
- **Usage**: `hash(object)`.
- **Exemple**:

```
1 # Example of using the hash() function
2 x = 42
3 print("Hash de x:", hash(x))
```

Opérateur `is`

- L'opérateur `is` vérifie si deux variables font référence au même objet en mémoire.
- Si les deux variables référencent le même objet, `is` renvoie `True`; sinon, il renvoie `False`.
- **Usage**: `object1 is object2`
- Documentation officielle
- Exemple GFG
- **Exemple**:

```
1 # Example of using the is operator
2 x = [1, 2, 3]
3 y = x
4 print(x is y)  # True, because x and y references the same object
```

Les chaînes de caractères unicode (str)

La classe `str` en Python

- La classe `str` est utilisée pour représenter les chaînes de caractères en Python.
- Les objets de type `str` sont **ordonnées immutables**, ce qui signifie qu'ils ne peuvent pas être modifiés après leur création.
- Documentation officielle
- **Exemple**:

```
1 # Example of using the str class
2 s = "Hello, world!"
3 print(s) # Displays: Hello, world!
4 print(type(s).__name__) # Displays: 'str'
5 print(type(s)) # Displays: <class 'str'>
6 print(s.__class__) # Displays: <class 'str'>
```

Méthodes importantes de la classe `str`

- La classe `str` offre de nombreuses méthodes pour manipuler les chaînes de caractères, telles que `split()`, `replace()`, `lower()`, `upper()`, `strip()`, `join()`, etc.
- Ces méthodes permettent de réaliser des opérations courantes sur les chaînes de caractères, comme diviser une chaîne en plusieurs parties, remplacer des sous-chaînes, convertir la casse, etc.
- Documentation officielle des méthodes str

Ordonnancement de la classe str

```python
# Example showing the ordering of the str class in Python
s = "hello"
print("Original:", s)

# Access by positive and negative index
print("First character:", s[0])  # Displays the first character 'h'
print("Second character:", s[1])  # Displays the second character 'e'
print("Last character:", s[-1])  # Displays the last character 'o'

# Using a for loop to iterate over each character with index
print("Traversal with a for loop and indices:")
for i in range(len(s)):
    print("Character at index", i, ":", s[i])

# Slicing with positive and negative indices
print("Slicing with positive and negative indices:")
print("First three characters:", s[:3])  # Displays 'hel'
print("Last three characters:", s[-3:])  # Displays 'llo'

# Using a for loop to iterate over each character
print("Traversal with a for loop:")
for char in s:
    print(char)
```

Manipulation des Chaînes de Caractères

- En Python, les chaînes de caractères (`str`) offrent une variété de méthodes intégrées pour effectuer des opérations sur les chaînes. Vous pouvez consulter la documentation officielle pour en savoir plus : Documentation Python - String Methods.

Manipulation des Chaînes de Caractères - Méthodes

```
# String methods examples
message = "Hello, World!"

# String methods
string_methods = dir(message)
print("String methods:", string_methods)
```

Manipulation de Chaînes de Caractères - Exemples

```python
# Concatenation
message = "Hello, World!"
new_message = message + " Welcome to Python Tutorial!"
print("Concatenation:", new_message)

# String length
length = len(message)
print("String length:", length)

# Accessing a specific character
first_char = message[0]
print("First character:", first_char)

# Conversion to uppercase
print("Uppercase:", message.upper())
```

Définition avec guillemets simples

- Les guillemets simples (`''`) sont utilisés pour définir une chaîne de caractères.
- Exemple :

```
1 single_quote_string = 'Hello, world!'
```

Définition avec guillemets doubles

- Les guillemets doubles (`""`) sont également utilisés pour définir une chaîne de caractères.
- Exemple :

```
1 double_quote_string = "Hello, world!"
```

Définition avec guillemets simples et doubles

- Les guillemets simples (`''`) ou doubles (`" "`) peuvent être utilisés pour définir une chaîne de caractères.
- Exemple :

```
1 sentance = "He said: 'Hello, world!'"
```

Chaînes multilignes avec triple guillemets simples

- Les triple guillemets simples (`''' '''`) sont utilisés pour définir des chaînes de caractères multilignes.
- Exemple :

```
multiline_text_single_quote_string =
'''Hello,
This is a
multiline string.'''
```

Chaînes multilignes avec triple guillemets doubles

- Les triple guillemets doubles (""" """) sont également utilisés pour définir des chaînes de caractères multilignes.
- Exemple :

```
multiline_text_double_quote_string =
"""Hello,
This is another
multiline string."""
```

Le Mode Raw pour les Chaînes de Caractères

- Le mode raw est un mode spécial pour les chaînes de caractères en Python.
- En utilisant le mode raw, Python ne traite pas les séquences d'échappement spéciales dans les chaînes de caractères.
- Pour activer le mode raw, préfixez la chaîne de caractères avec un `r` (minuscule ou majuscule).
- Utile lors de la manipulation de chaînes de caractères qui contiennent de nombreux caractères d'échappement et que vous ne souhaitez pas que Python les interprète.
- Documentation officielle sur les Chaînes de Caractères

Exemple d'Utilisation du Mode Raw (1/3)

```python
# Example of using raw strings in Python

# Example of regular strings with escape sequences
# The line below will generate a SyntaxError:
# SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated \UXXXXXXXX escape
# string_1 : str = "C:\Users\username\Documents"
# print(string_1)
# To avoid the error, we can use raw strings
raw_string_1 : str = r"C:\Users\username\Documents"
print(raw_string_1)  # Output: C:\Users\username\Documents

# Example of strings with newline characters
string_2 : str = "Line 1\nLine 2\nLine 3"
print(string_2) # Output: Line 1
                #         Line 2
                #         Line 3
raw_string_2 : str = r"Line 1\nLine 2\nLine 3"
print(raw_string_2)  # Output: Line 1\nLine 2\nLine 3
```

Exemple d'Utilisation du Mode Raw (2/3)

```python
# Using raw strings with regular expressions
import re
pattern = r'\d+'
text = 'There are 10 apples and 20 oranges'
matches = re.findall(pattern, text)
print(matches)  # Output: ['10', '20']
```

Exemple d'Utilisation du Mode Raw (3/3)

```python
# Using raw strings with file paths
file_path = r"C:\Users\username\Documents\example.txt"
with open(file_path, 'r') as file:
content = file.read()
print(content)
```

Constructeurs de la classe `str`

- La classe `str` offre plusieurs constructeurs pour créer des objets de type chaîne de caractères.
- Voici quelques-uns des constructeurs les plus couramment utilisés :
  1. Constructeur vide : Crée une chaîne de caractères vide.
  2. Constructeur avec une chaîne de caractères : Crée une copie de la chaîne de caractères spécifiée.
  3. Constructeur avec un objet convertible en chaîne : Convertit l'objet spécifié en une chaîne de caractères.
  4. Constructeur avec une séquence : Crée une chaîne de caractères à partir de la séquence spécifiée.
- Documentation officielle des constructeurs de la classe `str`

Exemples d'Utilisation des Constructeurs de la classe `str`

```python
# Empty constructor
empty_string = str()
print("# Output: Empty string:", empty_string) # Output:

# Constructor with a string
original_string = "Hello"
copied_string = str(original_string)
print("# Output: Copy of the string:", copied_string) # Output: Hello"

# Constructor with an object convertible to a string
number = 42
converted_string = str(number)
print("# Output: String from a number:", converted_string) # Output: 42

# Constructor with a sequence
sequence = ['a', 'b', 'c']
sequence_string = str(sequence)
print("# Output: String from a sequence:", sequence_string) # Output: ['a', 'b', 'c']
```

Indiçage positif et négatif

- En Python, les chaînes de caractères peuvent être accédées en utilisant des indices positifs et négatifs.
- Les indices positifs commencent à 0 pour le premier caractère et augmentent de 1 pour chaque caractère suivant.
- Les indices négatifs commencent à -1 pour le dernier caractère et diminuent de 1 pour chaque caractère précédent.
- Exemple :

```
1 s = "hello"
2 print("First character:", s[0])
3 # Output : First character: h
4 print("Last character:", s[-1])
5 # Output : Last character: o
```

Tranche de valeurs (slice)

- La tranche de valeurs en Python vous permet d'accéder à une plage de caractères d'une chaîne de caractères.
- Syntaxe : `string[début:fin:pas]`
- Exemple :

```
1 s = "hello"
2 print("Slice:", s[1:4])
3 # Output : Slice: ell
```

Opérateurs +, *, et in

- Python propose plusieurs opérateurs pour la manipulation de chaînes de caractères.
- `+` est utilisé pour la concaténation de chaînes de caractères.
- `*` est utilisé pour la répétition de chaînes de caractères OU pour l'unpacking (voir l'exemple dans le frame suivant).
- `in` est utilisé pour vérifier si une sous-chaîne existe dans une chaîne de caractères.
- Exemple :

```
s1 = "hello"
s2 = "world"
print("Concatenated:", s1 + s2)         # Output: Concatenated: helloworld
print("Repeated:", s1 * 3)              # Output: Repeated: hellohellohello
print("Contains 'lo'?", 'lo' in s1)     # Output: Contains 'lo'? True
```

Itération

- Les chaînes de caractères en Python peuvent être itérées à l'aide d'une boucle `for`.
- Chaque caractère de la chaîne est parcouru individuellement.
- Exemple :

```
s = "hello"
for char in s:
    print(char)
# Or via index:
for i in range(len(s)):
    print(s[i])
# Or via * starred expression:
print(*s) # Output : h e l l o
```

Méthodes de manipulation des chaînes de caractères

Voici quelques méthodes de manipulation des chaînes de caractères en Python :

- `split()`: Divise une chaîne en une liste de sous-chaînes en fonction d'un séparateur spécifié.
- `replace()`: Remplace toutes les occurrences d'une sous-chaîne par une autre dans la chaîne.
- `lower()`: Convertit tous les caractères de la chaîne en minuscules.
- `upper()`: Convertit tous les caractères de la chaîne en majuscules.
- `strip()`: Supprime les espaces (ou d'autres caractères spécifiés) au début et à la fin de la chaîne.
- `join()`: Joindre les éléments d'une séquence en une chaîne, en les séparant par une chaîne délimiteur.

Exemples

```python
# Example of string manipulation methods in Python
s = "Hello, world!"
# Splitting the string into a list of words
words = s.split(",")
print("Split:", words)  # Output: Split: ['Hello', ' world!']
# Replacing 'world' with 'Python'
new_s = s.replace("world", "Python")
print("Replace:", new_s)  # Output: Replace: Hello, Python!
# Converting to lowercase
lower_s = s.lower()
print("Lowercase:", lower_s)  # Output: Lowercase: hello, world!
# Converting to uppercase
upper_s = s.upper()
print("Uppercase:", upper_s)  # Output: Uppercase: HELLO, WORLD!
# Stripping whitespace
stripped_s = s.strip()
print("Stripped:", stripped_s)  # Output: Stripped: Hello, world!
# Joining a list of words into a single string
words = ['Hello', 'world']
joined_s = ' '.join(words)
print("Joined:", joined_s)  # Output: Joined: Hello world
```

Les f-strings en Python

- Les f-strings (formatted string literals) ont été introduites dans Python 3.6 en tant que fonctionnalité puissante de formatage de chaînes.
- Elles permettent l'insertion directe d'expressions Python dans des littéraux de chaînes.
- Les f-strings sont créées en préfixant une chaîne avec le caractère 'f' ou 'F'.
- Les expressions Python sont placées entre accolades à l'intérieur de la chaîne.

Sortie formatée avec les f-strings

```python
# Example of formatted output with f-string
name = "Ali"
age = 30
print(f"Hello, {name}! You are {age} years old.")

# Explanation
# The f-string is a concise way to format strings in Python.
# It allows embedding expressions inside string literals,
# using curly braces {} to enclose the expressions.
# In this example, variables 'name' and 'age' are inserted
# into the string to create a personalized greeting.
```

Sortie formatée avec les f-strings - Suite

```python
# Example 2
decimal = 2
amount = 45.6789
print(f"The total amount is ${amount:.{decimal}f}.")
# Output: The total amount is $45.68.
# Equivalent .format() method:
print("The total amount is ${:.{}f}.".format(amount, decimal))

# Explanation
# The f-string also supports formatting options.
# In this example, the variable 'amount' is formatted
# as a floating-point number with two decimal places,
# creating a string that displays the total amount with proper currency formatting.
```

La méthode format() en Python

- La méthode format() est une autre méthode de formatage de chaînes disponible en Python.
- Elle utilise des espaces réservés, appelés dans la chaîne de format, auxquels des valeurs sont passées en tant qu'arguments à la méthode format().
- La méthode format() peut effectuer un formatage plus avancé, tel que spécifier la largeur et la précision des nombres.

Sortie formatée avec la méthode format()

```
# Example of formatted output with the format() method
item = "apple"
qty = 5
print("You have {} {}{}."
.format(qty, item, 's' if qty > 1 else ''))

# Explanation
# The format() method is another way to format strings.
# It uses placeholders enclosed in curly braces {} within the string.
# The values to be inserted are provided as arguments to the format() method.
# In this example, the quantity and item variables are inserted into the string,
# and 's' is included conditionally based on the quantity.
```

Sortie formatée avec la méthode format() - Suite

```python
# Example 2
price = 19.99
discount = 0.15
final_price = price * (1 - discount)
print("The final price after a {}% discount is ${:.2f}.".format(int(discount * 100), final_price))

# Explanation
# The format() method allows for more complex formatting.
# In this example, the final price after applying a discount
# is formatted to display the percentage discount and the final price
# with two decimal places.
```

Les tuples

Tuple

- Les tuples sont des structures de données immuables et ordonnées en Python. Ils peuvent contenir des éléments de différents types et sont définis par des parenthèses.
- **Exemple de constructeur en Python:**

```python
# Example of tuple constructor in Python
t = (1, 2, 3)
print("Tuple:", t)  # Output: Tuple: (1, 2, 3)
```

Tuple - Constructeur

- En Python, les tuples peuvent être créés à l'aide du constructeur `tuple()` en spécifiant les éléments à inclure dans le tuple. Les éléments peuvent être de n'importe quel type de données.
- **Exemple en Python:**

```python
# Example of tuple constructor in Python
t = tuple([1, 2, 3, 4, 5])
print("Tuple:", t)  # Output: Tuple: (1, 2, 3, 4, 5)
```

Tuple - Indiçage

- Les tuples peuvent être indéxés pour accéder à leurs éléments individuels. L'indiçage commence à partir de 0 pour accéder aux éléments du tuple.
- **Exemple d'indiçage en Python**:

```
# Example of indexing in tuple in Python
t = (1, 2, 3)
print("First element:", t[0])  # Output: First element: 1
print("Last element:", t[-1])   # Output: Last element: 3
```

Tuple - Itération

- Les tuples peuvent être itérés à l'aide d'une boucle `for`, ce qui permet d'accéder à chaque élément du tuple.
- **Exemple d'itération en Python:**

```python
# Example of iteration over tuple in Python
t = (1, 2, 3)
for element in t:
    print(element)
```

Tuple - Opérateurs +, * et in

- Les tuples supportent les opérateurs + et *, ainsi que l'opérateur `in` pour la vérification d'appartenance.
- **Exemple d'opérateurs en Python:**

```python
# Example of operators in tuple in Python
t1 = (1, 2, 3)
t2 = (4, 5, 6)
print("Concatenated tuple:", t1 + t2)
# Output: Concatenated tuple: (1, 2, 3, 4, 5, 6)
print("Repeated tuple:", t1 * 2)
# Output: Repeated tuple: (1, 2, 3, 1, 2, 3)
print("Check if 2 in tuple:", 2 in t1)
# Output: Check if 2 in tuple: True
```

Tuple - Méthodes `count()` et `index()`

- Les tuples prennent en charge les méthodes `count()` et `index()` pour compter le nombre d'occurrences d'un élément et pour trouver l'index de la première occurrence d'un élément donné, respectivement.
- **Exemple de méthodes en Python:**

```
1 # Example of methods in tuple in Python
2 t = (1, 2, 2, 3, 4)
3 print("Count of 2:", t.count(2))     # Output: Count of 2: 2
4 print("Index of 3:", t.index(3))     # Output: Index of 3: 3
```

La classe `Tuple` en Python

- Un tuple est une collection de valeurs qui est **ordonnée** et **immutable**.
- Les tuples sont créés à l'aide de parenthèses `()` et chaque valeur est séparée par une virgule :

```
1 # Examples of tuple manipulation
2 colors = ("red", "green", "blue")
3 print(colors[1])  # Result: "green"
```

- Vous pouvez **accéder** aux éléments individuels d'un tuple en utilisant leur **indice**, mais **vous ne pouvez pas modifier** les éléments eux-mêmes. Les tuples sont utiles pour représenter des collections **fixes de valeurs**.
- Documentation officielle de la classe Tuple.
- Documentation GFG des méthodes de la classe Tuple.

Méthodes importantes de la classe `tuple`

- La classe `tuple` offre quelques méthodes pour travailler avec les tuples, telles que `count()` et `index()`.
- Ces méthodes permettent de compter le nombre d'occurrences d'un élément dans un tuple et de trouver l'index de la première occurrence d'un élément donné.
- Documentation officielle des méthodes tuple

Ordonnancement de la classe tuple

```python
# Extended example showing the ordering of the tuple class in Python
t = (1, 2, 3, 4, 5)
print("Original:", t)

# Access by positive and negative index
print("First element:", t[0])  # Displays the first element 1
print("Second element:", t[1])  # Displays the second element 2
print("Last element:", t[-1])  # Displays the last element 5

# Using a for loop to iterate over each element with index
print("Traversal with a for loop and indices:")
for i in range(len(t)):
    print("Element at index", i, ":", t[i])

# Slicing with positive and negative indices
print("Slicing with positive and negative indices:")
print("First three elements:", t[:3])  # Displays (1, 2, 3)
print("Last three elements:", t[-3:])  # Displays (3, 4, 5)

# Using a for loop to iterate over each element
print("Traversal with a for loop:")
for element in t:
    print(element)
```

Principe du depacking de variables

- Principe du depacking de variables, permet d'extraire les valeurs contenues dans une structure de données complexe (comme un tuple ou une liste) et de les assigner à des variables individuelles.
- **Exemple en Python**:

```python
# Example of variable unpacking in Python
coordinates = (3, 4)
x, y = coordinates
print("x:", x, "y:", y)  # Output: x: 3 y: 4
```

Les frozensets

Frozenset

- Les objets de type `frozenset` en Python sont des ensembles immuables, ce qui signifie qu'une fois créés, leurs éléments ne peuvent pas être modifiés.
- **Exemple de constructeur en Python:**

```
# Example of frozenset constructor in Python
fs = frozenset([1, 2, 3])
print("Frozenset:", fs)  # Output: Frozenset: frozenset({1, 2, 3})
```

Tableau d'octets (bytes)

Tableau d'octets (bytes)

- Les objets de type `bytes` en Python sont utilisés pour stocker une séquence d'octets. Ils sont immuables et peuvent contenir des valeurs dans la plage de 0 à 255.
- **Exemple de constructeur en Python**:

```python
# Example of bytes constructor in Python
b = bytes([65, 66, 67])
print(f'Bytes object: {b}')  # Output: Bytes object: b'ABC'
```

Objet None et fonction `repr()`

- L'objet `None` est utilisé pour représenter la nullité ou l'absence de valeur. La fonction `repr()` renvoie une représentation sous forme de chaîne de caractères de l'objet passé en argument.
- **Exemple en Python**:

```
1 # Example of None object and repr() function in Python
2 x = None
3 print("None object:", x)
4 # Output: None object: None
5 print("Representation of x:", repr(x))
6 # Output: Representation of x: None
```

Structures de données mutables

Les listes (list)

La classe `List` en Python

- Une liste est une collection de valeurs qui est **ordonnée** et **mutable**.
- Les listes sont créées à l'aide de crochets `[]` et chaque valeur est séparée par une virgule :

```python
# Example of list manipulation
fruits = ["apple", "banana", "orange"]
print(fruits[0])  # Result: "apple"
fruits.append("grape")
print(fruits)  # Result: ["apple", "banana", "orange", "grape"]
```

- Vous pouvez **accéder** et **modifier** des éléments individuels d'une liste en utilisant leur **indice**. Vous pouvez également **ajouter** ou **supprimer** des éléments à l'aide de méthodes intégrées telles que `append()`, `insert()`, `remove()` et `pop()`.
- Documentation officielle de la classe List

Méthodes importantes de la classe `liste`

- La classe `liste` offre quelques méthodes pour travailler avec les listes, telles que `append()`, `extend()`, `insert()`, `remove()`, `pop()`, `clear()`, `index()`, `count()` et `sort()`.
- Ces méthodes permettent d'ajouter des éléments à une liste, d'étendre une liste avec les éléments d'une autre liste, d'insérer un élément à une position donnée, de supprimer un élément spécifique, de retirer et retourner le dernier élément, de vider la liste, de trouver l'index de la première occurrence d'un élément, de compter le nombre d'occurrences d'un élément et de trier la liste.
- Documentation officielle des méthodes liste

Ordonnancement de la classe liste

```python
# Extended example showing list indexing and ordering in Python
l = [1, 2, 3, 4, 5]
print("Original:", l)

# Access using positive and negative indices
print("First element:", l[0])   # Displays the first element 1
print("Second element:", l[1])  # Displays the second element 2
print("Last element:", l[-1])   # Displays the last element 5

# Using a for loop to iterate through each element by index
print("Iterating with a for loop using indices:")
for i in range(len(l)):
    print("Element at index", i, ":", l[i])

# Slicing with positive and negative indices
print("Slicing with positive and negative indices:")
print("First three elements:", l[:3])   # Displays [1, 2, 3]
print("Last three elements:", l[-3:])   # Displays [3, 4, 5]

# Using a for loop to iterate directly through each element
print("Iterating with a for loop directly through elements:")
for element in l:
    print(element)
```

Listes - Constructeur

- En Python, les listes peuvent être créées à l'aide du constructeur `list()` ou `[]` en spécifiant les éléments à inclure dans la liste. Les éléments peuvent être de n'importe quel type de données.
- **Exemple en Python**:

```python
# Example of list constructor in Python
l = list([1, 2, 3, 4, 5])
print("List:", l)  # Output: List: [1, 2, 3, 4, 5]
anotherl = [1, 2, 3, 4, 5]
print("List:", anotherl)  # Output: List: [1, 2, 3, 4, 5]
```

Listes - Indiçage

- Les listes peuvent être indéxées pour accéder à leurs éléments individuels. L'indiçage commence à partir de 0 pour accéder aux éléments de la liste.
- **Exemple d'indiçage en Python**:

```python
# Example of indexing in list in Python
l = [1, 2, 3]
print("First element:", l[0])  # Output: First element: 1
print("Last element:", l[-1])   # Output: Last element: 3
```

Listes - Itération

- Les listes peuvent être itérées à l'aide d'une boucle `for`, ce qui permet d'accéder à chaque élément de la liste.
- **Exemple d'itération en Python:**

```
# Example of iteration over list in Python
l = [1, 2, 3]
# foreach loop-like
for element in l:
    print(element)
# for loop \textit{via} indexes
print("Traditional for loop:")
for i in range(len(l)):
    print("Element at index ", i, ": ", l[i])
```

Listes - Opérateurs +, * et in

- Les listes supportent les opérateurs + et *, ainsi que l'opérateur `in` pour la vérification d'appartenance.
- **Exemple d'opérateurs en Python:**

```python
# Example of operators in list in Python
l1 = [1, 2, 3]
l2 = [4, 5, 6]
print("Concatenated list:", l1 + l2)
# Output: Concatenated list: [1, 2, 3, 4, 5, 6]
print("Repeated list:", l1 * 2)
# Output: Repeated list: [1, 2, 3, 1, 2, 3]
print("Check if 2 in list:", 2 in l1)
# Output: Check if 2 in list: True
print(*l1)
# Output: 1 2 3
print(*l2)
# Output: 4 5 6
```

Listes - Méthodes et fonctions

- Les listes prennent en charge diverses méthodes pour la manipulation des éléments, y compris l'ajout, l'insertion, la suppression, le tri, l'inversion, etc.
- Voici la Documentation officielle des méthodes de la classe List.

Listes - Méthodes et fonctions - Exemple

- **Exemple de méthodes en Python:**

```
# Example of methods in list in Python
l = [1, 2, 3, 4]
l.append(5)               # Output: [1, 2, 3, 4, 5]
l.insert(2, 10)           # Output: [1, 2, 10, 3, 4, 5]
del l[1]                  # Output: [1, 10, 3, 4, 5]
l.sort()                  # Output: [1, 3, 4, 5, 10]
l.reverse()               # Output: [10, 5, 4, 3, 1]
l.remove(3)               # Output: [10, 5, 4, 1]
l.extend([6, 7, 8])       # Output: [10, 5, 4, 1, 6, 7, 8]
l.pop()                   # Output: [10, 5, 4, 1, 6, 7]
l.clear()                 # Output: []
```

Listes - Copie et références

- Les listes peuvent être copiées d'une manière **superficielle** en utilisant l'opérateur d'assignation =, via l'opérateur d'indiçage `:`, ou encore la méthode `copy()`, tandis que les copies **en profondeur** nécessitent la fonction `deepcopy()` du module `copy`.
- **Exemple de copies en Python:**

```
1 # Example of copying lists in Python
2 import copy # for function deepcopy()
3 l1 = [1, 2, 3]
4 l2 = l1
5 l3 = l1[:]
6 l4 = l1.copy()
7 l5 = copy.deepcopy(l1)
```

Listes - Copie et références - Exemple

```python
import copy

# Creating an original list
l1 = [1, 2, 3]

# Copies of the original list
l2 = l1
l3 = l1[:]
l4 = l1.copy()
l5 = copy.deepcopy(l1)
lists = [l1, l2, l3, l4, l5]

def print_lists():
    for list in lists:
        print(list)

print("# Before modification of l1")
print_lists()

# Modifying l1
l1.append(4)

print("# After modification of l1")
print_lists()
```

Listes - Différence entre `copy` et `deepcopy`

- La différence entre les méthodes `copy` et `deepcopy` réside dans la manière dont elles traitent les objets imbriqués lors de la copie de listes en Python.
- La méthode `copy` effectue une copie superficielle, créant une nouvelle liste mais partageant les objets imbriqués avec l'original. En revanche, la fonction `deepcopy` réalise une copie en profondeur, dupliquant tous les objets imbriqués pour créer une copie indépendante de l'original.
- Cet exemple illustre les propos discutés.

Fonction `sorted()`

- La fonction `sorted()` peut être utilisée pour trier une liste et renvoyer une nouvelle liste triée.
- **Exemple de tri en Python**:

```python
# Example of sorting list in Python
l = [3, 1, 2, 5, 4]
sorted_list = sorted(l)
print("Sorted list:", sorted_list)
# Output: Sorted list: [1, 2, 3, 4, 5]
```

Principe de fonctionnement des objets itérables

- Les objets itérables sont des éléments essentiels en Python.
- Ils permettent de parcourir séquentiellement les éléments d'une collection de données.
- Les objets itérables implémentent le protocole d'itération.
- Ils peuvent être parcourus avec une boucle `for` ou avec les fonctions `iter()` et `next()`.
- `__iter__()`: Renvoie un itérateur sur l'objet.
- `__next__()`: Récupère le prochain élément de l'objet itérable.
- En comprenant le principe de fonctionnement des objets itérables et en les utilisant correctement, l'on écrit ainsi un code plus efficace et plus lisible pour manipuler les collections de données dans les programmes.

Exemple de fonctionnement des objets itérables

```python
class MyIterable:
    def __init__(self, max):
        self.max = max
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.current < self.max:
            self.current += 1
            return self.current
        else:
            raise StopIteration

# Using the iterable
my_iterable = MyIterable(5)
for element in my_iterable:
    print(element)
```

Constructeur reversed()

- Le constructeur `reversed()` intégré en Python est utilisé pour inverser l'ordre des éléments d'une séquence.
- Il prend un objet séquentiel (comme une liste, un tuple ou une chaîne de caractères) en tant qu'argument et renvoie un itérateur inversé.
- L'itérateur renvoyé par `reversed()` parcourt les éléments de la séquence de la fin au début.
- Usage: `reversed(sequence)`
- Paramètres:
  - `sequence`: La séquence à inverser.
- Documentation: Documentation officielle de la fonction reversed

Exemples du constructeur reversed()

```python
# Example with a string
my_string = "Hello"
my_string_reversed = reversed(my_string)
print(type(my_string_reversed))
# Output: <class 'reversed'>
for char in my_string_reversed:
    print(char)

# Example with a tuple
my_tuple = (1, 2, 3, 4, 5)
my_tuple_reversed = reversed(my_tuple)
print(type(my_tuple_reversed))
# Output: <class 'reversed'>
for value in my_tuple_reversed:
    print(value)

# Example with a list
my_list = [1, 2, 3, 4, 5]
my_list_reversed = reversed(my_list)
print(type(my_list_reversed))
# Output: <class 'list_reverseiterator'>
for element in my_list_reversed:
    print(element)
```

Remarque sur la différence de types d'itérateurs inversés

- Lorsque vous utilisez la fonction `reversed()` en Python pour inverser les éléments d'une séquence, le type de l'itérateur retourné peut varier en fonction du type de séquence d'entrée.
  - Pour les tuples, `reversed()` renvoie un objet itérateur de type `reversed`, spécifiquement conçu pour parcourir les éléments du tuple dans l'ordre inverse.
  - Pour les listes, `reversed()` renvoie un objet itérateur de type `list_reverseiterator`, qui offre des fonctionnalités spécifiques aux listes en plus de permettre l'itération dans l'ordre inverse.
- Il est important de noter cette différence de types, car cela peut affecter le comportement et les opérations que vous pouvez effectuer sur l'itérateur inversé, en fonction du type de la séquence d'origine.

Opérations spécifiques aux listes pour l'itérateur inversé

- L'objet itérateur de type `list_reverseiterator`, retourné par la fonction `reversed()` pour les listes, prend en charge certaines opérations spécifiques aux listes en plus de permettre l'itération dans l'ordre inverse.
  - Il est possible de modifier la liste d'origine pendant l'itération sans générer d'erreur.
  - On peut accéder aux éléments de la liste par leur indice pendant l'itération.
  - Il est également possible d'utiliser les méthodes spécifiques aux listes telles que `append()`, `extend()`, `pop()`, `remove()`, etc., pendant l'itération.
  - Enfin, l'itérateur peut être converti à nouveau en une liste à l'aide du constructeur `list()`.
- Pour plus d'informations sur ces opérations, consultez la documentation officielle.

Opérations spécifiques aux listes pour l'itérateur inversé - Exemples

```python
# Example of modifying the original list during iteration
original_list = [1, 2, 3, 4, 5]
for element in reversed(original_list):
    original_list.append(element * 2)
print(original_list)  # Output: [1, 2, 3, 4, 5, 10, 8, 6, 4, 2]

# Example of accessing elements by index during iteration
for index, element in enumerate(reversed(original_list)):
    print(f"Index: {index}, Element: {element}")

# Example of using list-specific methods during iteration
for element in reversed(original_list):
    if element % 2 == 0:
        original_list.remove(element)
print(original_list)  # Output: [1, 3, 5]

# Example of converting the iterator back to a list
list_reverseiterator_object = reversed(original_list)
reversed_list = list(list_reverseiterator_object)
print(reversed_list)  # Output: [5, 3, 1]
```

Limitations de reversed() sur les ensembles et les dictionnaires

- Les ensembles (sets) en Python sont des collections non ordonnées d'éléments uniques.
- Les dictionnaires (dictionaries) en Python sont des collections de paires clé-valeur non ordonnées.
- Les ensembles et les dictionnaires ne garantissent pas d'ordre spécifique pour leurs éléments.
- Les paires clé-valeur dans un dictionnaire et les éléments dans un ensemble sont stockés de manière non séquentielle.
- Par conséquent, il n'a pas de sens de "renverser" un ensemble ou un dictionnaire car ils n'ont pas de notion intrinsèque d'ordre pour leurs éléments.

L'objet `range`

- `range` n'est pas une fonction, mais une **classe intégrée** (*built-in class*) en Python.
- Elle crée un **objet itérable** représentant une séquence d'entiers.
- Cet objet ne stocke pas directement les valeurs mais les **génère à la volée** (paresseusement).
- Très utilisée avec les boucles `for` pour parcourir des séquences d'entiers.
- Constructeur : `range(start, stop, step)`
- Paramètres :
  - `start` : Valeur de départ (par défaut : 0)
  - `stop` : Valeur de fin (non incluse)
  - `step` : Incrément (par défaut : 1)
- Documentation officielle : `func-range`

Exemples de l'objet range()

```python
def print_line():
    print("----------------------------")

# Usage with only stop parameter (start defaults to 0, step defaults to 1)
for i in range(5):
    print(i)
print_line()

# Usage with specified start and specified stop (start defaults to 0, step defaults to 1)
for j in range(2,6):
    print(j)
print_line()

# Usage with specified start, stop, and step parameters
for k in range(1, 8, 2):
    print(k)
print_line()

# Usage with a negative step to generate a decreasing sequence
for l in range(6, 0, -1):
    print(l)
print_line()
```

Exemples de l'objet range() avec Opérateur *

```python
def print_line():
    print("----------------------------")

# Usage with only stop parameter (start defaults to 0, step defaults to 1)
R1 = range(5)
print(*R1)
print_line()

# Usage with specified start and specified stop (start defaults to 0, step defaults to 1)
R2 = range(2,6)
print(*R2)
print_line()

# Usage with specified start, stop, and step parameters
R3 = range(1, 8, 2)
print(*R3)
print_line()

# Usage with a negative step to generate a decreasing sequence
R4 = range(6, 0, -1)
print(*R4)
print_line()
```

Exemples de l'objet range() avec Opérateur _

```python
def print_line():
    print("----------------------------")

# Example using the range() function to generate a sequence of numbers
for _ in range(4):
    print("Hello")
print_line()

# Example using the range() function with specified start and step parameters
for _ in range(1, 6, 2):
    print("Ignored")
print_line()

# Example using the range() function with only the stop parameter specified
for _ in range(5):
    print("Skipped")
print_line()

# Example using the range() function to generate an empty sequence
for _ in range(0):
    print("No output")
print_line()
```

Exemples de l'objet range() avec Opérateurs * et _

```python
# Usage with the * operator
n = 5
for i in range(*[n]):
    print(i)

# Usage with the * operator to unpack a list
start_stop = [1, 10]
for j in range(*start_stop):
    print(j)

# Usage with the * operator to unpack a tuple
params = (10, 20, 2)
for k in range(*params):
    print(k)

# Usage with the _ operator to ignore a value
for _ in range(3):
    print("Hello")
```

Constructeur `reversed()` et `range()`

- Comme nous l'avons vu, les constructeurs `reversed()` et `range()` sont donc utilisés pour créer des objets itérables en Python.
- **Exemple d'utilisation en Python**:

```
# Example of iterable objects in Python
r = range(5)
print("Range:", list(r))
# Output: Range: [0, 1, 2, 3, 4]
reversed_r = reversed(r)
print("Reversed range:", list(reversed_r))
# Output: Reversed range: [4, 3, 2, 1, 0]
```

Reverse d’un string

- La fonction `reversed()` peut également être utilisée pour inverser l’ordre des caractères dans une chaîne de caractères (string) en Python.
- Voici toutes les manières de faire afin d’inverser une chaîne de caractères en Python : reverse string

D’avantage d’exemples

- Depacking, range et opérateurs * et _.
- Range et opérateur _.
- Listes et opérateur _.
- Fonction reversed et structures de données.
- Fonction reversed et listes.
- Opérateurs * et _, fonction range et structures de données.

Dictionnaires - Constructeur

- En Python, les dictionnaires peuvent être créés à l'aide du constructeur `dict()` en spécifiant les paires clé-valeur.
- **Exemple en Python:**

```python
# Example of dictionary constructor in Python
d = dict({'a': 1, 'b': 2, 'c': 3})
print("Dictionary:", d)
# Output: Dictionary: {'a': 1, 'b': 2, 'c': 3}
```

Dictionnaires - Indiçage

- Les dictionnaires peuvent être indéxés pour accéder à leurs éléments par clé.
- **Exemple d'indiçage en Python**:

```
1 # Example of indexing in dictionary in Python
2 d = {'a': 1, 'b': 2, 'c': 3}
3 print("Value of key 'b':", d['b'])
4 # Output: Value of key 'b': 2
```

Dictionnaires - Opérateur `in`, fonction `del()`

- Les dictionnaires prennent en charge l'opérateur `in` pour la vérification de l'existence de clés et la fonction `del()` pour la suppression d'éléments par clé.
- **Exemple en Python:**

```python
# Example of operators and functions in dictionary in Python
d = {'a': 1, 'b': 2, 'c': 3}
print("Check if key 'b' exists:", 'b' in d)
# Output: Check if key 'b' exists: True
del d['b']
print("Dictionary after deletion:", d)
# Output: Dictionary after deletion: {'a': 1, 'c': 3}
print(*d)
# Output: a c
```

Dictionnaires - Méthodes `keys()`, `values()`, `items()`, `update()`, `get()`

- Les dictionnaires fournissent diverses méthodes pour accéder aux clés, aux valeurs, aux paires clé-valeur, mettre à jour, et obtenir des valeurs par clé.
- **Exemple de méthodes en Python:**

```python
# Example of methods in dictionary in Python
d = {'a': 1, 'b': 2, 'c': 3}
print("Keys:", d.keys())       # Output: Keys: dict_keys(['a', 'b', 'c'])
print("Values:", d.values())   # Output: Values: dict_values([1, 2, 3])
print("Items:", d.items())     # Output: Items: dict_items([('a', 1), ('b', 2), ('c', 3)])
d.update({'d': 4})
print("Updated dictionary:", d)  # Output: Updated dictionary: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
print("Value for key 'a':", d.get('a'))  # Output: Value for key 'a': 1
```

Dictionnaires - Copie et références

- Les dictionnaires peuvent être copiés d'une manière **superficielle** en utilisant l'opérateur d'assignation =, via la méthode `copy()`, ou encore la méthode `dict()`, tandis que les copies **en profondeur** nécessitent la fonction `deepcopy()` du module `copy`.
- **Exemple de copies en Python:**

```
1 # Example of copying dictionaries in Python
2 import copy # for function deepcopy()
3 d1 = {'a': 1, 'b': 2, 'c': 3}
4 d2 = d1
5 d3 = d1.copy()
6 d4 = dict(d1)
7 d5 = copy.deepcopy(d1)
```

Dictionnaires - Copie et références - Exemple

```
import copy

# Creating an original dictionary
d1 = {'a': 1, 'b': 2, 'c': 3}

# Copies of the original dictionary
d2 = d1
d3 = d1.copy()
d4 = copy.deepcopy(d1)
dicts = [d1, d2, d3, d4]

def print_dicts():
    for dict in dicts:
        print(dict)

# Before modifying d1
print("# Before modifying d1")
print_dicts()

# Modifying d1
d1['d'] = 4

# After modifying d1
print("# After modifying d1")
print_dicts()
```

Dictionnaires - Différence entre `copy` et `deepcopy`

- La différence entre les méthodes `copy` et `deepcopy` réside dans la manière dont elles traitent les objets imbriqués lors de la copie de dictionnaires en Python.
- La méthode `copy` effectue une copie superficielle, créant un nouveau dictionnaire mais partageant les objets imbriqués avec l'original. En revanche, la fonction `deepcopy` réalise une copie en profondeur, dupliquant tous les objets imbriqués pour créer une copie indépendante de l'original.
- Cet exemple illustre les propos discutés.

La classe `Dict` en Python

- Un dictionnaire est une collection de paires `clé-valeur` qui est **non ordonnée** et **mutable**.
- Les dictionnaires sont créés à l'aide d'accolades `{}` et chaque paire `clé-valeur` est séparée par deux-points `:` ,illustration:

```python
# Example of dictionary manipulation
person = {"name": "Ali", "age": 30, "city": "MTP"}
print(person["name"])  # Result: "Ali"
person["age"] = 31
print(person)  # Result: {"name": "Ali", "age": 31, "city": "MTP"}
```

- Vous pouvez **accéder** et **modifier** des valeurs dans un dictionnaire en utilisant leurs **clés**. Vous pouvez également **ajouter** ou **supprimer** des paires `key-value` à l'aide de méthodes intégrées telles que `update()`, `pop()` et `copy()`.

Set - Constructor

- En Python, les ensembles peuvent être créés en utilisant le constructeur `set()` en passant un itérable contenant les éléments à inclure dans l'ensemble.
- Alternativement, les ensembles peuvent également être créés en utilisant des accolades {} en listant les éléments séparés par des virgules.
- **Exemple en Python**:

```python
# Example of set constructor in Python
set1 = set([1, 2, 3, 4, 5])
print("Set:", set1)  # Output: Set: {1, 2, 3, 4, 5}
another_set = {1, 2, 3, 4, 5}
print("Set:", another_set)  # Output: Set: {1, 2, 3, 4, 5}
```

Set - Opérateurs - |, &, et ^

- Les ensembles supportent différents opérateurs tels que la différence (-), l'union (|), l'intersection (&), et la différence symétrique (^).
- Ces opérateurs permettent d'effectuer efficacement des opérations sur les ensembles.
- **Exemple des opérateurs en Python**:

```
# Example of set operators in Python
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print("Difference:", set1 - set2)
# Output: Difference: {1, 2}
print("Union:", set1 | set2)
# Output: Union: {1, 2, 3, 4, 5}
print("Intersection:", set1 & set2)
# Output: Intersection: {3}
print("Symmetric Difference:", set1 ^ set2)
# Output: Symmetric Difference: {1, 2, 4, 5}
```

La classe `Set` en Python

- Un ensemble est une collection de valeurs qui est **non ordonnée** et **unique**.
- Les ensembles sont créés à l'aide d'accolades `{}` ou de la fonction `set()` et chaque valeur est séparée par une virgule :

```python
# Example of set manipulation
fruits = {"apple", "banana", "orange"}
print("apple" in fruits)  # Result: True
fruits.add("grape")
print(fruits)  # Result: {"apple", "banana", "orange", "grape"}
```

- Vous pouvez vérifier si une valeur est dans un ensemble en utilisant l'opérateur `in`. Vous pouvez également **ajouter** ou **supprimer** des éléments à l'aide de méthodes intégrées telles que `add()`, `remove()` et `discard()`.

Compréhensions en Python

- Les compréhensions sont une fonctionnalité de Python permettant de créer de manière concise des listes, des ensembles et des dictionnaires à partir d'itérables. Cependant, elles ne sont pas disponibles pour les tuples en raison de leur immutabilité.
  - Les listes (`list`) sont des collections mutables, ce qui signifie que leurs éléments peuvent être modifiés après leur création. Les compréhensions permettent de créer rapidement des listes en appliquant une opération à chaque élément d'un itérable.
  - Les ensembles (`set`) sont également mutables, mais ils exigent l'unicité parmi leurs éléments. Les compréhensions sont utiles pour créer des ensembles car elles garantissent automatiquement l'unicité des éléments.
  - Les dictionnaires (`dict`) sont des collections de paires clé-valeur mutables. Les compréhensions pour les dictionnaires permettent de créer des dictionnaires de manière concise en itérant sur un itérable et en générant des paires clé-valeur.

Compréhensions en Python - Suite

- Suite:
  - Cependant, les tuples (`tuple`) sont des collections immuables. Étant donné que les compréhensions impliquent la création d'une nouvelle collection en itérant sur un itérable et en appliquant une opération à chaque élément, il n'y a pas de moyen direct d'appliquer des compréhensions aux tuples.
- Bien que vous puissiez utiliser des expressions de générateur pour obtenir des résultats similaires avec les tuples, elles ne possèdent pas la même syntaxe que les compréhensions. Les expressions de générateur produisent un objet générateur, qui peut être utilisé pour évaluer les éléments de manière différente.

Liste en intension (comprehension list)

- Les compréhensions de listes sont des constructions syntaxiques concises et élégantes pour la création de listes en Python.
- Elles permettent de créer rapidement et efficacement des listes en appliquant une expression à chaque élément d'une séquence.
- Les compréhensions de listes sont écrites entre crochets `[ ]`.
- Syntaxe :
  - `[expression for item in iterable if condition]`
  - `[expression_if_true if condition else expression_if_false for item in iterable]`

Exemples de liste en intension (comprehension list)

```python
# Example 1: List of squares using list comprehension
squares_list = [x**2 for x in range(10)]
print(squares_list)
# Output: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
print(type(squares_list))
# Output: <class 'list'>


# Example 2: List of squares for even numbers, cubes for odd numbers
squares_or_cubes_list = [x**2 if x % 2 == 0 else x**3 for x in range(10)]
print(squares_or_cubes_list)
# Output: [0, 1, 4, 27, 16, 125, 36, 343, 64, 729]
print(type(squares_or_cubes_list))
# Output: <class 'list'>
```

Ensemble en intension (comprehension set)

- Les compréhensions d'ensembles sont des constructions syntaxiques concises et élégantes pour la création d'ensembles en Python.
- Elles permettent de créer rapidement et efficacement des ensembles en appliquant une expression à chaque élément d'une séquence.
- Les compréhensions d'ensembles sont écrites entre accolades `{ }`.
- Syntaxe : `{expression for item in iterable if condition}`

```
# Example of set comprehension
squares_comprehension_set = {x**2 for x in range(10)}
print(squares_comprehension_set)
# Output: {0, 1, 64, 4, 36, 9, 16, 49, 81, 25}
print(type(squares_comprehension_set))
# Output: <class 'set'>
```

Dictionnaire en intension (comprehension dict)

- Les compréhensions de dictionnaires sont des constructions syntaxiques concises et élégantes pour la création de dictionnaires en Python.
- Elles permettent de créer rapidement et efficacement des dictionnaires en appliquant une expression à chaque paire clé-valeur d'une séquence.
- Syntaxe : `{key_expression: value_expression for item in iterable if condition}`

```
# Example of dict comprehension
squares_comprehension_dict = {x: x**2 for x in range(10)}
print(squares_comprehension_dict)
# Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
print(type(squares_comprehension_dict))
# Output: <class 'dict'>
```

Tuple en intension (comprehension tuple)

- Comme évoqué, la notion de compréhension de tuple n'existe pas à proprement parler, cependant en voulant appliquer une compréhension l'on obtient un objet de type `generator`
- Syntaxe : `(expression for item in iterable if condition)`

```
# Example of tuple comprehension
squares_comprehension_tuple = (x**2 for x in range(10))
# Works fine, but the result is not of type tuple
print(squares_comprehension_tuple)
# Output: <generator object <genexpr> at 0x104deb970>
print(*squares_comprehension_tuple)
# Output: 0 1 4 9 16 25 36 49 64 81
print(type(squares_comprehension_list))
# Output: <class 'generator'>
```

Tableau sur les structures de données

Data Structure	Unique	Assignable	Ordered	Mutable	Hashable	Copiable
STR						
TUPLE						
LIST						
SET						
FROZEN SET						
DICT						

- Remarques :
  - Les dictionnaires possèdent des clés uniques, tandis que les valeurs peuvent être les mêmes pour différentes clés.
  - Subscriptible veut dire ordered.
  - Voici un script python illustrant l'ensemble de ces concepts : data_structures_summary.py.

Relations

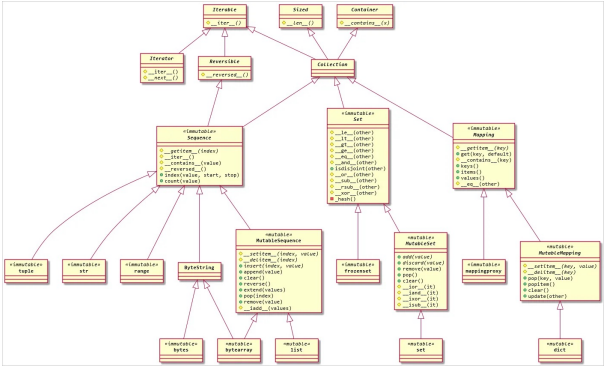


Figure: DataStruct Class Diagram

** Modules de la bibliothèque standard **

Interaction avec l'interpréteur : module `sys`

- Le module `sys` fournit des fonctionnalités permettant d'interagir avec l'interpréteur Python.
- Il permet d'accéder à des informations sur l'environnement d'exécution, de manipuler le chemin de recherche des modules, et plus encore.

Exemple d'utilisation

```python
import sys

# Affichage de la version de Python
print("Python version:", sys.version)

# Affichage du chemin de recherche des modules
print("Module search path:", sys.path)
```

Interaction avec le système d'exploitation : modules `os` et `pathlib`

- Les modules `os` et `pathlib` fournissent des fonctionnalités pour interagir avec le système d'exploitation et manipuler les chemins de fichiers de manière efficace.
- Le module `os` permet d'exécuter des opérations système telles que la création de répertoires, la navigation dans le système de fichiers, etc.
- Le module `pathlib` fournit une interface orientée objet pour manipuler les chemins de fichiers et de répertoires de manière portable.

Exemple d'utilisation

```python
import os
from pathlib import Path

# Création d'un répertoire
os.makedirs("mydir")

# Récupération du répertoire courant
current_dir = Path.cwd()
print("Current directory:", current_dir)

# Vérification de l'existence d'un fichier
file_path = Path("myfile.txt")
if file_path.exists():
    print("File exists!")
else:
    print("File does not exist.")
```

Interaction avec le système de fichiers : module `os.path`

- Le module `os.path` fournit des fonctionnalités pour manipuler les chemins de fichiers de manière efficace.
- Il permet de vérifier l'existence de fichiers, de répertoires, de manipuler les extensions de fichier, etc.

Exemple d'utilisation

```python
import os.path

# Vérifier l'existence d'un fichier
file_path = "example.txt"
if os.path.exists(file_path):
    print("Le fichier existe:", file_path)
else:
    print("Le fichier n'existe pas:", file_path)

# Obtenir le répertoire parent d'un fichier
parent_directory = os.path.dirname(file_path)
print("Répertoire parent:", parent_directory)

# Obtenir l'extension d'un fichier
file_extension = os.path.splitext(file_path)[1]
print("Extension de fichier:", file_extension)
```

Expressions rationnelles : module `re`

- Le module `re` permet de travailler avec des expressions rationnelles en Python.
- Il fournit des fonctionnalités pour rechercher, extraire et manipuler des motifs de texte basés sur des modèles définis.

Module `datetime`

- Le module `datetime` fournit des classes pour manipuler des dates et des heures en Python.
- Il permet de créer, manipuler et formater des objets de date et d'heure.

Exemple d'utilisation

```python
from datetime import datetime

# Création d'un objet de date et d'heure
now = datetime.now()
print("Date and time:", now)

# Formatage de la date et de l'heure
formatted_date = now.strftime("%Y-%m-%d %H:%M:%S")
print("Formatted date and time:", formatted_date)

# Extraction des composants de la date et de l'heure
year = now.year
month = now.month
day = now.day
hour = now.hour
minute = now.minute
second = now.second
print("Year:", year)
print("Month:", month)
print("Day:", day)
print("Hour:", hour)
print("Minute:", minute)
print("Second:", second)
```

Module `math`

- Le module `math` fournit des fonctions mathématiques standard en Python.
- Il inclut des fonctions trigonométriques, logarithmiques, de puissance, etc.

Exemple d'utilisation

```python
import math

# Exemples de fonctions mathématiques
print("Sin(30 degrees):", math.sin(math.radians(30)))  # Output: 0.49999999999999994
print("Cos(45 degrees):", math.cos(math.radians(45)))  # Output: 0.7071067811865476
print("Tan(60 degrees):", math.tan(math.radians(60)))  # Output: 1.7320508075688767
print("Logarithm base 2 of 16:", math.log2(16))          # Output: 4.0
print("Square root of 25:", math.sqrt(25))               # Output: 5.0
print("Factorial of 5:", math.factorial(5))              # Output: 120
```

Module `timeit`

- Le module `timeit` est utilisé pour mesurer le temps d'exécution de petits fragments de code Python.
- Il fournit un moyen simple de comparer les performances de différentes implémentations.

Exemple d'utilisation

```python
import timeit

# Exemple de mesure du temps d'exécution
time_taken = timeit.timeit(stmt='[i**2 for i in range(1000)]', number=10000)
print("Time taken:", time_taken)
```

Module `urllib`

- Le module `urllib` est utilisé pour récupérer des données à partir d'URLs en Python.
- Il fournit des fonctionnalités pour ouvrir, lire et analyser des pages web.

Exemple d'utilisation

```python
import urllib.request

# Exemple de récupération de données à partir d'une URL
response = urllib.request.urlopen('https://www.example.com')
html = response.read()
print(html)
```

Module `collections`

- Le module `collections` fournit des classes spécialisées de conteneurs de données en Python.
- Il inclut des classes telles que `Counter`, `deque`, `namedtuple`, etc., qui étendent les fonctionnalités des types de données intégrés.

Exemple d'utilisation

```python
import collections

# Exemple d'utilisation de Counter
word_counts = collections.Counter(['apple', 'banana', 'apple', 'orange', 'banana'])
print("Word counts:", word_counts)
```

Module `csv`

- Le module `csv` est utilisé pour lire et écrire des fichiers CSV (Comma-Separated Values) en Python.
- Il fournit des fonctionnalités pour travailler avec des données tabulaires stockées dans des fichiers CSV.

Exemple d'utilisation

```python
import csv

# Exemple de lecture d'un fichier CSV
with open('data.csv', 'r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Module `json`

- Le module `json` est utilisé pour travailler avec des données JSON (JavaScript Object Notation) en Python.
- Il fournit des fonctionnalités pour sérialiser et désérialiser des objets Python en JSON et vice versa.

Exemple d'utilisation

```python
import json

# Exemple de sérialisation et désérialisation JSON
data = {'name': 'John', 'age': 30}
json_string = json.dumps(data)
print("JSON string:", json_string)

parsed_data = json.loads(json_string)
print("Parsed data:", parsed_data)
```

Module `sqlite3`

- Le module `sqlite3` est utilisé pour travailler avec des bases de données SQLite en Python.
- Il fournit des fonctionnalités pour exécuter des requêtes SQL, récupérer des données et gérer les transactions.

Exemple d'utilisation

```python
import sqlite3

# Exemple de création d'une base de données SQLite
conn = sqlite3.connect('example.db')
cursor = conn.cursor()

# Création d'une table
cursor.execute('''CREATE TABLE IF NOT EXISTS stocks
                  (date text, trans text, symbol text, qty real, price real)''')

# Insertion de données
cursor.execute("INSERT INTO stocks VALUES ('2024-04-30', 'BUY', 'GOOG', 100, 1225.12)")

# Sauvegarde des modifications
conn.commit()

# Fermeture de la connexion
conn.close()
```

** Exceptions **

Principe de fonctionnement

- Les exceptions sont des événements qui se produisent lors de l'exécution d'un programme et qui interrompent le flux normal d'exécution.
- En Python, les exceptions sont des objets qui représentent des erreurs ou des conditions anormales.
- Lorsqu'une exception se produit, le programme s'arrête et affiche un message d'erreur à l'utilisateur.

Exemple d'utilisation

```
try:
    # Code that may generate an exception
    result = 10 / 0
except ZeroDivisionError:
    # Handling of ZeroDivisionError exception
    print("Division by zero error!")
```

Exceptions pré-définies et arbre d'héritage

- Python fournit un certain nombre d'exceptions pré-définies pour gérer différents types d'erreurs.
- Ces exceptions sont organisées dans une hiérarchie d'héritage, où les exceptions plus spécifiques héritent des exceptions plus générales.

Exemple d'utilisation

```
try:
    # Code that may generate an exception
    file = open("example.txt", "r")
    content = file.read()
    file.close()
except FileNotFoundError:
    # Handling of FileNotFoundError exception
    print("File not found!")
```

Instructions `try ... except ... else ... finally`

- L'instruction `try ... except ... else ... finally` est utilisée pour gérer les exceptions de manière plus complexe.
- La clause `else` est exécutée si aucune exception n'est levée dans la clause `try`.
- La clause `finally` est exécutée quel que soit le résultat de la clause `try`, que des exceptions aient été levées ou non.

Exemple d'utilisation

```
try:
    # Code that may generate an exception
    file = open("example.txt", "r")
except FileNotFoundError:
    # Handling of FileNotFoundError exception
    print("File not found!")
else:
    # Code executed if no exception is raised
    print("File read successfully!")
finally:
    # Code executed anyways
    file.close()
```

Propagation des exceptions

- Lorsqu'une exception est levée dans une fonction, elle peut être propagée vers les fonctions appelantes jusqu'à ce qu'elle soit capturée.
- La propagation des exceptions permet de gérer les erreurs à différents niveaux de la hiérarchie d'appels de fonctions.

Exemple d'utilisation

```python
def read_file(filename):
    try:
        file = open(filename, "r")
        content = file.read()
        file.close()
        return content
    except FileNotFoundError:
        print("File not found!")
        raise

try:
    data = read_file("example.txt")
except FileNotFoundError:
    print("Error reading file!")
```

Déclenchement d'exceptions

- Les exceptions peuvent être déclenchées manuellement à l'aide de l'instruction `raise`.
- Cela permet de signaler des erreurs ou des conditions anormales dans le code.

Exemple d'utilisation

```python
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative!")

try:
    validate_age(-5)
except ValueError as e:
    print("Invalid age:", e)
```

Définition d'une exception

- En Python, il est possible de définir ses propres exceptions en créant de nouvelles classes dérivées de la classe de base `Exception`.
- Cela permet de personnaliser le comportement des exceptions et de les adapter aux besoins spécifiques de l'application.

Exemple d'utilisation

```python
class CustomError(Exception):
    pass

def process_data(data):
    if not data:
        raise CustomError("Empty data!")

try:
    process_data([])
except CustomError as e:
    print("Error:", e)
```

** Manipulation des fichiers **

Entrées/Sorties de fichiers en Python

Entrées/Sorties de fichiers en Python

- Les opérations d'entrée et de sortie sur les fichiers sont une partie essentielle de nombreux programmes.
- En Python, vous pouvez utiliser la fonction intégrée `open()` et la méthode `close()` pour travailler avec des fichiers.
- Pour lire des données à partir d'un fichier, vous pouvez utiliser la méthode `read()`.
- Pour écrire des données dans un fichier, vous pouvez utiliser la méthode `write()`.

Ouverture et fermeture de fichiers

- La fonction `open()` est utilisée pour ouvrir un fichier en mode lecture, écriture ou ajout.
- La méthode `close()` est utilisée pour fermer le fichier ouvert.

```
# Example of using open() and close()
file = open("example.txt", "r")
content = file.read()
file.close()
```

- **Usage:** L'ouverture et la fermeture de fichiers sont utilisées pour lire ou écrire dans des fichiers texte.
- **Documentation:** Documentation officielle de open(), Documentation officielle de la manipulation des fichiers

Lecture de fichiers ligne par ligne

- La méthode `readline()` est utilisée pour lire une ligne à la fois à partir du fichier.
- La méthode `readlines()` est utilisée pour lire toutes les lignes du fichier et les retourner dans une liste.
- Les fichiers eux-mêmes peuvent être utilisés comme objets itérables.

```
# Example of using readline() and readlines()
with open("example.txt", "r") as file:
    line = file.readline()
    lines = file.readlines()
```

- **Usage:** Ces méthodes sont utiles pour lire des fichiers texte ligne par ligne ou pour récupérer toutes les lignes dans une liste.
- **Documentation:**
  Documentation officielle de la manipulation des fichiers

Utilisation de l'instruction `with`

- L'instruction `with` est utilisée avec les fichiers pour garantir la fermeture automatique du fichier après son utilisation.
- Cela simplifie le code et évite les erreurs de gestion des ressources.

```
1 # Example of using the with instruction
2 with open("example.txt", "r") as file:
3     content = file.read()
```

- **Usage:** L'instruction `with` est recommandée pour la manipulation des fichiers car elle garantit la fermeture automatique du fichier après son utilisation.
- **Documentation:**
  Documentation officielle des méthodes d'objets de fichier

Travailler avec des fichiers en Python (1/5)

Par défaut, `open()` ouvre le fichier en mode texte, ce qui signifie qu'il lit et écrit des chaînes de caractères. Si vous souhaitez travailler avec des données binaires, vous pouvez utiliser la fonction open() avec le modificateur de mode `b` (`wb` ou `rb`). N'oubliez pas de toujours fermer vos fichiers lorsque vous avez terminé de travailler avec eux, car cela libère toutes les ressources système associées au fichier et contribue à éviter la corruption des données.

Travailler avec des fichiers en Python (2/5)

Travailler avec les chemins de fichiers en Python

- Lorsque vous travaillez avec des fichiers en Python, il est important de spécifier le chemin du fichier correct.
- Le chemin du fichier est l'emplacement du fichier sur votre ordinateur.
- Sous Windows, les chemins de fichiers utilisent le caractère barre oblique inverse \ (backslash) pour séparer les répertoires.
- Sous Linux et macOS, les chemins de fichiers utilisent le caractère barre oblique / (slash ou forward slash).

Travailler avec des fichiers en Python (3/5)

```
import os
'''     Use the os module in Python to work with file paths.
The os.path.join() method is a platform-independent way to join path components.    Here is an example of how
      to use os.path.join() to build a file path:       '''
file_path = os.path.join("mydirectory", "myfile.txt")
# Open the file
file = open(file_path, "r")
# Read the content of the file
data = file.read()
# Close the file
file.close()
# Display the data
print(data)
```

Travailler avec des fichiers en Python (4/5)

```python
import os
# Join path components to build a file path
file_path = os.path.join("mydirectory", "myfile.txt")
# Open the file
file = open(file_path, "r")
# Read the content of the file
data = file.read()
# Close the file
file.close()
# Display the data
print(data)
```

Travailler avec des fichiers en Python (5/5)

- Dans cet exemple, `os.path.join()` joint le nom du répertoire `mydirectory` et le nom du fichier `myfile.txt` pour construire le chemin du fichier `mydirectory/myfile.txt`.
- Cela est fait de manière indépendante de la plate-forme, de sorte que le code fonctionnera à la fois sur les systèmes Windows et Unix.
- En utilisant le module `os` et la fonction `open()`, vous pouvez travailler avec des fichiers et des chemins de fichiers de manière à la fois facile et fiable.

Ajouter des données à un fichier en Python

- En plus de lire et d'écrire des données dans un fichier, vous pouvez également ajouter des données à un fichier existant en Python. Cela vous permet d'ajouter de nouvelles données à la fin d'un fichier sans écraser son contenu existant.
- Pour ajouter des données à un fichier, vous pouvez utiliser le modificateur de mode `a` (pour `append`) lors de l'ouverture du fichier. Voici un exemple :

```
# Open the file in append mode
file = open("myfile.txt", "a")
# Write data to the file
file.write("Hello, again!")
# Close the file
file.close()
```

Ajouter des données à un fichier en Python - Suite

- Dans cet exemple, le fichier `myfile.txt` est ouvert en mode ajout en utilisant le modificateur de mode `a` (`append`).
- La méthode `write()` est utilisée pour ajouter la chaîne `"Hello, again!"` à la fin du fichier, sans écraser son contenu existant.
- N'oubliez pas de toujours fermer vos fichiers lorsque vous avez terminé de travailler avec, car cela libère toutes les ressources système associées au fichier et contribue à éviter la corruption des données.

Exemple de lecture de données dans les fichiers en Python

- Voici un exemple de lecture de données à partir d'un fichier :

```
# Open the file in read mode
my_file = open("myFile.txt", "r")

# Read the contents of the file
data = my_file.read()

# Close the file
my_file.close()

# Display the data
print(data)
```

Exemple d'écriture de données dans les fichiers en Python

- Et voici un exemple d'écriture de données dans un fichier :

```
# Open the file in write mode
file = open("myFile.txt", "w")

# Write data to the file
file.write("Hello, world!")

# Close the file
file.close()
```

Méthodes `tell()` et `seek()`

- La méthode `tell()` retourne la position actuelle du curseur dans le fichier.
- La méthode `seek()` permet de déplacer le curseur à une position spécifique dans le fichier.

```python
# Exemple d'utilisation des méthodes tell() et seek()
with open("example.txt", "r") as file:
    position = file.tell()  # Récupère la position actuelle
    file.seek(0)            # Déplace le curseur au début du fichier
```

- **Usage:** Les méthodes `tell()` et `seek()` sont utiles pour la gestion avancée des fichiers, notamment pour la lecture et l'écriture à des positions spécifiques dans le fichier.
- **Documentation:**
  Documentation officielle des méthodes d'objets de fichier

Méthode `writelines()`

- La méthode `writelines()` permet d'écrire une liste de chaînes de caractères dans un fichier.

```
# Example of using writelines() method
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("example.txt", "w") as file:
    file.writelines(lines)
```

- **Usage:** La méthode `writelines()` est utile lorsque vous avez une liste de chaînes de caractères à écrire dans un fichier, sans ajouter de saut de ligne supplémentaire.
- **Documentation:**
  Documentation officielle des méthodes d'objets de fichier

Modules complémentaires : `struct`, `csv`, `json`, `xml`

- Python offre plusieurs modules pour travailler avec différents formats de fichiers et données structurées tels que CSV, JSON et XML.
- Les principaux modules sont `struct`, `csv`, `json` et `xml`.

Module `struct`

- Le module `struct` permet de convertir des objets Python en chaînes d'octets et vice versa, selon un format spécifié.
- Il est utile pour travailler avec des données binaires structurées.

Exemple d'utilisation du module `struct`

```python
import struct

# Packing data
packed_data = struct.pack('i4s', 25, b'abcd')

# Unpacking data
unpacked_data = struct.unpack('i4s', packed_data)

print("Packed data:", packed_data)
print("Unpacked data:", unpacked_data)
```

Module `csv`

- Le module `csv` permet de lire et d'écrire des fichiers CSV (Comma-Separated Values).
- Il fournit des fonctions pour manipuler les données CSV de manière simple et efficace.

Exemple d'utilisation du module `csv`

```python
import csv

# Writing to a CSV file
with open('data.csv', 'w', newline='') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(['Name', 'Age'])
    writer.writerow(['Alice', 30])
    writer.writerow(['Bob', 25])

# Reading from a CSV file
with open('data.csv', newline='') as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        print(row)
```

Module `json`

- Le module `json` permet de lire et d'écrire des données au format JSON (JavaScript Object Notation).
- Il offre des fonctions pour convertir des objets Python en JSON et vice versa.

Exemple d'utilisation du module `json`

```python
import json

# Writing to a JSON file
data = {"name": "Alice", "age": 30}
with open('data.json', 'w') as jsonfile:
    json.dump(data, jsonfile)

# Reading from a JSON file
with open('data.json') as jsonfile:
    data = json.load(jsonfile)
    print(data)
```

Module `xml`

- Le module `xml` permet de manipuler des données au format XML (eXtensible Markup Language).
- Il fournit des fonctionnalités pour analyser, générer et manipuler des documents XML.

Exemple d'utilisation du module `xml`

```python
import xml.etree.ElementTree as ET

# Parsing XML from a string
xml_string = '<root><item>1</item><item>2</item><item>3</item></root>'
root = ET.fromstring(xml_string)

# Accessing XML elements
for item in root.findall('item'):
    print(item.text)
```

Sérialisation avec le module `pickle`

- La **sérialisation** est le processus de conversion d'un objet Python en un format pouvant être stocké ou transféré, tel qu'une chaîne de caractères, un fichier ou un flux de données.
- Le module `pickle` de Python fournit des fonctions pour la sérialisation et la désérialisation des objets Python.
- Utilisations courantes de la sérialisation avec `pickle` :
  - Sauvegarde et restauration de données complexes.
  - Stockage de l'état des objets pour une utilisation ultérieure.
  - Transfert d'objets sur un réseau.
- Documentation : Module pickle.

Module `pickle`

- Le module `pickle` permet de sérialiser et désérialiser des objets Python en un flux d'octets.
- Il est utile pour sauvegarder et restaurer des objets Python de manière efficace.

Exemple d'utilisation du module `pickle`

```python
import pickle

# Serializing data to a file
data = {'name': 'Alice', 'age': 30}
with open('data.pickle', 'wb') as file:
    pickle.dump(data, file)

# Deserializing data from a file
with open('data.pickle', 'rb') as file:
    loaded_data = pickle.load(file)
    print(loaded_data)
```

Sérialisation avec le module `shelve`

- Le module `shelve` de Python fournit une interface pour la sérialisation d'objets Python en utilisant un format de stockage de type dictionnaire.
- Il permet de stocker des objets Python dans une base de données de style clé-valeur sur le disque.
- Utilisations courantes de la sérialisation avec `shelve` :
  - Stockage de données structurées pour une utilisation ultérieure.
  - Gestion de configurations ou de paramètres d'application.
  - Mise en cache de données pour améliorer les performances.
- Documentation : Module shelve

Exemple d'utilisation du module `shelve`

```python
import shelve

# Opening the shelve file
with shelve.open('data') as db:
    # Storing data
    db['name'] = 'Alice'
    db['age'] = 30

# Retrieving data
with shelve.open('data') as db:
    name = db['name']
    age = db['age']
    print(name, age)
```

** Les modules et les paquets **

Modules en Python : Introduction (1/3)

- **Qu'est-ce que les modules en Python ?** En Python, un module est un fichier contenant du code Python qui peut être importé dans d'autres scripts Python. Les modules sont utilisés pour organiser le code dans des fichiers distincts et fournissent un moyen de réutiliser le code dans différents programmes. Pour créer un module, il suffit d'écrire du code Python dans un fichier avec une extension `.py`. Vous pouvez ensuite importer le module dans un autre script Python à l'aide de l'instruction **import**.

Modules en Python : Importation (2/3)

- **Comment importer des modules en Python ?** Pour importer un module en Python, utilisez l'instruction **import** suivie du nom du module. Par exemple, si vous avez un module nommé `my_module.py`, vous pouvez l'importer dans un autre script Python avec l'instruction suivante :

```
1    import my_module
```

Modules en Python : Utilisation (3/3)

- Une fois que vous avez importé le module, vous pouvez utiliser ses fonctions et variables dans votre code. Pour appeler une fonction du module, utilisez la notation pointée, comme ceci :

```
1    my_module.my_function()
```

- Vous pouvez également utiliser le mot-clé **from** pour importer des classes, des fonctions ou des variables spécifiques d'un module, comme ceci :

```
1    from my_module import my_class, my_function, my_variable
```

- Cela vous permet d'utiliser les classes, les fonctions et les variables importées sans avoir à les préfixer par le nom du module.

Bloc `if __name__ == "__main__"`

- En Python, le bloc `if __name__ == "__main__"` est utilisé pour déterminer si le fichier est exécuté en tant que script principal ou s'il est importé en tant que module dans un autre script.
- Le code à l'intérieur du bloc `if __name__ == "__main__"` ne sera exécuté que si le fichier est exécuté en tant que script principal.
- Cela empêche l'exécution non voulue du code lorsque le fichier est importé en tant que module.

Bloc `if __name__ == "__main__"` - Exemple

```python
# module.py
def function():
    print("Function called")

if __name__ == "__main__":
    # Code to execute only if this script is run as the main program
    print("Module is being run directly")
    function()
else:
    # Code to execute if this script is imported as a module
    print("Module is being imported")
```

Bonnes pratiques pour les modules en Python (1/2)

- Lors de l'importation de modules en Python, il existe certaines bonnes pratiques que vous devriez suivre pour vous assurer que votre code est propre, **lisible** et **maintenable**. Voici quelques conseils:
  - Utilisez des importations absolues pour spécifier le chemin complet du module que vous souhaitez importer. Cela permet de préciser le module que vous importez et aide à éviter les conflits de noms.
  - Évitez d'utiliser des importations par joker (par exemple: `from mon_module import *`) car elles peuvent rendre difficile la compréhension d'où proviennent les fonctions et variables.
  - Utilisez des alias pour raccourcir les noms de module et les rendre plus faciles à utiliser. Par exemple, vous pourriez importer le module `numpy` de cette manière : `import numpy as np`.

Bonnes pratiques pour les modules en Python (2/2)

- Suite :
  - Regroupez les importations liées ensemble en haut de votre script pour indiquer clairement les modules dont dépend votre code.
  - Évitez les importations circulaires, où deux modules dépendent l'un de l'autre. Cela peut créer des dépendances confuses et rendre votre code plus difficile à comprendre.
- En suivant ces bonnes pratiques, vous pouvez vous assurer que votre code Python est bien organisé, facile à lire et maintenable au fil du temps.
- Bonnes pratiques.

Importation de paquet

- En Python, un paquet est simplement un répertoire contenant un fichier spécial `__init__.py`.
- L'importation de paquet permet d'organiser et de structurer le code en regroupant des modules connexes dans un même répertoire.
- L'importation de paquet se fait avec la syntaxe `import name_package.name_module`.
- Pour importer tous les modules d'un paquet, on utilise la syntaxe `from name_package import *`, mais cela est généralement déconseillé pour éviter les conflits de noms.

Importation de paquet - Exemple

```python
# Paquet: mypackage
# Structure:
# mypackage/
#       __init__.py
#       module1.py
#       module2.py

# Importing a module from a package
import mypackage.module1
# Using the module
mypackage.module1.function()

# Importing of all modules from a package
from mypackage import *
# Utilisation des modules importés
module1.function()
module2.function()

# Importing using alias
import mypackage.module1 as m1
# Using with alias
m1.function()
```

Création de paquet

- En Python, un paquet est simplement un répertoire contenant un fichier spécial `__init__.py`.
- Le fichier `__init__.py` peut être vide ou contenir des initialisations.
- La création de paquet permet d'organiser et de structurer le code en regroupant des modules connexes dans un même répertoire.
- Pour créer un paquet, il suffit de créer un répertoire et d'y ajouter un fichier `__init__.py`.

Création de paquet - Exemple

```
# Paquet: mypackage
# Structure tree:
# mypackage/
#       __init__.py
#       module1.py
#       module2.py

# Content of file __init__.py
# (may be empty or containing initializations)
# __init__.py

# Content of module1.py
# module1.py
def function1():
    print("Function 1 in module 1")

# Content of module2.py
# module2.py
def function2():
    print("Function 2 in module 2")
```

Création d'un paquet

- Pour créer un paquet en Python, il suffit de créer un répertoire avec un fichier `__init__.py` à l'intérieur.
- Le fichier `__init__.py` peut être vide ou contenir du code d'initialisation pour le paquet.
- Voici un exemple de structure de répertoire pour un paquet nommé `mypackage`:

```
mypackage/
    __init__.py
    module1.py
    module2.py
```

- Dans cet exemple, `mypackage` est un paquet Python contenant deux modules, `module1` et `module2`.

Example 1: Creating a Package - Structure

```
mypackage/
    __init__.py
    module1.py
    module2.py
```

Example 1: Package Initialization (__init__.py)

```python
# __init__.py

from .module1 import MyClass1
from .module2 import MyClass2
```

Example 1: Module 1 (module1.py)

```python
# module1.py

class MyClass1:
    def __init__(self):
        print("Initializing MyClass1")

    def method1(self):
        print("Method 1 in MyClass1")
```

Example 1: Module 2 (module2.py)

```python
# module2.py

class MyClass2:
    def __init__(self):
        print("Initializing MyClass2")

    def method2(self):
        print("Method 2 in MyClass2")
```

Example 2: Creating a Package - Structure

```
mypackage/
    __init__.py
    utils/
        __init__.py
        helper1.py
        helper2.py
    main.py
```

Example 2: Package Initialization (__init__.py)

```python
# __init__.py

from .utils.helper1 import Helper1
from .utils.helper2 import Helper2
```

Example 2: Module 1 (helper1.py)

```python
# helper1.py

class Helper1:
    def __init__(self):
        print("Initializing Helper1")

    def method1(self):
        print("Method 1 in Helper1")
```

Example 2: Module 2 (helper2.py)

```python
# helper2.py

class Helper2:
    def __init__(self):
        print("Initializing Helper2")

    def method2(self):
        print("Method 2 in Helper2")
```

Example 2: Main Module (main.py)

```python
# main.py

from mypackage.utils import Helper1, Helper2

helper1 = Helper1()
helper1.method1()

helper2 = Helper2()
helper2.method2()
```

** Tests unitaires **

Tests unitaires : instruction `assert`, module `unittest`

- L'instruction `assert` est utilisée pour vérifier si une expression est vraie. Si l'expression est fausse, une erreur AssertionError est levée.
- Le module `unittest` fournit un framework pour écrire et exécuter des tests unitaires en Python.

Exemple d'utilisation

```python
import unittest

def add(a, b):
    return a + b

class TestAddFunction(unittest.TestCase):

    def test_add_positive_numbers(self):
        self.assertEqual(add(1, 2), 3)

    def test_add_negative_numbers(self):
        self.assertEqual(add(-1, -2), -3)

    def test_add_mixed_numbers(self):
        self.assertEqual(add(1, -2), -1)
        self.assertEqual(add(-1, 2), 1)

if __name__ == "__main__":
    unittest.main()
```

Framwork `pytest`

- `pytest` est un framework de test populaire et flexible pour Python.
- Il permet d'écrire des tests simples et complexes de manière concise.
- `pytest` offre des fonctionnalités avancées telles que la découverte automatique des tests, la gestion des fixtures et des rapports détaillés.
- Il supporte également les tests de performance et les tests de manière asynchrone.

Exemple d'utilisation

```python
# Exemple de test avec pytest

def test_upper():
    assert 'hello'.upper() == 'HELLO'

def test_isupper():
    assert 'HELLO'.isupper()
    assert not 'Hello'.isupper()
```

- Il suffit de naviguer dans le dossier où se trouve le fichier des tests unitaires exemple: `test_myfile.py` et de lancer la commande: `pytest` afin de lancer les tests unitaires.

Utilisation des Fixtures

- Les fixtures permettent de configurer un état initial avant d'exécuter des tests.
- Elles peuvent être partagées entre plusieurs tests, ce qui évite la répétition de code.

```python
import pytest

@pytest.fixture
def sample_data():
    return [1, 2, 3]

def test_sum(sample_data):
    assert sum(sample_data) == 6
```

** Divers **

** Argh! Les fameuses Expressions Régulières (RegEx) **

Introduction aux expressions régulières

- **Définition** :
  - RegEx (Regular Expressions) désigne une suite de caractères permettant de définir un motif de recherche dans une chaîne.
- **Utilité en DevOps** :
  - Analyser des fichiers logs pour en extraire des informations précises (dates, niveaux de log, messages d'erreur, etc.).
  - Filtrer et valider des données textuelles.
- **Module Python** : `re` offre des fonctions puissantes pour manipuler les expressions régulières.

Syntaxe de base

- **Caractères spéciaux :**
  - `\d` : chiffre (digit)
  - `\D` : non-chiffre
  - `\w` : caractère alphanumérique (word character)
  - `\s` : espace (whitespace)
- **Quantificateurs :**
  - `*` : zéro ou plusieurs fois.
  - `+` : une ou plusieurs fois.
  - `?` : zéro ou une fois.
  - `{n}` : exactement n fois.
  - `{n,m}` : entre n et m fois.
- **Exemple de pattern pour une date :** `(\d{4}-\d{2}-\d{2})`

Syntaxe de base - Exemple

Code illustratif :

```
1 import re
2 pattern = re.compile(r'(\d{4}-\d{2}-\d{2})')
```

Tableau des patterns courants (1/2)

Pattern	Description	Exemple
`\d`	Un chiffre (0-9).	`5`
`\D`	Un caractère non numérique.	`a`
`\w`	Un caractère alphanumérique (a-z, A-Z, 0-9, _).	`test_1`
`\s`	Un espace blanc (espace, tabulation, etc.).	`" "`
`.`	N'importe quel caractère sauf le saut de ligne.	`a` ou `1`

Tableau des patterns courants (2/2)

Pattern	Description	Exemple
`*`	Zéro ou plusieurs occurrences du motif précédent.	`a*`
`+`	Une ou plusieurs occurrences du motif précédent.	`a+`
`?`	Zéro ou une occurrence du motif précédent.	`a?`
`^`	Indique le début d'une ligne ou d'une chaîne.	`^Hello`
`{n}`	Exactement n occurrences.	`a{3}` $\rightarrow$ `aaa`
`{n,m}`	Entre n et m occurrences.	`a{2,4}`

Exemples simples de patterns (1/3)

1. **Numéro de téléphone (format simple):**
   `\d{3}-\d{3}-\d{4}`
   Exemple : `123-456-7890`
2. **Code postal américain (ZIP):**
   `\d{5}(-\d{4})?`
   Exemple : `12345` ou `12345-6789`
3. **Date (YYYY-MM-DD):**
   `\d{4}-\d{2}-\d{2}`
   Exemple : `2023-04-15`
4. **Heure (HH:MM:SS):**
   `\d{2}:\d{2}:\d{2}`
   Exemple : `14:30:00`

Exemples simples de patterns (2/3)

5. **Identifiant alphanumérique**:
   `[\w]{6,12}`
   Exemple : `abc123XYZ`
6. **Extraction de mots (au moins 4 lettres)**:
   `[A-Za-z]{4,}`
   Exemple : `chat, maison`
7. **Match d'une chaîne spécifique**:
   `^Hello`
   Correspond exactement à `Hello` (à partir du début)

Exemples simples de patterns (3/3)

8. **Balises HTML simples:**
   `<[^>]+>`
   Exemple : `<div class="test">`
9. **URL (format basique):**
   `http[s]?://[\w.-]+(\.[\w.-]+)+`
   Exemple : `https://www.example.com`
10. **Adresse e-mail:**
    `[\w.-]+@[\w.-]+\.[a-zA-Z]{2,6}`
    Exemple : `user.name@example.com`

Application : Analyser des logs (1/2)

- **Objectif** : Extraire des informations essentielles depuis un fichier log (par exemple, date, heure, niveau et message).
- **Approche** :
  1. Lire le fichier ligne par ligne.
  2. Appliquer le pattern défini avec `re.search()`.
  3. Extraire et afficher les informations capturées.

Application : Analyser des logs (2/2)

Code illustratif :

```python
import re

# Pattern pour extraire date, heure, niveau et message
pattern = re.compile(
    r'(\d{4}-\d{2}-\d{2})\s+'       # Date    :     YYYY-MM-DD
    r'(\d{2}:\d{2}:\d{2})\s+'       # Heure   :     HH:MM:SS
    r'\[(\w+)\]\s+'                 # Niveau  :     ex. INFO, ERROR
    r'(.*)'                         # Message :     reste de la ligne
)

def parse_log(file_path):
    with open(file_path, "r") as file:
        for line in file:
            match = pattern.search(line)
            if match:
                date, time, level, message = match.groups()
                print(f"Date: {date}, Time: {time}, Level: {level}, Message: {message}")

if __name__ == "__main__":
    parse_log("example_log.txt")
```

Synthèse et recommandations

- Les expressions régulières sont un outil puissant pour l'analyse de textes et de logs.
- Il est essentiel de bien comprendre chaque caractère spécial et quantificateur pour créer des patterns efficaces.
- Pensez à tester vos patterns avec des outils en ligne (comme regex101 ou regexr) pour valider leur comportement.
- La lisibilité et la modularité de votre code sont primordiales : documentez vos expressions et commentez votre code.

** Initiation à Pandas **

Introduction à argparse

- **Objectif** : Rendre vos scripts interactifs en permettant de passer des arguments en ligne de commande.
- **Avantages** :
  - Flexibilité d'exécution selon différents scénarios.
  - Personnalisation du comportement du script sans modification du code source.
  - Amélioration de l'expérience utilisateur via des messages d'aide.
- **Module utilisé** : `argparse`, présent dans la bibliothèque standard de Python.

Création d'un parseur d'arguments

Exemple de code :

```python
import argparse

def main():
    # Création du parseur avec une description du script
    parser = argparse.ArgumentParser(
        description="Script de traitement avec des paramètres en ligne de commande."
    )
    # Ajout d'un argument obligatoire pour le chemin d'accès du fichier
    parser.add_argument("--input", required=True, help="Chemin du fichier d'entrée.")
    # Ajout d'un argument optionnel pour définir le mode de fonctionnement
    parser.add_argument("--mode", default="default", help="Mode de traitement du script.")

    # Parse les arguments fournis lors de l'exécution du script
    args = parser.parse_args()

    # Affichage des paramètres pour vérification
    print(f"Fichier d'entrée : {args.input}")
    print(f"Mode de traitement : {args.mode}")

if __name__ == "__main__":
    main()
```

Utilisation avancée

- **Arguments obligatoires vs. optionnels :**
  - `required=True` pour rendre un argument indispensable.
  - Valeurs par défaut pour les arguments optionnels.
- **Groupes d'arguments :**
  - Permet de regrouper logiquement les options similaires et de fournir une aide contextuelle plus claire.
- **Validation et conversion automatique :**
  - Utilisez le paramètre `type` pour convertir automatiquement les types (e.g. `int`, `float`).
  - Utilisez `choices` pour limiter les valeurs possibles d'un argument.

Exemple avancé avec groupes et conversion de types (1/2)

Exemple de code :

```python
import argparse

def main():
    parser = argparse.ArgumentParser(
        description="Script avancé de traitement d'arguments."
    )

    # Groupement des arguments relatifs au fichier
    groupe_fichier = parser.add_argument_group("Fichier")
    groupe_fichier.add_argument("--input", required=True, help="Chemin du fichier d'entrée.")
    groupe_fichier.add_argument("--output", help="Chemin du fichier de sortie.")

    # Groupement des arguments relatifs au traitement
    groupe_traitement = parser.add_argument_group("Traitement")
    groupe_traitement.add_argument("--mode", choices=["default", "avancé"], default="default",
                                   help="Mode de traitement du script.")
```

Exemple avancé avec groupes et conversion de types (2/2)

Suite du code :

```
    groupe_traitement.add_argument("--nb-iterations", type=int, default=10,
                                   help="Nombre d'itérations (entier).")

    args = parser.parse_args()

    print("=== Paramètres fournis ===")
    print(f"Fichier d'entrée : {args.input}")
    print(f"Fichier de sortie : {args.output}")
    print(f"Mode : {args.mode}")
    print(f"Nombre d'itérations : {args.nb_iterations}")

if __name__ == "__main__":
    main()
```

Conclusion sur argparse

- **Flexibilité** : argparse offre une manière simple et robuste de gérer la personnalisation de vos scripts via la ligne de commande.
- **Clarté** : L'utilisation de groupes d'arguments améliore la lisibilité du message d'aide, facilitant ainsi la compréhension pour l'utilisateur.
- **Validation** : La possibilité de convertir les types et de restreindre les choix contribue à éviter les erreurs au moment de l'exécution.

** Exécution de commandes système **

Exécution de commandes système avec Python

- **Module recommandé** : `subprocess`
- **Utilisation** :
  - Lancer des commandes externes directement depuis Python.
  - Capturer la sortie standard et les messages d'erreur.
- **Cas d'usage** : Automatiser des diagnostics systèmes, contrôler des services, surveiller l'utilisation des ressources, etc.

Exemple pratique d'exécution de commande

Exemple de code :

```python
import subprocess

def run_command(command: str):
  try:
      result = subprocess.run(
          command, shell=True, check=True,
          stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True
      )
      return result.stdout
  except subprocess.CalledProcessError as e:
      # Retourne l'erreur en cas d'échec de la commande
      return f"An error occurred: {e.stderr}"

if __name__ == "__main__":
  # Exécuter la commande pour afficher l'utilisation des disques
  output = run_command("df -h")
  print("Disk Usage Information:")
  print(output)
```

Synthèse et applications

- **Automatisation :** Vous pouvez combiner plusieurs commandes système pour obtenir un diagnostic complet de votre environnement.
- **Intégration :** L'exécution de commandes peut être intégrée dans des scripts d'automatisation pour la gestion des services, la surveillance des ressources ou la maintenance prédictive.
- **Robustesse :** La gestion des erreurs via les exceptions assure une meilleure robustesse de vos scripts en cas de commande défaillante.

** Programmation Orientée Objet **

Présentation du paradigme orienté objet

Le paradigme **Orienté Objet**

Définition : La programmation orientée objet (POO) est l'un des principes fondamentaux de la programmation informatique, dont les origines remontent aux années 1970 avec les langages Simula et Smalltalk. Le principe a rapidement pris de l'ampleur avec la création du langage C++, qui est une extension du langage C, dans le but de rendre les logiciels plus robustes.

Principes fondamentaux de la POO

Une aide mnémotechnique utile pour les **quatre principes fondamentaux** de la programmation orientée objet est `A PIE`.

- **A***bstraction*
- **P***olymorphism*
- **I***nheritance*
- **E***ncapsulation*

Objet

- En informatique, un objet est un conteneur symbolique autonome qui contient des informations et des fonctions/méthodes liées à un sujet, manipulé dans un programme. Le sujet est souvent quelque chose de tangible appartenant au monde réel.

Classe

- Une **classe** est une structure fondamentale des langages de programmation orientés objet.
- Elle sert de **modèle (ou plan)** pour créer des **objets**, en définissant leurs **attributs** (données) et leurs **méthodes** (comportements).
- Les objets créés à partir d'une classe sont appelés **instances** de cette classe.
- Une classe permet donc de regrouper et d'organiser les données et le comportement d'un type d'objet spécifique, favorisant la **réutilisabilité**, la **modularité** et l'**encapsulation**.

Encapsulation

- L'encapsulation consiste à regrouper des données et des méthodes qui les manipulent en une seule entité appelée *capsule*.
- Cela permet d'organiser le code, de limiter l'accès à certaines parties depuis l'extérieur de la capsule et de disposer d'un code robuste.

Abstraction

- L'abstraction est l'un des concepts clés des langages de programmation orientés objet. Son objectif principal est de gérer la complexité en cachant les détails inutiles à l'utilisateur.
- L'abstraction est le processus de représentation d'un objet réel sous forme de modèle informatique. Cela implique essentiellement d'extraire les variables pertinentes attachées aux objets que nous voulons manipuler et de les placer dans un modèle informatique approprié.

Héritage

- Il s'agit d'un mécanisme de transmission de tous les membres ainsi que toutes les méthodes d'une classe dite "mère" à une autre appelée "fille".

Polymorphisme

- Le nom du polymorphisme vient du grec et signifie ”plusieurs formes”. Cette caractéristique est l’un des concepts essentiels de la programmation orientée objet. Alors que l’héritage concerne les classes (et leur hiérarchie), le polymorphisme est lié aux méthodes des objets.

Définition d'une classe d'objet

- En Python, une classe est un modèle permettant de créer des objets.
- Elle définit la structure et le comportement des objets qui en sont des instances.
- Une classe contient des attributs (variables) et des méthodes (fonctions).
- Les méthodes sont des fonctions définies à l'intérieur de la classe et agissent sur les objets de cette classe.
- L'utilisation de classes permet une meilleure organisation du code et favorise la réutilisabilité.
- Les classes sont largement utilisées pour modéliser des entités du monde réel ou abstrait.
- Documentation : Python Classes

Template d'une classe d'objet

```
class ClassName:
    # Constructor __init__ and attributes (members)
    def __init__(self, param_1, param_2, ..., param_n):
        self.member_1 = param_1     # Member or attribute 1
        self.member_2 = param_2     # Member or attribute 2
        # etc.
    # Methods of class ClassName
    def method_1(self, arg_1, arg_2, ..., arg_n):
        # Code of method 1
    def method_2(self, arg_1, arg_2, ..., arg_n):
        # Code of method 2
    # etc.
```

Définition d'un objet

Définition: Un **objet** est une **instance** d'une **classe**.

Définition d'un objet (état, comportement, identité)

- **État :** Un objet a un **état** qui est défini par ses **attributs**. Ces attributs représentent les caractéristiques de l'objet à un moment donné.
- **Comportement :** Un objet a un **comportement** défini par ses **méthodes**. Les méthodes représentent les actions ou opérations que l'objet peut effectuer.
- **Identité :** Chaque objet a une **identité unique** qui le distingue des autres objets. L'identité est généralement attribuée par le système.

Exemples

Exemple - (1/3)

```python
class Person:
    def __init__(self, name, age):
        self.name = name        # Member or attribute 1
        self.age = age          # Member or attribute 2

# Creating objects via constructor __init__
person1 = Person("Ali", 30)     # Object (instance of class) 1
person2 = Person("Alison", 25)  # Object (instance of class) 2

# Displaying identity
print(f"{person1.name} is {person1.age} years old")
print(f"{person2.name} is {person2.age} years old")
```

Exemple - (2/3)

```python
class Animal:
    def __init__(self, name, species):
        self.name = name        # Member or attribute 1
        self.species = species  # Member or attribute 2

    def display_info(self):     # Method of class Animal
        print(f"{self.name} is a {self.species}")

# Creating objects via constructor __init__
lion = Animal("Simba", "lion")  # Object (instance of class) 1
snake = Animal("Kaa", "snake")  # Object (instance of class) 2

# Calling methods of the class Animal
lion.display_info()
snake.display_info()
```

Exemple - (3/3)

```python
class Car:
    def __init__(self, brand, model):
        self.brand = brand          # Member or attribute 1
        self.model = model          # Member or attribute 2
        self.speed = 0              # Member or attribute 3

    def accelerate(self, increment):    # Method 1 of class Car
        self.speed += increment
        print(f"{self.brand} {self.model} accelerates to {self.speed} km/h")

    def brake(self, decrement):         # Method 2 of class Car
        self.speed -= decrement
        print(f"{self.brand} {self.model} brakes to {self.speed} km/h")

# Creating objects
my_car = Car("Porsche", "Cayenne")      # Object (instance of class) 1

# Calling methods of the class Car
my_car.accelerate(20)
my_car.brake(10)
```

Définition d'une classe

Définition : Une **classe** est une **structure de données** fondamentale en **programmation orientée objet** qui **encapsule** les **attributs** et les **méthodes**.
Ces éléments définissent les **caractéristiques** et le comportement d'un **objet**, permettant ainsi la création d'**instances** spécifiques de cette classe.

Définition d'un attribut

Définition : un **attribut**, également appelé **membre de classe**, est une **variable** déclarée à l'**intérieur d'une classe** qui a pour but de **stocker des données spécifiques** à cette classe.
Les attributs sont ainsi des **caractéristiques** ou des **propriétés d'un objet** qui sont définies dans la classe et qui peuvent être utilisées pour décrire l'état ou les **caractéristiques de l'objet**.

Définition d'une méthode

Définition : Une **méthode** est une fonction ou un bloc de code **associé à une classe** ou **à un objet**. Elle permet de définir le **comportement** de l'**objet** ou de la **classe**.

- **Méthode d'instance (méthode régulière)**: Appelée par une instance de la classe, agit sur les attributs de cette instance.
- **Méthode statique**: Appelée par la classe elle-même, indépendante des instances, souvent utilisée pour des fonctionnalités utilitaires.
- **Méthode de classe**: Appelée par la classe, agit sur les attributs de la classe plutôt que sur les instances.

Table: Comparison of Regular, Static, and Class Methods

	Regular Method	Static Method	Class Method
Definition Syntax	Defined without decorators	Defined with `@staticmethod` decorator	Defined with `@classmethod` decorator
First Argument	`self` (instance)	None	`cls` (class)
Access toAttributes/Methods	Can access instance attributes and methods directly	No implicit access toinstance attributes/methods	Can access class-levelattributes and methods *via* `cls`
Instance Specific	Requires an instance	Not tied to any instance	Not tied to any instance
Class Specific	Cannot access class-levelattributes/methods	Cannot access class-levelattributes/methods	Can access class-level attributes/methods *via* `cls`
Inheritance	Inherited by subclasses	Not inherited by subclasses	Inherited by subclasses
Usage	Use when instance-specific behavior is needed	Use when the method does not depend on instance or class attributes/methods	Use when class-level behavior is needed, such as factory methods or alternative constructors
Example	`def method(self, arg):`	`@staticmethod` `def method(arg):`	`@classmethod` `def method(cls, arg):`

Exemples

- Voir l'exemple (1/4) pour plus de détails ;
- Voir l'exemple (2/4) pour plus de détails ;
- Voir l'exemple (3/4) pour plus de détails ;
- Voir l'exemple (4/4).

Les méthodes Getters & Setters

Définition : Les **getters** et les **setters** sont des méthodes utilisées **respectivement** pour **accéder** et **modifier** les valeurs des **membres** (attributs / variables) d'une **classe**.

Signature des Getters & Setters **sans** type de retour

- **Les getters retournent** la valeur d'un membre et ont la signature :

```
def get_var(self):
    return self.var
```

- **Les setters modifient** la valeur d'un membre et ont la signature :

```
def set_var(self, _var):
    self.var = _var
```

Exemple

```python
class Person:
    def __init__(self, name, age):
        self._name = name    # PROTECTED Member or attribute 1
        self._age = age      # PROTECTED Member or attribute 2

    # Getter for name
    def get_name(self):
        return self._name

    # Setter for name
    def set_name(self, name):
        self._name = name

    # Getter for age
    def get_age(self):
        return self._age

    # Setter for age
    def set_age(self, age):
        self._age = age

# Creating objects via constructor __init__
person1 = Person("Alison", 25)  # Object or instance of class Person
# Displaying identity
print(f"{person1.get_name()} is {person1.get_age()} years old")
```

Signature des Getters & Setters **avec** type de retour

- **Les getters retournent** la valeur d'un membre et ont la signature :

```
def get_var(self) -> typename:
    return self.var
```

- **Les setters modifient** la valeur d'un membre et ont la signature :

```
def set_var(self, _var: typename) -> None:
    self.var = _var
```

Exemple

```python
class BankAccount:
    def __init__(self, account_holder: str, balance: float):
        self._account_holder = account_holder   # PROTECTED Member or attribute 1
        self._balance = balance                 # PROTECTED  Member or attribute 2

    # Getter for account holder
    def get_account_holder(self) -> str:
        return self._account_holder

    # Setter for account holder
    def set_account_holder(self, account_holder: str):
        self._account_holder = account_holder

    # Getter for balance
    def get_balance(self) -> float:
        return self._balance

    # Setter for balance
    def set_balance(self, balance: float):
        self._balance = balance

# Creating objects via constructor __init__
account1 = BankAccount("John Doe", 1000.0)   # Object or instance of class BankAccount
# Displaying identity
print(f"{account1.get_account_holder()} has a balance of ${account1.get_balance()}")
```

Spécificateurs d'accès en Python

- En Python, il existe trois spécificateurs d'accès: `public`, `_protected` et `__private`.
- Les attributs ou méthodes `public` peuvent être accessibles de n'importe où dans le programme.
- Les attributs ou méthodes `_protected` sont accessibles dans la classe où ils sont définis et dans les sous-classes.
- Les attributs ou méthodes `__private` ne peuvent être accessibles que dans la classe où ils sont définis.
- Les spécificateurs d'accès sont définis par des conventions plutôt que par des fonctionnalités de langage strictes.

Spécificateurs d'accès en Python - Exemple 1

```python
class MyClass:
    def __init__(self):
        self.public_attribute = 42
        self._protected_attribute = "protected"
        self.__private_attribute = "private"

    def access_attributes(self):
        print("Public attribute:", self.public_attribute)
        print("Protected attribute:", self._protected_attribute)
        print("Private attribute:", self.__private_attribute)

# Creating an object of MyClass
obj = MyClass()

# Accessing attributes
obj.access_attributes()
```

Spécificateurs d'accès en Python - Exemple 2

```python
class MyClass:
    def __init__(self):
        self.public_attribute = 42
        self._protected_attribute = "protected"
        self.__private_attribute = "private"

    def access_attributes(self):
        print("Public attribute:", self.public_attribute)
        print("Protected attribute:", self._protected_attribute)
        print("Private attribute:", self.__private_attribute)

    def _protected_method(self):
        print("This is a protected method")

    def __private_method(self):
        print("This is a private method")

# Creating an object of MyClass
obj = MyClass()

# Accessing attributes and methods
obj.access_attributes()
obj._protected_method()
# obj.__private_method()  # This will raise an error
```

Résumé

Type d'accès	Même classe	Sous-classe	Même package	Autres packages
Public				
Protégé				
Privé				

Name Mangling en Python

- Le **name mangling** est un mécanisme utilisé en Python pour rendre les attributs ou méthodes d'une classe plus difficiles à accéder à partir de l'extérieur de la classe.
- Cela est réalisé en ajoutant le nom de la classe comme préfixe au nom de l'attribut ou de la méthode, précédé par deux traits de soulignement.
- Le **name mangling** est déclenché lorsqu'un nom d'attribut ou de méthode commence par deux traits de soulignement __.
- Python renomme alors cet attribut ou cette méthode en y ajoutant le nom de la classe, précédé par un trait de soulignement.
- En utilisant le "name mangling", les attributs et méthodes d'une classe peuvent être rendus moins accessibles depuis l'extérieur de la classe, bien que cela ne les rende pas complètement privés.

Name Mangling en Python - Exemple illustratif

```python
class MyClass:
    def __init__(self):
        self.__private_attritube = 420

    def __private_method(self):
        print("This is a private method")

# Creating an instance of the class
obj = MyClass()

# Accessing the private attribute from outside the class
# print(obj.__private_attritube)  # This will raise an AttributeError

# Calling the private method from outside the class
# obj.__private_method()  # This will also raise an AttributeError

# Accessing the private attribute using name mangling
print(obj._MyClass__private_attritube)  # This will print 420

# Calling the private method using name mangling
obj._MyClass__private_method()  # This will print "This is a private method"
```

Principe de l'encapsulation

Définition : L'encapsulation est un mécanisme qui permet de cacher les détails d'**implémentation d'une classe** aux **autres classes**.
Il s'agit d'un concept clé en **programmation orientée objet** qui **protège les données** d'une classe et **garantit leur intégrité**.

Objectifs de l'encapsulation

- Réduction des erreurs et des conflits de noms.
- Amélioration de la sécurité.
- Facilitation de la maintenance du code.
- Modification facile du code sans affecter les autres parties du programme.

Exemple d'encapsulation

Supposons que nous avons une classe **BankAccount** qui contient des données sensibles telles que le solde du compte et le numéro de compte bancaire. Afin de protéger ces données, nous pouvons les déclarer comme `protected` ou `private` et utiliser des méthodes `public` pour y accéder et les modifier, par exemple `get_balance()` ou encore `deposit()`. De cette manière, les autres classes ne peuvent pas accéder directement aux données sensibles de la classe `BankAccount`, mais doivent passer par les méthodes publiques qui garantissent que les données sont manipulées de manière sûre et sécurisée.

Conclusion

- En encapsulant les données de cette manière, nous assurons que:
  - **les informations sensibles ne sont pas accessibles depuis l'extérieur de la classe ;**
  - leur manipulation est effectuée de **manière contrôlée.**
- Cela permet de **garantir l'intégrité des données** et d'**améliorer la sécurité de notre programme.**

Instanciation d'objets

- En Python, les objets sont des instances de classes.
- Pour instancier un objet, utilisez le nom de la classe suivi de parenthèses.
- L'instance est créée en appelant le constructeur de la classe, généralement défini par la méthode `__init__()`.
- Exemple d'instanciation :

```
# Définition de la classe
class MyClass:
    def __init__(self, x):
        self.x = x

# Instanciation de l'objet
obj = MyClass(10)
```

Fonction `isinstance()`

- La fonction `isinstance()` est utilisée pour vérifier le type d'un objet.
- Elle prend deux arguments : l'objet à vérifier et le type ou la classe attendu(e).
- Elle renvoie `True` si l'objet est une instance de la classe spécifiée, sinon `False`.
- Exemple d'utilisation :

```
1  # Definition of the class MyClass
2  class MyClass:
3      pass
4
5  obj = MyClass() # Object Instantiation
6
7  # Verification if obj is an instance of class MyClass
8  if isinstance(obj, MyClass):
9      print("Obj is an instance of MyClass")
10 else:
11     print("Obj n'est pas une instance de MyClass")
```

Constructeur (`__init__`)

- Le constructeur `__init__()` est une **méthode spéciale** utilisée pour initialiser les objets lors de leur création.
- Il est appelé automatiquement lorsque vous instanciez la classe pour créer un nouvel objet.
- Le premier paramètre est conventionnellement nommé `self` et fait référence à l'objet lui-même.
- Vous pouvez définir des **attributs** d'instance dans le constructeur en utilisant la notation `self.attribute_name`.
- Exemple :

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6 person1 = Person("Alice", 30) # Creating a Person object
```

Attributs (membres)

- Les **attributs** sont des variables liées à une classe ou à un objet.
- Les attributs sont accessibles à travers l'instance de la classe à l'aide de la notation pointée (par exemple, `object.attribute` ou `ClassName.class_variable`).
- Exemple :

```
1 from math import pi as PI
2     class Circle:
3     def __init__(self, radius):
4         self.radius = radius
5
6 circle1 = Circle(PI) # Creating a Circle object
7 print("Accessing the attribute radius of circle:", circle1.radius) # Output: PI value
```

Méthodes

- Les **méthodes** sont des fonctions définies à l'intérieur d'une classe et sont utilisées pour effectuer des opérations sur les objets de cette classe.
- Les méthodes sont accessibles à travers l'instance de la classe à l'aide de la notation pointée (par exemple, `object.method()` ou `MyClass.static_or_class_method()`).
- Exemple :

```
from math import pi as PI
    class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return PI * self.radius ** 2

circle1 = Circle(PI) # Creating a Circle object
print("Area of circle:", circle1.area())
```

Le paramètre `self`

- En Python, `self` est utilisé comme premier paramètre dans la définition de méthodes régulières.
- Il fait référence à l'instance de la classe elle-même.
- Lorsque vous appelez une méthode sur un objet, Python passe automatiquement l'objet lui-même en tant que premier argument à la méthode, donc vous ne devez pas spécifier `self` lors de l'appel de la méthode.
- Exemple :

```
1 from math import pi as PI
2 class Circle:
3     def __init__(self, radius):
4         self.radius = radius
5     def area(self):
6         return PI * self.radius ** 2
7
8 circle1 = Circle(PI) # Creating a Circle object
9 print("Area of circle:", circle1.area())
```

Surcharge d'affichage (`__str__`)

- La méthode spéciale `__str__()` est utilisée pour retourner une représentation de chaîne de caractères de l'objet.
- Elle est invoquée lorsque la fonction `str()` ou la méthode `print()` est appelée sur l'objet.
- En surchargeant cette méthode, vous pouvez spécifier comment vous souhaitez que votre objet soit affiché sous forme de chaîne de caractères.

Surcharge d'affichage (`__str__`) - Exemple

- Exemple :

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Point({self.x}, {self.y})"

# Utilisation
p = Point(3, 4)
print(str(p))  # Output: Point(3, 4)
```

Surcharge d'opérateurs

- En Python, vous pouvez surcharger de nombreux opérateurs pour définir le comportement des objets de vos classes.
- Voici une liste des opérateurs pouvant être surchargés avec leur utilisation typique :
  - `__add__()` : Addition (+)
  - `__sub__()` : Soustraction (-)
  - `__mul__()` : levelplication (*)
  - `__truediv__()` : Division (/)
  - `__floordiv__()` : Division entière (//)
  - `__mod__()` : Modulo (%)
  - `__pow__()` : Puissance (**)

Surcharge d'opérateurs (suite)

- `__eq__()` : Égalité ==
- `__ne__()` : Différent !=
- `__gt__()` : Supérieur strict >
- `__ge__()` : Supérieur ou égal >=
- `__le__()` : Inférieur ou égal <=
- `__and__()` : Et logique &
- `__or__()` : Ou logique |
- `__xor__()` : Ou exclusif ^
- `__invert__()` : Inversion de bits ~

Documentation : Surcharge d'opérateurs en Python

Surcharge d'opérateurs (`__eq__`)

- La méthode spéciale `__eq__()` est utilisée pour comparer l'égalité entre deux objets.
- Elle est invoquée lorsque l'opérateur d'égalité (==) est utilisé entre deux objets.
- En surchargeant cette méthode, vous pouvez définir votre propre logique d'égalité entre les objets de votre classe.
- La méthode devrait renvoyer `True` si les objets sont égaux et `False` sinon.

Surcharge d'opérateurs (__eq__) - Exemple

- Exemple :

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

p1 = Point(3, 4)
p2 = Point(4, 3)
print(p1 == p2)  # Output: False
```

Surcharge d'opérateurs (`__add__`)

- La méthode spéciale `__add__()` est utilisée pour définir le comportement de l'opérateur d'addition + pour les objets de votre classe.
- Elle est invoquée lorsque l'opérateur d'addition est utilisé entre deux objets.
- En surchargeant cette méthode, vous pouvez définir votre propre logique pour l'addition des objets de votre classe.

Surcharge d'opérateurs (__add__) - Exemple

- Exemple :

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

# Utilisation
p1 = Point(1, 2)
p2 = Point(3, 4)
result = p1 + p2
print(result.x, result.y)  # Output: 4 6
```

Propriété, accesseur et mutateur

- Les **propriétés** (`property()`) en Python offrent un moyen de contrôler l'accès aux données d'un objet de manière encapsulée et cohérente. Elles sont définies à l'aide de la fonction spéciale `property()`, permettant d'associer des accesseurs et des mutateurs à un attribut.
  - Un **accesseur** est une méthode permettant de récupérer la valeur d'un attribut.
  - Un **mutateur** est une méthode permettant de modifier la valeur d'un attribut.
- L'utilisation de propriétés garantit l'encapsulation des données et permet d'appliquer des validations ou des transformations lors de l'accès ou de la modification des attributs. Cela contribue à la robustesse et à la cohérence des classes en facilitant le contrôle précis de l'accès aux données.

L'objet `property` - Exemple 1

- Voici un exemple de définition d'une propriété avec des accesseurs et des mutateurs :

```
class Person:
    def __init__(self, name):
        self._name = name       # _name: private member

    def get_name(self):
        return self._name       # get_name: getter

    def set_name(self, value):  # set_name: setter
        if isinstance(value, str):
            self._name = value
        else:
            raise TypeError("Name must be a string")
    # property object, here we instantiate the class "property"
    name = property(get_name, set_name)

# Usage of property
person = Person("Alice")    # Instantiation of Person
print(person.name)          # Usage of the accessor <==> Getter
person.name = "Bob"         # Usage of the mutator  <==> Setter
print(person.name)
```

L'objet `property` - Exemple 1 - Suite

- Dans cet exemple, la classe `Person` définit une propriété `name` avec un accesseur `get_name()` et un mutateur `set_name()`. L'utilisation de la propriété permet d'accéder à l'attribut `_name` de manière contrôlée et encapsulée.

Fonction spéciale `property` - Exemple 2

- Voici un autre exemple de définition d'une propriété:

```
class Circle:
    def __init__(self, radius):
        self.__radius = radius
    def _get__radius(self):
        print("Get radius")
        return self.__radius
    def _set__radius(self, value):
        print("Set radius")
        self.__radius = value
    def _del__radius(self):
        print("Delete radius")
        del self.__radius
    radius = property(
        fget=_get__radius,
        fset=_set__radius,
        fdel=_del__radius,
        doc="The radius property."
    )
```

L'objet `property` - Exemple 2 - Suite

- Suite :

```
>>> from circle import Circle
>>> circle = Circle(42.0)
>>> circle.radius
Get radius
42.0
>>> circle.radius = 100.0
Set radius
>>> circle.radius
Get radius
100.0
>>> del circle.radius
Delete radius
>>> circle.radius
Get radius
Traceback (most recent call last):
    ...
AttributeError: 'Circle' object has no attribute '__radius'
```

Décorateur `property` - Exemple 3

- Bonne pratique pour le même exemple avec la notion de `@decorator`:

```
class Circle:
    def __init__(self, radius):
        self.__radius = radius

    @property
    def radius(self):
        """The radius property."""
        print("Get radius")
        return self.__radius

    @radius.setter
    def radius(self, value):
        print("Set radius")
        self.__radius = value

    @radius.deleter
    def radius(self):
        print("Delete radius")
        del self.__radius
```

Espaces de noms global, de l'objet, de la classe

- L'espace de noms global contient les noms définis au niveau du module.
- L'espace de noms de l'objet contient les attributs propres à chaque instance de la classe.
- L'espace de noms de la classe contient les attributs partagés par toutes les instances de la classe.
- L'accès aux attributs se fait en utilisant la notation pointée.

Variable de classe

- Une variable de classe est une variable définie au niveau de la classe et partagée par toutes les instances de la classe.
- Elle est définie en dehors des méthodes de la classe.
- Elle est accessible à partir de n'importe quelle instance de la classe ou même de la classe elle-même.

Exemple d’utilisation des variables de classe

- Exemple complet d’utilisation.

Constructeur à nombre d'arguments arbitraire (*args, **kwargs)

- Les paramètres `*args` et `**kwargs` permettent de définir des constructeurs à nombre d'arguments arbitraire.
- `*args` permet de recevoir un nombre variable d'arguments positionnels.
- `**kwargs` permet de recevoir un nombre variable d'arguments nommés.
- Cela offre une grande flexibilité lors de l'instanciation d'objets.

Exemple d'utilisation de `*args` et `**kwargs` dans un constructeur

```python
class DynamicObject:
    def __init__(self, *args, **kwargs):
        # Store positional arguments as a list
        self.args = list(args)

        # Dynamically store keyword arguments as instance attributes
        for key, value in kwargs.items():
            setattr(self, key, value)

# Create objects with various arguments
obj1 = DynamicObject(1, 2, 3, name="Alice", age=30)
obj2 = DynamicObject("Hello", name="Bob", age=25)

# Display internal attributes
print(obj1.__dict__)
print(obj2.__dict__)
# Output:
# {'args': [1, 2, 3], 'name': 'Alice', 'age': 30}
# {'args': ['Hello'], 'name': 'Bob', 'age': 25}
```

Relations UML et diagrammes de classes

Relations UML : introduction

- Les **relations UML** (Unified Modeling Language) permettent de représenter les liens logiques entre les différentes classes d'un système orienté objet.
- Elles traduisent la **structure** et les **interactions** entre les objets.
- Les principales relations sont : **Association**, **Agrégation**, **Composition**, **Dépendance**, **Héritage** et **Réalisation**.
- Elles sont essentielles pour comprendre comment les objets coopèrent et interagissent dans le système.

Les trois grandes familles de diagrammes UML

- UML regroupe ses diagrammes en trois grandes catégories :
  - **Diagrammes structurels** : représentent les éléments statiques du système (ex. : classes, objets, composants, déploiement).
  - **Diagrammes comportementaux** : décrivent les interactions et les dynamiques (ex. : cas d'utilisation, séquence, activité).
  - **Diagrammes d'interaction** : se concentrent sur les échanges entre objets (ex. : communication, séquence, timing).
- Ensemble, ces familles permettent une modélisation complète d'un système logiciel.

Famille 1 : Diagrammes structurels

- Représentent la **structure statique** du système.
- Décrivent les éléments qui composent le système (classes, interfaces, composants, nœuds...).
- Les principaux diagrammes structurels :
  - **Diagramme de classes**
  - **Diagramme de composants**
  - **Diagramme de déploiement**
  - **Diagramme d'objets**
  - **Diagramme de packages**

Famille 2 : Diagrammes comportementaux

- Illustrent la **dynamique du système** et le comportement des objets au fil du temps.
- Montrent comment les éléments collaborent pour réaliser une fonctionnalité.
- Exemples :
  - **Diagramme de cas d'utilisation**
  - **Diagramme d'activités**
  - **Diagramme d'états-transitions**

Famille 3 : Diagrammes d'interaction

- Sous-catégorie des diagrammes comportementaux.
- Décrivent les **échanges de messages** entre objets ou composants.
- Utiles pour représenter la logique d'exécution et les communications.
- Exemples :
  - **Diagramme de séquence**
  - **Diagramme de communication**
  - **Diagramme de timing**

Diagrammes de classes : définition

- Le **diagramme de classes** est le **diagramme structurel principal** en UML.
- Il modélise la **structure statique** d'un système orienté objet.
- Il montre :
  - Les **classes** du système.
  - Leurs **attributs** et **méthodes**.
  - Les **relations** entre les classes.
- Il sert de base à la conception orientée objet et à la génération de code.

Diagrammes de classes : rôle et utilité

- Utilisé pour :
  - Comprendre la structure du système.
  - Identifier les dépendances entre classes.
  - Préparer l'implémentation du code.
- Permet d'assurer une **cohérence** entre l'analyse, la conception et le développement.
- Sert de **documentation technique** du modèle objet.

Diagrammes de classes : éléments principaux

- Les éléments clés d'un diagramme de classes :
  - **Classes** : représentées sous forme de rectangles divisés en trois parties (nom, attributs, opérations).
  - **Attributs** : décrivent les données détenues par la classe.
  - **Méthodes (opérations)** : définissent les comportements.
  - **Relations** : illustrent les liens entre les classes.
- Chaque relation est représentée par un symbole graphique spécifique (lignes, flèches, losanges...).

Multiplicités

- Les multiplicités définissent le **nombre d'instances** d'une classe pouvant être associées à une instance d'une autre classe.
- Elles précisent la cardinalité de la relation :
  - 1..1 : une seule instance.
  - 0..1 : une instance facultative.
  - 0..* ou * : plusieurs instances possibles.
  - 1..* : au moins une instance.
- En UML, les multiplicités sont notées près des extrémités des associations.

Relations dans les diagrammes de classes

- Les principales relations entre classes sont :
  - **Association** : lien logique entre classes **has-a**.
  - **Agrégation** : relation tout-partie avec dépendance faible **has-a**.
  - **Composition** : relation tout-partie avec dépendance forte **has-a**.
  - **Dépendance** : utilisation temporaire d'une autre classe **uses-a**.
  - **Héritage** : relation hiérarchique **is-a**.
  - **Réalisation (Implémentation)** : une classe implémente une interface **is-a**.
- Ces relations définissent la **structure interne** du système et les **liens de responsabilité**.

Association

- L'association est une relation entre deux classes en Python.
- Elle permet à des objets de chaque classe de collaborer entre eux.
- Les objets de chaque classe peuvent exister indépendamment les uns des autres.
- Elle est souvent utilisée pour modéliser des relations **"has-a"** ou **"has-many"** (indépendance).

Association - Class Diagram

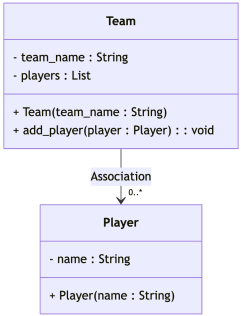


Figure: Diagramme UML - Association

Exemple d'association

```python
class Player:
    def __init__(self, name):
        self.__name = name

class Team:
    def __init__(self, team_name):
        self.__team_name = team_name
        self.__players = []

    def add_player(self, player):
        self.__players.append(player)

# Creating objects (instances)
player_1 = Player("John")
player_2 = Player("Alice")

team = Team("Team A")

# Associating objects
team.add_player(player_1)
team.add_player(player_2)
```

Agrégation

- L'agrégation est une technique de construction de classes en Python.
- Elle consiste à ajouter des objets d'autres classes comme attributs de la classe courante.
- Elle permet de créer des relations entre les classes et de réutiliser du code de manière efficace.
- Elle modélise une relation **"has-a"** ou **"has-many"** avec **dépendance faible**.

Agrégation - Class Diagram

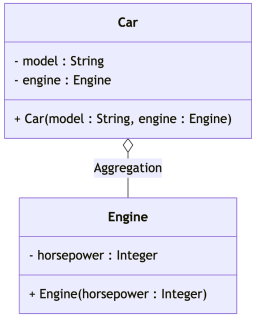


Figure: Diagramme UML - Agrégation

Exemple d'agrégation

```python
class Engine:
    def __init__(self, horsepower):
        self.__horsepower = horsepower

class Car:
    def __init__(self, model, engine):
        self.__model = model
        self.__engine = engine  # Aggregation

engine = Engine(200)
car = Car("Toyota Camry", engine)
```

Composition

- La composition est une technique de construction de classes en Python.
- Elle consiste à inclure des objets d'autres classes en tant qu'attributs de la classe courante et à les **initialiser directement** dans le constructeur.
- Elle permet de modéliser une relation **"has-a"** avec une **forte dépendance**.
- Si l'objet conteneur est détruit, les objets contenus le sont également.

Composition - Class Diagram

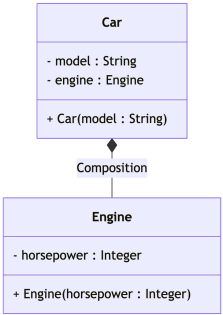


Figure: Diagramme UML - Composition

Exemple de composition

```python
class Engine:
    def __init__(self, horsepower):
        self.__horsepower = horsepower

class Car:
    def __init__(self, model):
        self.__model = model
        self.__engine = Engine(200)  # Composition

car = Car("Toyota Camry")
```

Dépendance

- La dépendance indique qu'une classe **utilise temporairement** une autre classe sans en être propriétaire.
- C'est une relation faible et ponctuelle.
- En UML, elle est représentée par une flèche en pointillés.
- Elle exprime qu'un changement dans une classe peut affecter la classe dépendante.

Dépendance - Class Diagram

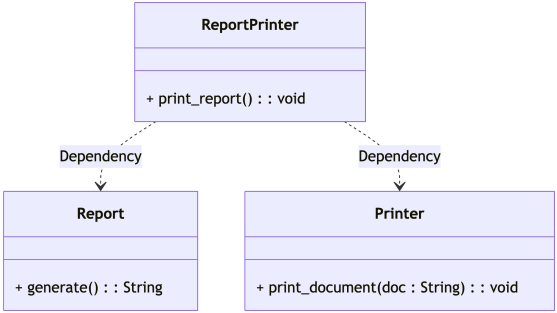


Figure: Diagramme UML - Dépendance

Exemple de dépendance

```python
class Printer:
    def print_document(self, doc):
        print(f"Printing: {doc}")

class Report:
    def generate(self):
        return "Annual Report"

class ReportPrinter:
    def print_report(self):
        report = Report()  # Temporary Dependance
        printer = Printer()
        printer.print_document(report.generate())

report_printer = ReportPrinter()
report_printer.print_report()
```

Héritage

- L'héritage établit une relation **is-a** entre une superclasse et une sous-classe.
- Il permet de **réutiliser, spécialiser ou redéfinir** le comportement d'une classe existante.
- Favorise la hiérarchisation et la factorisation du code.
- En UML, représenté par une flèche pleine avec un triangle vide pointant vers la superclasse.

Héritage - Class Diagram

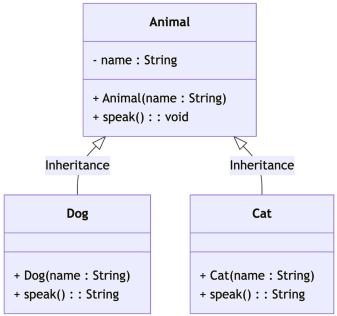


Figure: Diagramme UML - Héritage

Exemple d'héritage

```python
class Animal:
    def __init__(self, name):
        self.__name = name

    def speak(self):
        pass

class Dog(Animal): # Inheritance
    def speak(self):
        return "Woof!"

class Cat(Animal): # Inheritance
    def speak(self):
        return "Meow!"

dog = Dog("Buddy")
cat = Cat("Misty")

print(dog.speak())  # Output : Woof!
print(cat.speak())  # Output : Meow!
```

Réalisation / Implémentation

- La réalisation est une relation entre une classe et une interface.
- Elle indique qu'une classe **implémente** les opérations définies dans l'interface.
- En UML, représentée par une flèche en pointillés avec un triangle vide.
- En Python, on simule une interface à l'aide d'une **classe abstraite** (module `abc`).

Réalisation / Implémentation - Class Diagram

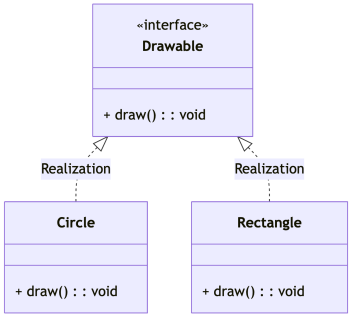


Figure: Diagramme UML - Réalisation / Implémentation

Exemple de réalisation / implémentation

```python
from abc import ABC, abstractmethod

class Drawable(ABC):        # Interface
    @abstractmethod
    def draw(self):
        pass

class Circle(Drawable):     # Implementation / Realization
    def draw(self):
        print("Drawing a circle")

class Rectangle(Drawable):  # Implementation / Realization
    def draw(self):
        print("Drawing a rectangle")

shapes = [Circle(), Rectangle()]
for shape in shapes:
    shape.draw()
```

Différences entre Association, Agrégation et Composition

- **Association** :
  - C'est une relation **générique et générale** entre deux classes (**has-a**).
  - Les objets de chaque classe peuvent **exister indépendamment** les uns des autres.
- **Agrégation** :
  - C'est une relation **tout-partie** (**has-a** faible).
  - L'objet de la classe agrégée peut exister indépendamment de l'objet de la classe conteneur.
- **Composition** :
  - C'est une relation **tout-ou-rien** (**has-a** forte).
  - C'est une forme **plus forte d'agrégation**.
  - L'objet de la classe composée ***ne peut pas exister* sans l'objet** de la classe conteneur.

Différences entre Dépendance, Héritage et Réalisation

- **Dépendance :**
  - Relation **temporaire** d'utilisation (**uses-a**).
  - Faible couplage.
- **Héritage :**
  - Relation **hiérarchique** (**is-a**).
  - Permet la **réutilisation** et la spécialisation du code.
- **Réalisation / Implémentation :**
  - Une classe **implémente** une interface ou une classe abstraite (**is-a**).

Concepts objets, et diagrammes de classes - lien avec la POO

- La figure ci-dessous explicite les diverses notions de relation existantes:

Class Diagram Relationship Type	Notation
Association	
Inheritance	
Realization/ Implementation	
Dependency	
Aggregation	
Composition	

Figure: relationships

Principe de l'héritage

Principe de l'héritage

Définition: L'**héritage** est un principe fondamental de la programmation orientée objet (POO) qui permet à **une classe d'hériter des caractéristiques (attributs et méthodes) d'une autre classe**.

- La classe qui hérite est appelée **sous-classe** ou **classe dérivée**.
- La classe dont on hérite est appelée **classe de base** ou **classe parente**.

Référence 1 : Documentation Python - Héritage
Référence 2 : d'avantage de documentation sur l'héritage.

Types d'héritage

L'Héritage en Python se décompose en différents types qui déterminent comment une classe dérive des propriétés d'une autre.

- **Héritage Simple:** Une classe hérite des propriétés d'une seule autre classe de base.
- **Héritage Multiple:** Une classe hérite des propriétés de plusieurs classes de base. Héritage Multiple
- **Héritage Multi-niveaux:** Une classe hérite d'une classe de base, puis une autre classe hérite de cette classe dérivée, établissant ainsi une chaîne d'héritage sur plusieurs niveaux.
- **Héritage Hiérarchique:** Plusieurs classes dérivent des propriétés d'une seule classe de base.
- **Héritage Hybride:** Combinaison d'héritage multiple et hiérarchique.

Exemple d'Héritage Simple en Python

```python
# Simple inheritance
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

# Derived class usecase
dog_instance = Dog()
dog_instance.speak()
dog_instance.bark()
```

Exemple d'Héritage Multiple en Python

```python
# Multiple inheritance
class Animal:
    def speak(self):
        print("Animal speaks")

class Flyable:
    def fly(self):
        print("Can fly")

class Bird(Animal, Flyable):
    pass

# Derived class usecase
bird_instance = Bird()
bird_instance.speak()
bird_instance.fly()
```

Exemple d'Héritage hiérarchique en Python

```python
# Hierarchical inheritance
class Feline:
    pass

class Cat(Feline):
    def meow(self):
        print("Can Meoow!")

class Tiger(Feline):
    def roar(self):
        print("Can Rooar!")

# Derived class usecase
cat_instance = Cat()
tiger_instance = Tiger()

cat_instance.meow()
tiger_instance.roar()
```

Exemple d'Héritage Multi-niveaux en Python

```python
# Multi-level Inheritance
class Animal:
    def speak(self):
        print("Animal speaks")

class Cat(Animal):
    def purr(self):
        print("Cat purrs")

class Tiger(Cat):
    def roar(self):
        print("Tiger roars")

# Derived class usecase
tiger_instance = Tiger()
tiger_instance.speak()
tiger_instance.purr()
tiger_instance.roar()
```

Exemple d'Héritage Hybride en Python

```python
# Hybrid Inheritance
class Animal:
    def speak(self):
        print("Animal speaks")

class Bird(Animal):
    def fly(self):
        print("Bird flies")

class Fish(Animal):
    def swim(self):
        print("Fish swims")

class Amphibian(Bird, Fish):
    def jump(self):
        print("Amphibian jumps")

# Derived class use case
amphibian_instance = Amphibian()
amphibian_instance.speak()  # Output: Animal speaks
amphibian_instance.fly()    # Output: Bird flies
amphibian_instance.swim()   # Output: Fish swims
amphibian_instance.jump()   # Output: Amphibian jumps
```

Problème du diamant

- En programmation orientée objet, le problème du diamant survient lorsqu'une classe hérite de deux classes qui elles-mêmes héritent d'une même classe.
- Cela crée une ambiguïté lors de l'appel de méthodes ou d'attributs qui sont définis dans la classe parente commune.
- Python gère le problème du diamant en utilisant l'ordre de résolution des attributs mro (Method Resolution Order), qui spécifie l'ordre dans lequel les classes sont explorées pour trouver les attributs.

Problème du diamant - Suite

- L'ordre de résolution des attributs est déterminé par l'algorithme C3 de Python, qui garantit la cohérence et la prévisibilité dans la résolution des méthodes et des attributs.
- Malgré cela, il est recommandé de concevoir des hiérarchies de classes de manière à éviter le problème du diamant, lorsque cela est possible, en utilisant des modèles de conception appropriés.

Problème du diamant - Exemple

```python
class A:
    def my_method(self):
        print("my_method called from class A")

class B(A):
    def my_method(self):
        print("my_method called from class B")
        A.my_method(self)

class C(A):
    def my_method(self):
        print("my_method called from class C")
        A.my_method(self)

class D(C, B):
    def my_method(self):
        print("my_method called from class D")
        C.my_method(self)
        B.my_method(self)

instance_of_class_D = D()
instance_of_class_D.my_method() # Output: DCABA
print(D.mro())                  # Output: DCBAo
```

Problème du diamant - Solution

```python
class A:
    def my_method(self):
        print("my_method called from class A")

class B(A):
    def my_method(self):
        print("my_method called from class B")
        super().my_method()

class C(A):
    def my_method(self):
        print("my_method called from class C")
        super().my_method()

class D(C, B):
    def my_method(self):
        print("my_method called from class D")
        super().my_method()

instance_of_class_D = D()
instance_of_class_D.my_method() # Output: DCBA
print(D.mro())                  # Output: DCBAo
```

Fonctions `issubclass()`, `super()` et méthode `mro()`

- La fonction `issubclass()`, le constructeur `super()` et la méthode `mro()` sont des outils puissants de la programmation orientée objet en Python.
- Elles permettent de gérer l'héritage des classes et d'accéder à la hiérarchie d'héritage.
- La fonction `issubclass()` permet de vérifier si une classe est une sous-classe d'une autre.
- Le constructeur `super()` permet d'accéder aux méthodes et attributs des classes parentes.
- La méthode `mro()` (Method Resolution Order) mro retourne l'ordre de résolution des méthodes pour une classe.

Exemples d'utilisation

```python
class Animal:
    def speak(self):
        return "Animal speaks"

class Dog(Animal):
    def speak(self):
        return "Dog barks"

class Labrador(Dog):
    def speak(self):
        return "Labrador barks"

# Usage of function issubclass()
print(issubclass(Dog, Animal))            # Output: True

# Usage of the super() constructor
labrador = Labrador()
print(super(Labrador, labrador).speak())  # Output: Dog barks
print(super(Dog, labrador).speak())       # Output: Animal speaks

# Usage of the mro() method
print(Labrador.mro())
# Output: [<class '__main__.Labrador'>, <class '__main__.Dog'>, <class '__main__.Animal'>, <class 'object'>]
```

Le constructeur super() en Python

- En Python, le constructeur `super()` est utilisé pour accéder aux méthodes et aux attributs de la classe parente dans les sous-classes.
- Il est souvent utilisé lorsque vous redéfinissez une méthode dans une sous-classe et que vous souhaitez toujours appeler la version de la méthode de la classe parente.

Accès aux attributs de la classe parente

- L'accès aux attributs de la classe parente se fait en utilisant `super()` dans le constructeur de la sous-classe.
- Cela permet d'initialiser les attributs hérités de la classe parente.
- L'on peut aussi l'utiliser afin d'accéder aux méthodes de la classe parente.
- Exemple:

```
1  class Parent:
2      def __init__(self, x):
3          self.x = x
4
5  class Child(Parent):
6      def __init__(self, x, y):
7          super().__init__(x)  # Call of the parent class constructor
8          self.y = y
9
10 child = Child(10, 20)
11 print(child.x)  # Output: 10
```

Accès aux méthodes de la classe parente

- Pour accéder à une méthode de la classe parente dans une sous-classe, utilisez `super()` suivi du nom de la méthode et des arguments appropriés.

```
class Parent:
    def method(self):
        print("Method from Parent class")

class Child(Parent):
    def method(self):
        super().method()                # Call of the parent class method
        print("Method from Child class")

child = Child()
child.method()                          # Call of the child class method
# Output: "Method from Parent class"
#         "Method from Child class"
super(Child, child).method()            # Call of the parent class method
# Output: "Method from Parent class"
```

Super en héritage multiple

- Lorsqu'il y a héritage multiple, `super()` suit l'ordre de la résolution des méthodes (MRO) pour appeler les méthodes des classes parentes dans un ordre cohérent.

Utilisation dans l'abstraction et le polymorphisme

- `super()` est souvent utilisé dans la mise en œuvre de l'abstraction et du polymorphisme pour créer des hiérarchies de classes avec des comportements cohérents et des méthodes redéfinies selon les besoins.

Méthode de Résolution des Ordres (MRO) en Python

- La méthode de résolution des ordres (MRO) est utilisée pour déterminer l'ordre dans lequel les méthodes sont recherchées et appelées dans une hiérarchie de classes.
- La MRO est calculée automatiquement pour chaque classe en utilisant l'algorithme C3.
- La MRO peut être accédée en utilisant la méthode `mro()` ou l'attribut spécial `__mro__`.
- La méthode `mro()` renvoie une liste ordonnée des classes dans l'ordre de recherche des méthodes, tandis que l'attribut `__mro__` renvoie un tuple.

Exemple d'utilisation de la MRO

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(C, B):
    pass

print(D.mro())
# Output:
# [<class '__main__.D'>, <class '__main__.C'>, <class '__main__.B'>, <class '__main__.A'>, <class 'object'>]
print(D.__mro__)
# Output:
# (<class '__main__.D'>, <class '__main__.C'>, <class '__main__.B'>, <class '__main__.A'>, <class 'object'>)
```

Principe de l'abstraction

Notion d'abstraction

- L'**abstraction** est un concept fondamental de la programmation orientée objet (POO).
- Elle consiste à masquer les détails d'implémentation d'un objet et à se concentrer uniquement sur son comportement ou ses caractéristiques essentielles.
- En Python, l'abstraction est souvent réalisée en définissant des classes abstraites et en utilisant des méthodes abstraites pour définir une interface commune.

Classes abstraites en Python

- En Python, une classe abstraite est une classe qui ne peut pas être instanciée directement et qui sert de modèle pour d'autres classes.
- On utilise le module `abc` (Abstract Base Classes) pour créer des classes abstraites.
- Pour déclarer une classe abstraite, on utilise le décorateur `@abstractmethod` pour marquer les méthodes abstraites, c'est-à-dire les méthodes qui doivent être implémentées par les classes dérivées.

Classes abstraites - Exemple

```python
from abc import ABC, abstractmethod
from math import pi as PI

class Shape(ABC):
    @abstractmethod
    def area(self) -> float:
        pass

    @abstractmethod
    def perimeter(self) -> float:
        pass

    def __str__(self):
        return f"Shape : {type(self).__name__}(area={self.area():.3f}, perimeter={self.perimeter():.3f})"

class Rectangle(Shape):
    def __init__(self, length: float, height: float):
        self.length : float = length
        self.height : float = height

    def area(self) -> float:
        return self.length * self.height

    def perimeter(self) -> float:
        return 2 * (self.length + self.height)
```

Classes abstraites - Exemple - Suite

```python
class Circle(Shape):
    def __init__(self, radius: float):
        self.radius : float = radius

    def area(self) -> float:
        return PI * self.radius ** 2

    def perimeter(self) -> float:
        return 2 * PI * self.radius

# Example usage
rectangle : Rectangle = Rectangle(5, 4)
circle : Circle = Circle(PI)

print(rectangle)  # Output: Shape : Rectangle(area=20.000, perimeter=18.000)
print(circle)     # Output: Shape : Circle(area=31.006, perimeter=19.739)
```

Notion d'interface

- En programmation orientée objet, une interface est un ensemble de méthodes qu'une classe doit implémenter.
- Les interfaces définissent un contrat ou une spécification pour les classes qui les implémentent.
- En Python, il n'y a pas de mot-clé pour définir explicitement une interface, mais on utilise souvent des classes abstraites pour ce but.
- Les classes qui implémentent une interface doivent fournir des définitions pour toutes les méthodes déclarées dans cette interface.

Utilisation des classes abstraites comme interfaces

- En Python, on utilise souvent des classes abstraites pour définir des interfaces.
- On utilise le module `abc` (Abstract Base Classes) pour créer des interfaces.
- Les classes qui implémentent une interface héritent de la classe abstraite correspondante et fournissent des implémentations pour toutes les méthodes abstraites définies dans cette classe.

Interfaces - Exemple

```python
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def process_payment(self, amount):
        pass

    @abstractmethod
    def refund_payment(self, transaction_id):
        pass

class PayPal(PaymentGateway):
    def process_payment(self, amount):
        print("Processing payment of amount", amount, "via PayPal")

    def refund_payment(self, transaction_id):
        print("Refunding payment for transaction ID", transaction_id, "via PayPal")
```

Interfaces - Exemple - Suite

```python
class Stripe(PaymentGateway):
    def process_payment(self, amount):
        print("Processing payment of amount", amount, "via Stripe")

    def refund_payment(self, transaction_id):
        print("Refunding payment for transaction ID", transaction_id, "via Stripe")

# Example usage
paypal = PayPal()
stripe = Stripe()

paypal.process_payment(100)    # Output: Processing payment of amount 100 via PayPal
stripe.process_payment(200)    # Output: Processing payment of amount 200 via Stripe

paypal.refund_payment("123456")    # Output: Refunding payment for transaction ID 123456 via PayPal
stripe.refund_payment("789012")    # Output: Refunding payment for transaction ID 789012 via Stripe
```

Classes abstraites vs Interfaces

Table: Différences et similitudes entre les classes abstraites et les interfaces

Classes abstraites	Interfaces
Peut contenir des méthodes concrètes et abstraites	Ne peut contenir que des méthodes abstraites
Peut avoir des méthodes normales	Ne peut pas avoir de méthodes normales
Les sous-classes peuvent fournir des implémentations pour certaines méthodes et laisser d'autres méthodes abstraites	Les classes qui implémentent une interface doivent fournir des implémentations pour toutes les méthodes déclarées dans cette interface
Peut avoir des variables de classe et d'instance	Ne peut pas avoir de variables de classe ou d'instance
Héritage multiple possible	Héritage multiple possible
Utilisation du module `abc`	Utilisation du module `abc`

Remarques sur les classes abstraites

1. Une classe dérivée d'une classe abstraite peut redéfinir que certaines méthodes abstraites, dans ce cas, elle devient elle-même abstraite, et par conséquent l'on ne peut plus l'instancier.
2. Une classe peut-elle dériver de plusieurs classes abstraites ?
   Oui, Python permet l'héritage multiple, donc une classe peut hériter de plusieurs classes abstraites.

Remarques sur les interfaces

3. Une classe abstraite peut-elle dériver d'une autre classe abstraite ?
   Oui, une classe abstraite peut hériter d'une autre classe abstraite en Python.
4. Une classe peut-elle dériver de plusieurs interfaces ?
   Oui, Python ne supporte pas nativement les interfaces, mais une classe peut implémenter plusieurs classes contenant uniquement des méthodes abstraites, ce qui est souvent utilisé comme des interfaces.
   Oui, en Python, une interface (une classe ne contenant que des méthodes abstraites) est représentée par une classe abstraite, de ce fait, une interface peut hériter d'une autre interface ou d'une classe abstraite.

Remarques sur les interfaces

5. Une classe peut-elle dériver d'une ou plusieurs classes abstraites ET interfaces ?
   Oui, une classe peut hériter d'une ou plusieurs classes abstraites et implémenter plusieurs interfaces.
6. Une interface peut-elle hériter d'une interface ou d'une classe abstraite ?
   Oui, en Python, une interface (une classe ne contenant que des méthodes abstraites) est représentée par une classe abstraite, de ce fait, une interface peut hériter d'une autre interface ou d'une classe abstraite.

Principe de polymorphisme

Polymorphisme

- Le polymorphisme est un concept clé de la programmation orientée objet (POO).
- Il fait référence à la capacité d'un objet à prendre plusieurs formes ou à se comporter différemment selon le contexte.
- Les langages statiquement typés comme **C++** ou **Java** distinguent :
  - **Polymorphisme de compile-time (résolu à la compilation)**
  - **Polymorphisme de runtime (résolu à l'exécution)**
- En Python, bien que dynamique, on peut simuler certains comportements de polymorphisme compile-time.

Types de polymorphisme en général

- **Compile-time polymorphism :** résolution du polymorphisme lors de la compilation.
  - Surcharge de méthode (function/method overloading)
  - Surcharge d'opérateur (operator overloading)
  - Templates / Generics
  - Arguments par défaut (fonction avec plusieurs signatures implicites)
- **Runtime polymorphism :** résolution du polymorphisme lors de l'exécution.
  - Redéfinition de méthode (method overriding)
  - Implémentation de méthodes abstraites via ABC
  - Duck typing (basé sur le comportement)

Résumé des polymorphismes et leurs sous-types

Méthode	Résolution	Commentaire
Function overloading	Compilation	Plusieurs fonctions avec le même nom mais signatures différentes
Operator overloading	Compilation	Polymorphisme opérateur selon les types des opérandes
Templates / Generics	Compilation	Polymorphisme paramétrique selon le type des paramètres
Arguments par défaut	Compilation	Polymorphisme de type “fonction avec plusieurs signatures implicites”
Virtual functions	Exécution	Polymorphisme basé sur l’héritage et le type réel de l’objet

Compile-time polymorphism - généralités

- Le polymorphisme au moment de la compilation se résout avant l'exécution du programme.
- Dans les langages statiquement typés comme C++ et Java, ce type de polymorphisme est naturel.
- En Python, il est simulé via des techniques comme :
  - `*args`, `**kwargs`
  - Valeurs par défaut
  - Type hints / annotations
  - Surcharge d'opérateur avec les méthodes magiques (__add__, __mul__, etc.)

Compile-time polymorphism - sous-types

- **Method overloading** : plusieurs méthodes avec le même nom mais des signatures différentes.
  - Python : valeurs par défaut, `*args` et `**kwargs`
  - Exemple : method_overloading_with_args.py
- **Operator overloading** : surcharge des opérateurs standard pour des objets personnalisés
  - Python : méthodes magiques __add__, __sub__, __mul__, __rmul__, etc.
  - Exemple : operator_overloading.py
- **Generics / Templates** : paramétrage par type
  - Python : `TypeVar`, `Generic`, Protocols
  - Exemple : generics_overloading.py

Runtime polymorphism - généralités

- Le polymorphisme au moment de l'exécution est basé sur le type réel des objets.
- Très utilisé en Python grâce au duck typing et à la nature dynamique.
- Principales techniques :
  - Redéfinition de méthode (overriding)
  - Implémentation de méthodes abstraites via ABC
  - Duck typing

Runtime polymorphism - sous-types

- **Method overriding** : redéfinition d'une méthode dans une sous-classe
  - Exemple Python : overriding.py
- **Abstract base classes (ABC)** : classes abstraites et méthodes abstraites
  - Exemple Python : shape.py
- **Duck typing** : polymorphisme basé sur le comportement plutôt que le type
  - Exemple Python : main.py

Principe de l'encapsulation

Encapsulation — Définition

- L'encapsulation est un **pilier fondamental** de la programmation orientée objet.
- Elle consiste à regrouper les **données** (attributs) et les **comportements** (méthodes) d'un objet au sein d'une même unité : la **classe**.
- Ce principe permet de **protéger les données internes** en limitant leur accès direct depuis l'extérieur de la classe.

Les autres principes au service de l'encapsulation

- Les trois autres principes de la POO - **abstraction**, **héritage** et **polymorphisme** - servent un objectif commun : **renforcer l'encapsulation**.

  → L'**abstraction** permet de **masquer les détails d'implémentation** pour ne révéler que les informations essentielles.

  → L'**héritage** facilite la **réutilisation du code** et la spécialisation des comportements sans violer l'encapsulation.

  → Le **polymorphisme** rend possible la manipulation d'objets différents via une **interface commune**, tout en respectant leur indépendance interne.

Les classes comme capsules

- Chaque **classe** agit comme une **capsule autonome** :

  $\rightarrow$ Elle protège ses données internes,

  $\rightarrow$ Définit ses propres règles de fonctionnement,

  $\rightarrow$ Et interagit avec d'autres classes uniquement *via* des **interfaces bien définies**.

- Les classes encapsulées peuvent ainsi **coopérer ou rester indépendantes**, selon la logique du système.

Conclusion
Une vision modulaire du système

- L'encapsulation permet de concevoir un système logiciel comme un ensemble de **modules cohérents et autonomes**.
- Chacun de ces modules peut **interagir ou évoluer indépendamment**, favorisant la maintenance, la réutilisation et la robustesse du code.
- Ainsi, l'encapsulation est la **finalité naturelle** de tous les autres principes de la POO : elle en concrétise la philosophie.

** Merci de votre attention ! **